UNIVERSITY
OF OSLO

Master's Thesis

ON NEUROMORPHIC COMPUTING

With Spiking Neural Networks

Brage Wiseth

Departement of Informatics
Faculty of Mathematics and
Natural Sciences

Philipp Häfliger - Supervisor
Yngve Hafting - Co-Supervisor

2026-05-15

ABSTRACT

The development of modern deep learning has achieved unprecedented performance across various domains, yet it remains fundamentally bottlenecked by the energy and memory inefficiencies of the von Neumann architecture. To address these limitations, this thesis investigates neuromorphic computing with Spiking Neural Networks (SNNs) as a biologically plausible, highly energy-efficient alternative. By shifting from synchronous, continuous-value matrix multiplications to asynchronous, event-driven sparse computations, neuromorphic systems emulate the physical principles of the biological brain.

This work explores the implementation of these principles in simulation on standard CPU/GPU hardware. Two primary methodologies are developed and evaluated on visual classification tasks: (1) an inference-optimized SNN that translates weights from a conventionally trained Fully Connected Feed-Forward Network (FCFN) using Time-To-First-Spike (TTFS) temporal encoding, and (2) an unsupervised, biologically inspired learning simulator incorporating synaptic plasticity. Results indicate that momentum-based zero-shot weight transfer successfully recovers near-baseline accuracy (94.5%), while native unsupervised STDP autonomously clusters geometric features (44.3%), successfully proving the viability of local learning while exposing key representational constraints in single-layer WTA topologies. Ultimately, this work highlights the potential of neuromorphic algorithms to drastically reduce the computational footprint of artificial intelligence.

ACKNOWLEDGEMENTS & DECLARATIONS

First and foremost, I would like to thank my supervisors, Philipp Häfliger and Yngve Hafting, for their invaluable mentorship throughout this project. Furthermore, I wish to extend my gratitude to the brilliant and supportive community within the ROBIN and NANO research groups at the University of Oslo's Department of Informatics. The collaborative environment, shared insights, and inspiring discussions have been fundamental to the realization of this thesis.

The repository containing all source code including simulation software, source code for this document and its figures can be found at https://github.com/nammenam/neuromorphics.git

DECLARATION OF THE USE OF GENERATIVE ARTIFICIAL INTELLIGENCE

In this scientific work, generative Artificial Inteligence (AI) has been used. All data and personal information have been processed in accordance with the University of Oslo's regulations, and I, as the author of the document, take full responsibility for its content, claims, and references. An overview of the use of generative AI is provided below.

The service Gemini, developed by Google [1], has been used to improve the content of the thesis. Sections of text like a subsection, paragraph or source code for figures was given to the model along with prompts to make the language free of errors and more professional. The text underwent multiple iterations using refined prompts to ensure structural coherence and academic tone. In the case for figures the model was used to speed up the development of the figures with the model generating numbers for positioning elements. The final result was cut out, fact-checked, and partly rewritten by the author.

CONTENTS

GLOSSARY

action potential: A rapid change in voltage across a neuron's cell membrane that propagates down the axon to the synapses.

***AER* – Adress Event Representation**

***AI* – Artificial Inteligence**

***ALU* – Arithmetic Logic Unit**

***ANN* – Artificial Neural Network**

***CMOS* – Complementary Metal-Oxide-Semiconductor**

***CNN* – Convolutional Neural Network**

***FCFN* – Fully Connected Feed-Forward Network**

***FPGA* – Field Programmable Gate Array**

***GLIF* – Generalized Leaky Integrate and Fire**

***GPT* – Generative Pre-trained Transformer**

***IF* – Integrate and Fire**

***LIF* – Leaky Integrate and Fire**

***LLM* – Large Language Model**

***LSTM* – Long Short-Term Memory**

***LTD* – Long-Term Depression**

***LTP* – Long-Term Potentiation**

***MAC* – Multiply And Accumulate**

***MLP* – Multi Layer Perceptron**

***PSC* – Post Synaptic Current**

***ReLU* – Rectified Linear Unit**

***RNN* – Recurrent Neural Network**

saccade: A rapid, simultaneous movement of both eyes that abruptly shifts the point of fixation from one area of interest to another in the visual field.

***SNN* – Spiking Neural Network**

spike: An action potential conceptualized as a discrete, instantaneous event. In neuromorphic and computational models, it represents a binary signal traveling through the network.

spike train: A sequence of action potentials (spikes) occurring over time. It typically represents the temporal firing pattern of a single neuron.

***STDP* – Spike-Timing-Dependent Plasticity**

***SyOPs* – Synaptic Operations**

***TTFS* – Time-To-First-Spike**

***WTA* – Winner-Take-All**

1 • INTRODUCTION

The development of intelligent machines is a significant objective in modern science and engineering. While the concept has historical roots in philosophy and early automata, the field has transitioned from speculative theory to practical application. Currently, artificial intelligence is central to technological and economic development. Understanding the mechanisms of intelligence and reproducing them in synthetic systems offers the potential for improved analysis of biological minds and the creation of tools for applications ranging from personalized medicine to automated scientific discovery.

In recent years, great strides have been made towards this goal. Deep learning, which utilizes multilayered neural networks, has exceeded previous performance benchmarks. These systems have demonstrated high proficiency in tasks previously limited to human capability. For example, AlphaFold has addressed complex problems in protein folding [2], reinforcement learning agents have mastered the complexity of games such as Go [3], and large language models have shown capabilities in text generation that approach human fluency. Consequently, AI is increasingly viewed as a general-purpose technology that may influence societal infrastructure.

However, despite these advances, there are significant limitations to the current approach. The success of modern deep learning relies heavily on scaling, which involves increasing data volume and computational power. This strategy is approaching physical and economic boundaries. Training state-of-the-art models consumes substantial energy and results in a large carbon footprint [4]. Although specialized hardware allows for more efficient computations, the underlying architecture and algorithms impose an intrinsic limit on scalability independent of the underlying hardware. Furthermore, the requirement for massive datasets presents challenges in sourcing and curation. Additionally, evidence suggests that this scaling approach yields diminishing returns. Models often function as statistical correlation engines; they lack common-sense reasoning, struggle with out-of-distribution generalization, and are prone to brittle failure modes [5].

These limitations are evident when comparing artificial systems to biological intelligence. The human brain demonstrates that high-level intelligence is possible without massive energy consumption or dataset sizes. The brain operates on approximately 20 watts [6]. With this limited energy budget, it manages biological functions, processes real-time multi-sensory data, and performs abstract reasoning. In contrast, deep learning models require GPU clusters with significantly higher power requirements to match a fraction of these capabilities. There is also a discrepancy in learning efficiency. Deep learning models are sample-inefficient, often requiring vast numbers of examples to learn a representation. Biological systems, however, are capable of one-shot or few-shot learning and can acquire new information without

catastrophic forgetting. This suggests the inefficiency of current AI is a paradigmatic issue rather than just an engineering problem.

The proposed direction for addressing these issues involves biological inspiration in both hardware and algorithm design, specifically neuromorphic computing. This field attempts to engineer computer architectures that mimic the biological structure of the nervous system. Unlike traditional AI, which runs as software on general-purpose hardware, neuromorphic engineering aims to align the algorithm with the physical substrate. It moves away from clock-driven processing toward asynchronous, event-driven systems. In this paradigm, information is encoded as sparse, discrete events or spikes. Similar to biological neurons, a neuromorphic processor consumes minimal energy when inactive, processing information only when triggered. This approach is being pursued by both industrial groups, such as Intel's Loihi [7], and academic projects like SpiNNaker [8]. These systems represent a shift from calculation-based machines to those capable of real-time adaptation.

Although neuromorphic systems achieve optimal performance on co-designed platforms—where the algorithm is embedded directly into the hardware—there is significant value in executing neuromorphic algorithms on traditional von Neumann architectures. In this thesis, we explore biologically inspired algorithms deployed on traditional CPU and GPU hardware. We examine how event-driven, biologically plausible computation can address limitations in scalability, data efficiency, and energy consumption, even when simulated on standard processors. We present approaches for efficient information coding and learning algorithms inspired by neural mechanisms. Concretely, this thesis aims to:

Sparse Efficient Computing: Investigate whether biologically inspired, event-driven algorithms reduce the computational footprint of visual classification when simulated on standard hardware.

Neuron Model Evaluation: Identify which temporal integration dynamics are compatible with TTFS rank-order decoding across systematic threshold regimes.

Inference Via Weight Transfer: Quantify the accuracy penalty of zero-shot Artificial Neural Network (ANN)-to-SNN weight transfer under TTFS encoding, isolating the cost of transitioning to event-driven integration.

Native Unsupervised Learning: Determine if local Spike-Timing-Dependent Plasticity (STDP) can autonomously extract meaningful geometric features from visual input without global error signals.

The succeeding chapters establish the historical and theoretical foundations of both neuroscience and artificial neural networks. We then synthesize relevant biological and machine learning concepts to inform our methodology, before detailing the neuromorphic implementation and evaluating its benchmark performance.

2 • HISTORY & DEVELOPMENTS

Historically, the understanding of neural tissue was dominated by the reticular theory, which claimed that the brain consisted of a continuous, fused network of nerve fibers. This paradigm was fundamentally challenged by the work of Santiago Ramón y Cajal. Through the application of novel staining techniques, Cajal established the neuron doctrine, demonstrating that the nervous system is composed of discrete, individual cells [9]. Building on these findings, Heinrich Wilhelm Gottfried von Waldeyer-Hartz proposed the neuron doctrine and coined the term neurons to describe these discrete cells [10]. Subsequent analysis using electron microscopy has provided irrefutable validation of this discrete cellular structure.

The conceptualization of the brain as a collection of discrete units facilitated the development of mathematical models describing neural function. In 1943, Warren McCulloch and Walter Pitts published *A Logical Calculus of the Ideas Immanent in Nervous Activity* [11], introducing the first formal model of the neuron.

The McCulloch-Pitts (M-P) neuron abstracted biological complexity into a binary decision device governed by the following logic:

Inputs: The neuron receives multiple binary inputs, weighted as either excitatory or inhibitory.

Summation: The unit calculates the linear sum of these weighted inputs.

Thresholding: If the aggregate sum exceeds a fixed threshold, the neuron outputs a 1 (firing); otherwise, it outputs a 0 (silence).

McCulloch and Pitts demonstrated that networks of these units could theoretically compute any logical operation (AND, OR, NOT) [11]. This abstraction established the foundational link between biological processes and digital logic, suggesting that neural function could be replicated in electronic hardware. Consequently, the M-P neuron serves as the common ancestor for both computational neuroscience and artificial intelligence.

However, despite its theoretical utility, the original M-P model presented significant functional limitations. The connectivity was static, requiring circuits to be manually designed rather than learned. Furthermore, the restriction to binary weights precluded the modeling of graded signal intensity, preventing the system from capturing the nuance of real-world sensory input.

In 1949, Donald Hebb addressed the critical issue of plasticity in his work *The Organization of Behavior*. He proposed a theoretical mechanism for synaptic modification, now known as Hebbian learning, which provided a biological basis for how neural networks could adapt over time. Hebb postulated:

> “Let us assume that the persistence or repetition of a reverberatory activity (or trace) tends to induce lasting cellular changes that add to its stability. … When an axon of cell A is near enough to excite a cell B and repeatedly or persistently takes part in firing it, some growth process or metabolic change takes place in one or both cells such that A’s efficiency, as one of the cells firing B, is increased” [12].

This principle is colloquially summarized as “neurons that fire together, wire together” [13]. Crucially, this describes a local and decentralized learning rule; a synapse does not require a global error signal or external supervision to adjust. It requires only the correlation between the pre-synaptic input and the post-synaptic output. The convergence of the M-P architectural model and the Hebbian plasticity framework established the prerequisite conditions for the development of modern neural networks.

2.1 • THE PERCEPTRON

In 1957, Frank Rosenblatt advanced these theoretical concepts by engineering the perceptron [14]. The mark I perceptron was a hardware implementation of the neural model, distinguished by a crucial innovation: a weight-adjustment mechanism based on Hebbian principles. Rosenblatt introduced the perceptron learning rule, an iterative algorithm capable of minimizing error automatically. The system processed an input pattern (e.g., a pixelated character) and produced a binary classification. When the output deviated from the target, the algorithm adjusted the weights proportional to the error: strengthening connections that should have contributed to a correct firing and weakening those that led to false positives.

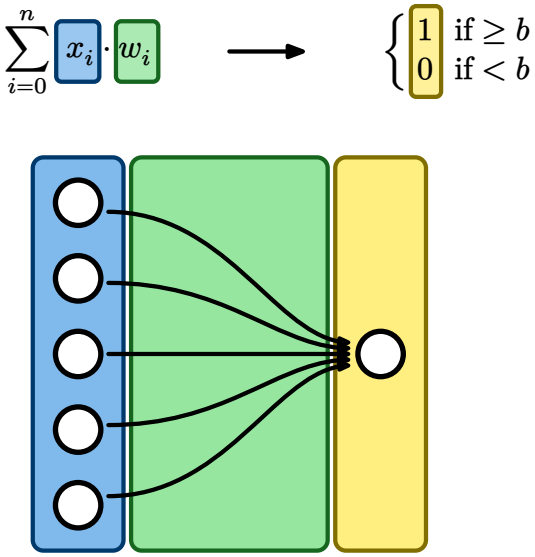

Figure 1: The perceptron model. Inputs x_i are multiplied by weights w_i and summed. If the linear combination $\sum x_i w_i$ exceeds the bias b, the neuron activates.

Consequently, the perceptron was capable of converging on a solution for any problem where the data was linearly separable. This success generated significant enthusiasm, with contemporary reports suggesting that such machines would soon mimic human consciousness [15].

These expectations were abruptly tempered by theoretical limitations. In 1969, Marvin Minsky and Seymour Papert published *Perceptrons*, a rigorous mathematical analysis of the architecture [16]. They demonstrated that a single-layer perceptron is fundamentally a linear classifier. While capable of learning operations like AND or OR, it is mathematically incapable of solving the XOR (Exclusive OR) problem. In the XOR case, the classes cannot be separated by a single hyperplane. This proof highlighted a severe boundary on the utility of single-layer networks for complex, non-linear tasks.

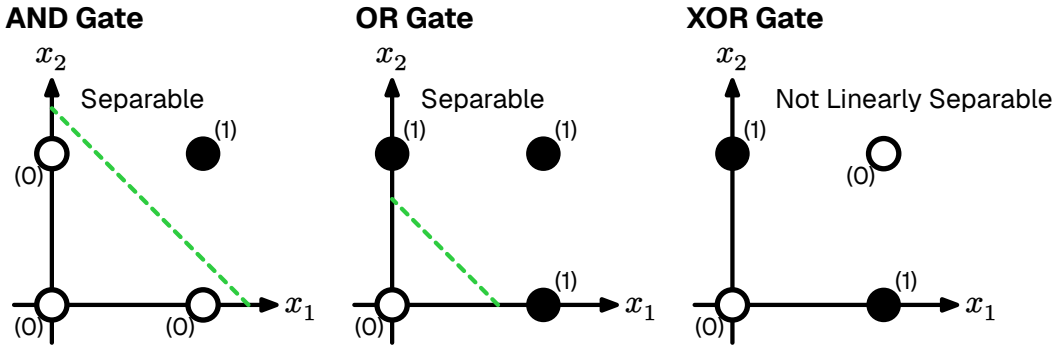

Figure 2: The XOR problem. Unlike AND/OR, the data points for XOR cannot be separated by a single linear boundary.

The publication of *Perceptrons* coincided with a significant reduction in neural network research funding, a period retrospectively termed the First AI Winter. It is worth noting that Minsky and Papert acknowledged that a Multi Layer Perceptron (MLP), a network stacking multiple layers of neurons, could theoretically solve the XOR problem by creating complex, non-linear decision boundaries.

However, a critical algorithmic gap remained: the credit assignment problem. While researchers knew that hidden layers could represent complex features, there was no known method to propagate error signals back through the layers to adjust the weights of hidden neurons effectively. Rosenblatt's rule was mathematically valid only for the output layer. The field remained stagnant until a method for training multi-layer networks could be formalized.

2.2 • DEEP LEARNING

The critique presented by Minsky and Papert precipitated a contraction in funding; despite this, theoretical inquiry persisted. It was widely hypothesized that the limitations of the single perceptron could be overcome by a MLP. By organizing neurons (single perceptrons) into hierarchical layers, the network could theoreti-

cally perform successive non-linear transformations on the input space, enabling the formation of complex decision boundaries. The primary impediment was not the architecture itself, but the absence of a viable learning algorithm.

In a single-layer perceptron, error attribution is immediate: if the output deviates from the target, the error is directly derived from the weights of the output layer. However, in a multi-layer architecture, quantifying the contribution of a specific neuron within the hidden layers to the final output error presents a significant challenge. This is formally known as the credit assignment problem [17], and it remained the central theoretical obstacle for over a decade.

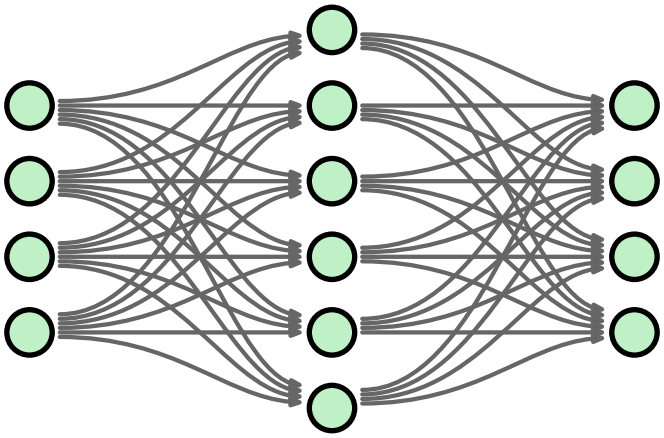

Figure 3: A Multi Layer Perceptron (MLP). By inserting hidden layers between input and output, the network can approximate non-linear functions such as XOR. The historical challenge lay in deriving a method to train these intermediate layers.

The solution to this theoretical impasse was popularized in 1986 by Rumelhart, Hinton, and Williams in their seminal paper *Learning representations by back-propagating errors* [18]. They demonstrated that the chain rule of calculus could be applied recursively to propagate the error signal from the output layer backwards through the hidden layers. This algorithm, known as backpropagation, allowed the network to calculate the gradient of the loss function with respect to every weight in the system. Effectively, it provided a mathematical method to tell each hidden neuron exactly how much it contributed to the total error, finally solving the credit assignment problem.

Unlike Hebbian plasticity, which is local and biological, backpropagation relies on global error signals and precise backward data flow—mechanisms effectively absent in organic tissue. Consequently, the field of ANN effectively decoupled from neuroscience. It transitioned into a branch of engineering and applied mathematics, prioritizing statistical optimization over biological realism. Paradoxically, it was this abandonment of biological fidelity that enabled the rapid scaling and performance breakthroughs that followed.

2.2.1 • ACHIEVEMENTS

With the training mechanism solved, the field exploded. The combination of back-propagation, massive datasets, and GPU hardware led to a rapid diversification of neural architectures, each solving domains previously thought impossible for computers.

The revolution began in earnest with computer vision. Convolutional Neural Networks (CNNs), such as AlexNet (2012) [19] and later ResNet [20], introduced the idea of learning hierarchical features—detecting edges, then shapes, then objects—much like the human visual cortex. This allowed machines to classify images with superhuman accuracy.

Soon after, the focus shifted to sequence data. Recurrent Neural Networks (RNNs) and Long Short-Term Memory (LSTM) architectures gave machines a short-term memory, enabling breakthroughs in speech recognition and machine translation. However, the true paradigm shift occurred with the introduction of the Transformer architecture in 2017 [21]. By utilizing an attention mechanism to parallelize the processing of language, Transformers allowed for the training of massive Large Language Models (LLMs) like the Generative Pre-trained Transformer (GPT).

These techniques have even transcended media generation. Deep learning has solved fundamental scientific problems; notably, DeepMind's AlphaFold utilized these architectures to predict the 3D structure of proteins from their amino acid sequences, a 50-year-old grand challenge in biology [2]

2.2.2 • SHORTCOMINGS

Deep learning's reliance on computational scaling masks fundamental inefficiencies in both its hardware implementation and underlying algorithms. By simulating biological concepts on digital architectures not designed for them, the current paradigm is approaching physical and economic limits.

A primary limitation is the von Neumann architecture, which physically separates processing units from memory. Deep neural networks, defined by massive matrices of synaptic weights, necessitate constant data transfer. For every inference step, billions of parameters must be fetched from off-chip DRAM, processed, and written back. This creates a severe memory bottleneck where system performance is bounded by bandwidth rather than processing speed [22].

Consequently, the energy cost of moving data significantly exceeds the cost of computation itself. Retrieving a single byte from DRAM consumes approximately three orders of magnitude more energy than performing a floating-point operation [23]. Compounding this hardware friction, the dense matrix multiplications required for training scale quadratically with network size, making the pursuit of trillion-parameter models increasingly unsustainable.

Furthermore, the optimization algorithms driving this scale are fundamentally incompatible with physical biological systems. backpropagation, while mathematically elegant, relies on a global error signal and suffers from the weight transport problem—the requirement that the backward pass utilizes the exact same synaptic weights as the forward pass. In organic tissue, synapses are unidirectional, and there is no known mechanism for a neuron to access the exact weight of a downstream synapse to calculate a gradient.

While a detailed technical analysis of these inefficiencies is presented in Section 4, the central issue is clear: modern AI prioritizes statistical optimization over physical realism. Overcoming the limitations of the von Neumann bottleneck and backpropagation requires a paradigm shift toward architectures that inherently co-locate memory and computation.

2.3 • BIRTH OF NEUROMORPHIC

While the artificial intelligence community debated symbolic logic versus connectionism during the AI Winter, significant developments were occurring in hardware physics. In the late 1980s at Caltech, physicist Carver Mead—a pioneer of Very Large Scale Integration (VLSI) design—began to question the trajectory of digital computing.

Mead observed that while digital computers were becoming exponentially faster, they were also becoming less efficient in terms of energy per operation. He noted that using transistors as rigid, high-power switches to perform boolean logic was energetically wasteful compared to the biological systems they aimed to emulate.

In 1990, Mead published his seminal paper, *Neuromorphic Electronic Systems* [24], coining the term neuromorphic to describe hardware that mimics the biological structure of the nervous system. His thesis proposed that rather than simulating neural equations via software on digital computers, engineers should construct physical hardware that exploits the same physical laws as the biological nervous system.

The foundational insight of the field was the physical analogy between silicon physics and ion-channel physics. In standard digital electronics, transistors are operated in strong inversion, driven by high voltages to act as binary switches. Mead realized that a single transistor, operating in its subthreshold region, follows the same exponential Boltzmann statistics that govern the flow of ions through biological channels.

This realization implied that a single transistor could physically compute the nonlinear functions used by biological neurons, but with significantly higher speed and lower power consumption. Consequently, synaptic functions could be implemented by single transistors rather than complex arrangements of logic gates.

To demonstrate this concept, Mead and his doctoral student Misha Mahowald developed the *Silicon Retina* in 1991 [25]. Unlike a standard camera, which captures full frames at fixed intervals (generating redundant data), the Silicon Retina operated asynchronously. It utilized analog circuits to compute spatial and temporal derivatives directly on-chip, outputting discrete events only when local light intensity changed.

This event-driven approach solved the redundancy problem inherent in frame-based sampling. If the scene remained static, the system transmitted no data and consumed negligible energy. This demonstrated that by aligning the hardware physics with the computational task, sensory information could be processed with a fraction of the power required by conventional digital systems.

Since the inception of neuromorphic computing, neuroscience has also advanced significantly. While Mead's early work was based on the physical intuition of the transistor, modern neuromorphic engineering now incorporates a richer understanding of neuronal dynamics, synaptic plasticity, and network architecture. To advance the field, we must combine these foundational hardware insights with the principles of modern mechanistic neuroscience.

3 • BIOLOGICAL PRINCIPLES

The biological brain remains the gold standard for energy-efficient, robust, and adaptive computation. Since the establishment of the Neuron Doctrine, modern neuroscience has uncovered the specific physical mechanisms that underpin this efficiency. To engineer systems that truly rival biological performance, we must transcend the highly simplified abstractions of early cybernetics. We cannot simply mimic the brain's output; we must emulate its internal dynamics. This requires viewing the neuron not as a static summing unit, but as it functions in reality: a complex, time-dependent, and event-driven processor.

This section provides a mechanistic overview of the nervous system, translating biological observations into the computational primitives required for neuromorphic engineering. It explores the structural hierarchy of the neuron, the physics of the action potential, and the mathematical models used to capture these dynamics in silicon.

3.1 • NEURON STRUCTURE & FUNCTION

In Section 2 we established the neuron as the fundamental computational unit of the brain. While it shares standard cellular machinery like mitochondria and a nucleus with other cells, it is morphologically specialized for information transmission. A neuron consists of three functional zones:

The Input (Dendrites): A branching tree structure that collects signals from thousands of upstream neurons. This is where inputs are integrated.

The Integration Zone (Soma): The cell body where electrical potentials from the dendrites summate.

The Output (Axon): A long, cable-like structure that transmits the neuron's own signal to downstream targets.

The neuron exhibits a distinct morphological polarization that dictates the direction of information flow. The process begins at the dendritic arbor, a complex branching structure that maximizes the surface area for synaptic connectivity. These dendrites serve as the primary receptor sites, where neurotransmitters binding to post-synaptic terminals induce local conductance changes. These signals propagate passively toward the soma (cell body), the neuron's central processing unit. The soma acts as an integrator, spatially and temporally summing the incoming synaptic currents. Finally, the processed signal is transmitted via the axon, a singular, elongated projection. In many vertebrate neurons, the axon is insulated by a myelin sheath,

which facilitates saltatory conduction—a mechanism that allows high-speed signal propagation over long distances with minimal signal degradation [26].

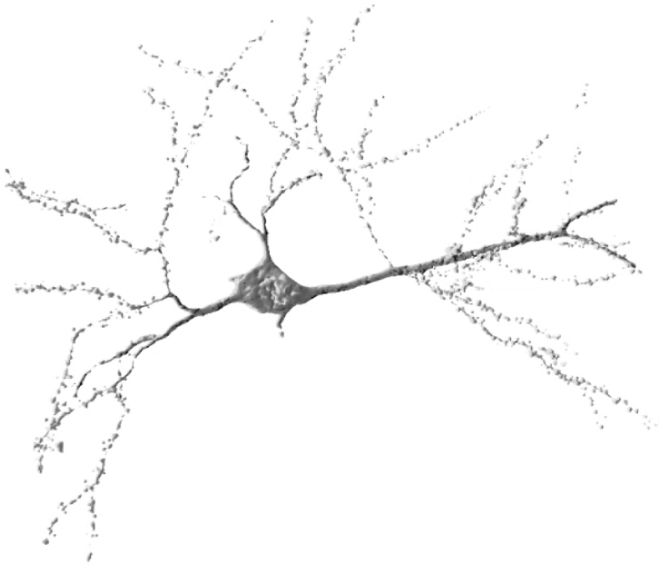

Figure 4: Structural components of a neuron, including the soma, axon, and dendrites. Image courtesy of Getz et al. [27]

Functionally, the neuron operates as an electrochemical system enclosed by a cell membrane, known as the lipid bilayer. This membrane is a thin, fatty structure that is impermeable to ions, acting as an electrical insulator. However, the fluids inside and outside the cell are conductive electrolytes. Consequently, the interaction between the insulating membrane and the conductive fluids creates a biological capacitor, capable of storing charge [28].

By actively pumping sodium (Na^+) out and potassium (K^+) in via the Na^+-K^+ ATPase pump, the cell maintains an electrochemical gradient across this capacitor, resulting in a stable resting potential of approximately -70 mV.

Computation occurs through the modulation of this voltage by competing synaptic inputs. Excitatory inputs cause ion channels to open, allowing positive ions to influx; this reduces the negative charge (depolarization) and pushes the potential toward the firing threshold. Conversely, inhibitory inputs activate channels for negative ions (like Chloride, Cl^-), driving the potential away from the threshold (hyperpolarization). The soma integrates these opposing push and pull signals. If the aggregate membrane potential surpasses a critical threshold (approximately -55 mV), the system undergoes a bifurcating phase transition. Voltage-gated sodium channels cascade open, triggering an action potential—a rapid, non-linear depolarization spike that propagates down the axon. This mechanism is governed by the all-or-nothing principle: the output is discrete and binary, effectively filtering out sub-threshold noise [26].

Immediately following a spike, the neuron enters a refractory period during which ion gradients are restored, imposing a hard limit on the maximum firing frequency and ensuring the temporal separation of events.

It is important to acknowledge that the biological brain exhibits significant cellular diversity beyond this idealized model. The nervous system contains non-neuronal cells known as glia, which provide structural support and manage energy delivery, though they are generally not considered direct participants in fast information transmission. Additionally, while the vast majority of cortical neurons communicate via uniform action potentials (spikes), certain sensory neurons utilize graded potentials, where the signal amplitude varies continuously. However, as spiking neurons represent the dominant computational paradigm for information processing in the cortex [28], this thesis focuses exclusively on the spiking model as the basis for neuromorphic emulation.

3.2 • ACTION POTENTIAL & SPIKE TRAINS

As established in the previous section, the action potential is an all-or-nothing event. It serves as the fundamental mechanism by which neurons transmit information. Crucially, the waveform of this event is stereotypical: for a given neuron, every spike exhibits a nearly identical amplitude and duration (typically 1–2 ms), independent of the input intensity that triggered it [28].

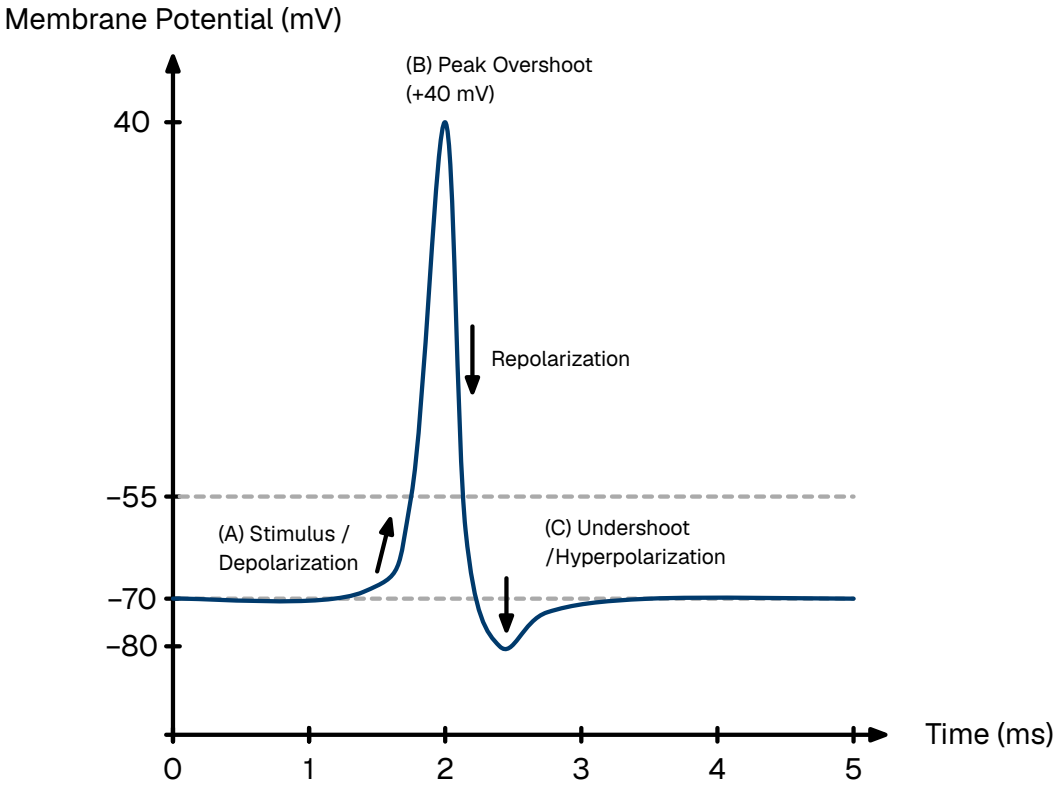

Figure 5: The phases of a typical neuronal action potential. (A) An incoming stimulus depolarizes the membrane past the threshold (−55 mV), triggering a rapid spike. (B) The membrane potential reaches a peak overshoot (+30 mV) before repolarizing. (C) A temporary undershoot (hyperpolarization) occurs before returning to the resting state (−70 mV). The neuron cannot fire during the absolute refractory period and requires a stronger stimulus to fire during the relative refractory period.

This biological invariance permits a fundamental simplification in neuromorphic modeling:

> If the spike waveform is invariant across neurons and time, the waveform itself carries no information. Consequently, the information content of the signal is encoded entirely in the precise timing of the spike.

To model this mathematically, we abstract the continuous biophysical voltage trace into a dimensionless point process. We treat the action potential not as a function of voltage over time, but as a singular event occurring at a precise instant, t_f, with negligible duration. The standard tool for this abstraction is the Dirac delta function denoted as $\delta(t)$.

The Dirac delta is a generalized distribution defined by the property that it is zero everywhere except at the origin, yet integrates to unity. This represents an idealized pulse of infinite height and zero width, containing a finite unit of effect.

$$\delta(t) = \begin{cases} \infty \text{ if } t = 0 \\ 0 \text{ if } t \neq 0 \end{cases}, \quad \int_{-\infty}^{+\infty} \delta(t)\,\mathrm{d}t = 1$$

Equation 1: The defining properties of the Dirac delta function.

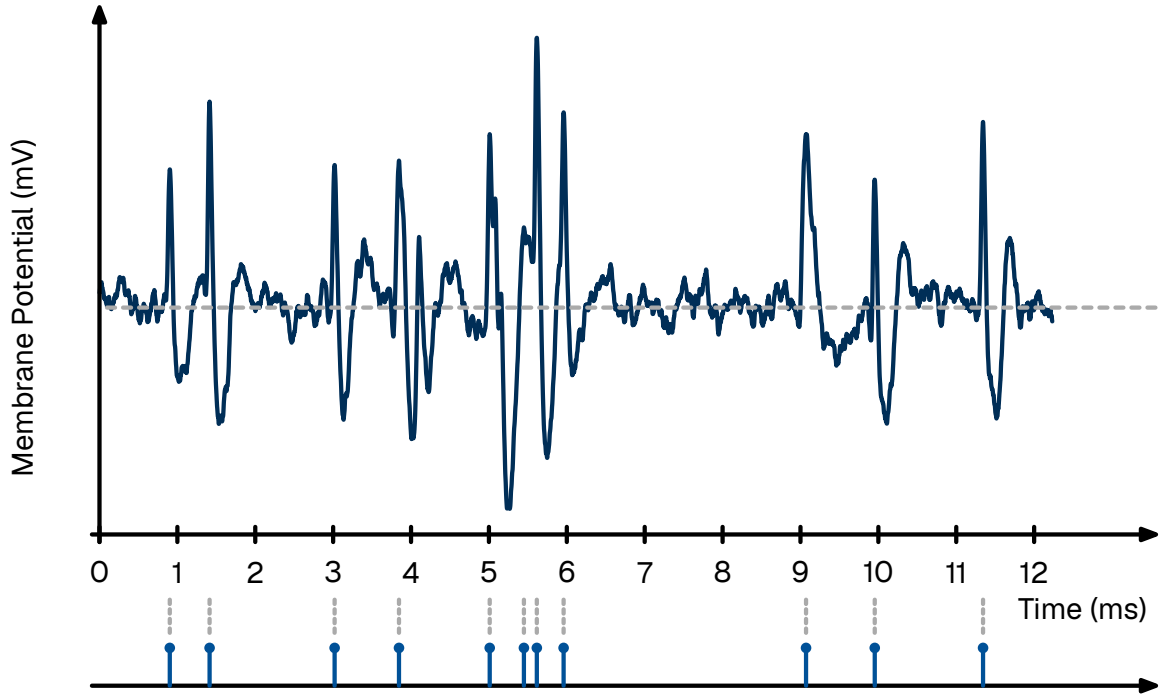

Figure 6: Transformation of continuous membrane voltage (top) into a discrete spike train (bottom). The membrane voltage trace is a recording from the L5 dorsal rootlet of a rat using a multiple electrode array [29]

Under this formalism, the output of a neuron is modeled not as a continuous signal, but as a spike train—a temporal sequence of these Dirac impulses [28]. For a neuron emitting N spikes at times $\{t_1, t_2, ..., t_N\}$, the output signal $S(t)$ is defined as:

$$S(t) = \sum_{f=1}^{N} \delta(t - t_f)$$

Equation 2: A spike train represented as a sum of Dirac delta functions.

This abstraction allows the post-synaptic effect to be modeled using linear systems theory. In neuron models that use this framework, the interaction is treated as instantaneous charge deposition: the arrival of a delta function $\delta(t - t_f)$ imparts a discrete step-change to the post-synaptic current. This mimics the rapid opening of ion channels without requiring the computational overhead of simulating the complex voltage trajectory. The shift from continuous values to discrete spike trains fundamentally alters the computational paradigm, moving from spatial representations (magnitude-based) to spatio-temporal representations (time-based).

3.3 • NEURON MODELS

In the quest to simulate the brain, there exists a fundamental trade-off between biological realism and computational efficiency. At the high end of the spectrum lie conductance-based models, most notably the Hodgkin-Huxley model. This formalism describes the neuron not as a simple computational unit, but as an electrical circuit with variable resistors representing the precise, non-linear opening and closing dynamics of specific ion channels (sodium, potassium, leak) [30].

Large-scale initiatives, such as the Blue Brain Project, utilize even more granular multi-compartment models. These simulations treat the neuron as a complex 3D structure, discretizing the dendritic arbor and axon into hundreds of segments to model how current flows through the specific morphology of the cell [31]. While invaluable for pharmacological research, these models are computationally prohibitive for large-scale neuromorphic engineering. Simulating a mere second of biological time for a small network using these equations requires supercomputing resources.

To build practical, scalable neuromorphic hardware, we must abstract these biophysical details into a phenomenological model. We seek a mathematical framework that captures the essential computational properties—integration, leakage, and thresholding—without simulating the underlying molecular physics.

3.3.1 • THE LIF MODEL

The standard approximation used in neuromorphic engineering is the Leaky Integrate and Fire (LIF) model [28]. This framework aligns perfectly with the point process abstraction established in the previous section, as it treats action potentials as instantaneous, discrete events. Its state is defined by a single scalar variable, the membrane potential $u(t)$. The sub-threshold dynamics are governed by a linear differential equation analogous to a simple RC (resistor-capacitor) circuit.

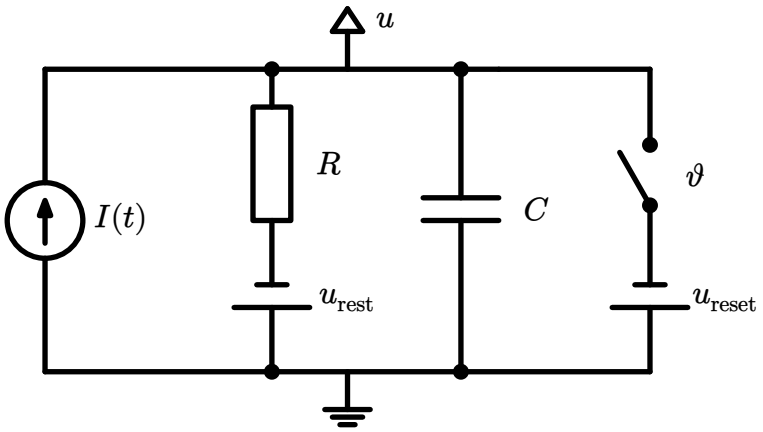

Figure 7: Electronic circuit modeling the dynamics of the LIF

$$\tau_m \frac{\mathrm{d}u}{\mathrm{d}t} = -(u - u_{\text{rest}}) + RI(t)$$

Equation 3: The LIF differential equation. The change in voltage is driven by the leak (decay to rest) and the input current. $\tau_m = RC$ is known as the time constant and dictates how fast the membrane potential drains, u_{rest} is the resting potential, R is the membrane resistance, and $I(t)$ is the input current.

Connecting this to the spike train abstraction derived in the previous section, the input current $I(t)$ is not continuous. Instead, it is a sequence of discrete events arriving from pre-synaptic neurons j with weights w_j. The weight is a mathematical abstraction of the underlying biological mechanics; it represents the efficiency of action potential transmission (a topic we will revisit in Section 3.5.1). Mathematically, this is modeled as a weighted sum of Dirac delta functions:

$$I(t) = \sum_{j=1}^{N} w_j \sum_f \delta\left(t - t_f^j\right)$$

Equation 4: Synaptic input modeled as a weighted sum of Dirac delta functions.

Because the differential equation is linear below the threshold, we can solve it analytically. The membrane potential $u(t)$ becomes a convolution of the input spike train with the system's impulse response (a decaying exponential kernel). This means the potential at any moment is simply the sum of the decaying traces of all past spikes. Crucially, this sum only considers spikes that have already occurred ($t_f^j < t$):

$$u(t) = u_{\text{rest}} + \sum_{j=1}^{N} w_j \sum_{t_f^j < t} \exp\left(-\frac{t - t_f^j}{\tau_m}\right)$$

Equation 5: The analytical solution for the membrane potential. The current voltage is the superposition of all past inputs, decayed by time constant τ_m.

The differential equation above describes the continuous sub-threshold dynamics. To complete the model, we must define the discrete fire mechanism. The LIF neuron operates as a hybrid dynamical system: it integrates continuously until a discontinuity is triggered.

When the membrane potential $u(t)$ exceeds a specific threshold voltage ϑ, the neuron emits a spike (a Dirac delta event). Immediately following this event, the membrane potential is not governed by the differential equation but is forced to a reset value, u_{reset}.

To mimic the biological limit on firing frequency, the model enforces an absolute refractory period, Δ_{ref}. After a spike occurs at time t_f, the neuron is clamped to the resting potential for a duration of Δ_{ref}. During this interval, the differential equation is suspended; the neuron ignores all inputs and cannot fire, regardless of the stimulus intensity.

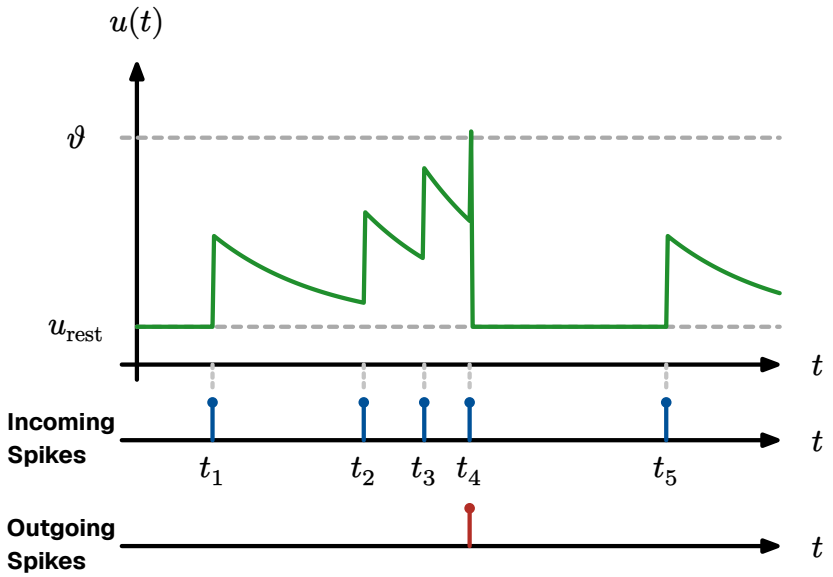

Figure 8: The dynamics of an LIF neuron. The membrane potential integrates inputs. Upon crossing the threshold ϑ, a spike is emitted and the voltage is reset. The voltage is clamped during the refractory period.

Mathematically, this firing condition is expressed as:

$$\text{If } \; u(t) > \vartheta \rightarrow \begin{cases} \text{Emit spike:} \;\; S(t) \rightarrow S(t) + \delta(t) \\ \text{Reset voltage:} \;\; u(t) \rightarrow u_{\text{reset}} \\ \text{Pause integration for} \;\; t \in \left(t_f, t_f + \Delta_{\text{ref}}\right] \end{cases}$$

Equation 6: The discrete firing and reset condition.

This equation represents the engine of most neuromorphic algorithms. It defines a system that integrates information over time and leaks it to ensure temporal relevance. However, once $u(t)$ crosses a threshold ϑ, the linearity breaks and the neuron emits a spike and $u(t)$ is manually reset to u_{reset}.

3.3.2 • THE GENERALIZED (ADAPTIVE) LIF MODEL

While the standard LIF model is computationally efficient, its one-dimensional nature limits it primarily to tonic spiking (regular firing under constant input). It

struggles to replicate the complex, non-linear behaviors observed in the cortex, such as bursting (clusters of rapid spikes) or spike-frequency adaptation (slowing down after sustained activity).

To capture these dynamics without reverting to the computationally heavy Hodgkin-Huxley equations, we employ the Generalized Leaky Integrate and Fire (GLIF) model [28]. This extends the system by introducing a second state variable, $w(t)$, representing cellular adaptation.

$$\tau_m \frac{\mathrm{d}u}{\mathrm{d}t} = -(u - u_{\text{rest}}) + RI(t) - w$$
$$\tau_w \frac{\mathrm{d}w}{\mathrm{d}t} = a(u - u_{\text{rest}}) - w$$

Equation 7: The Adaptive GLIF system. The adaptation variable w provides negative feedback, enabling complex dynamics like bursting and adaptation.

In this coupled system, w provides a negative feedback loop. Every time the neuron spikes, w increments by a constant b, acting as a physiological drag on the membrane potential. By adjusting the coupling parameters between u and w, this two-dimensional system can be tuned to emulate the full spectrum of biological firing patterns.

It is natural to question whether such a mathematically reduced model can genuinely capture the behavior of biological neurons. While the GLIF model discards the specific ionic mechanisms of the Hodgkin-Huxley equations, empirical validation demonstrates that it retains superior computational dynamics for large-scale modeling.

In the 2008 *Quantitative Single-Neuron Modeling Competition* organized by the INCF, phenomenological models like the GLIF (specifically the Adaptive Exponential Integrate-and-Fire) unexpectedly outperformed highly detailed biophysical models in predicting the precise spike times of real cortical neurons.

This counter-intuitive success is due to parameter sensitivity. Complex conductance-based models have dozens of unobservable parameters that are difficult to tune. In contrast, the GLIF model captures the net effect of these mechanisms using macroscopic parameters that can be robustly fitted to data. As demonstrated by Izhikevich (2003) [32], this simple system of two differential equations is capable of reproducing all known firing patterns observed in the mammalian cortex [33]. We leverage this state-dependent adaptation to implement model D (Section 6.3.2), evaluating its effectiveness in temporal sequence decoding.

Consequently, for the purpose of neuromorphic engineering, the GLIF model represents the optimal trade-off between biological fidelity and computational efficiency.

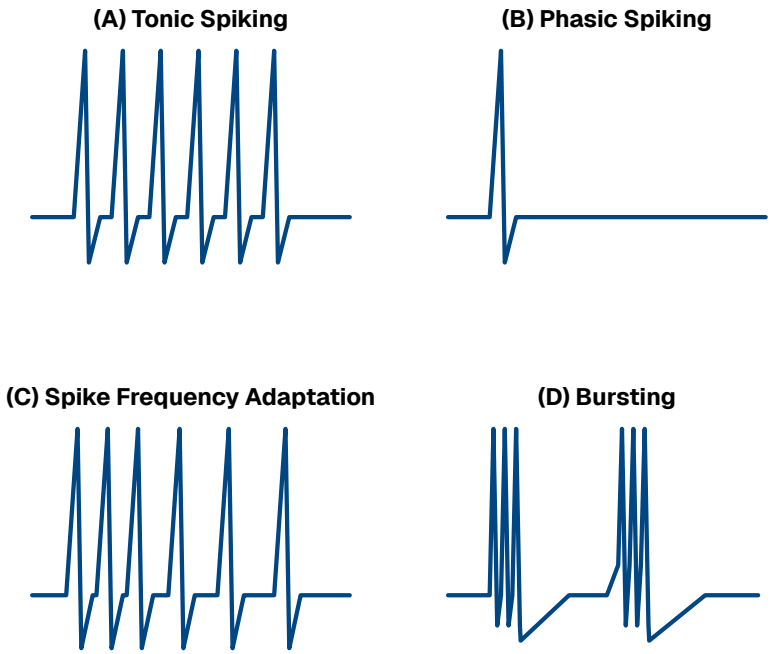

Figure 9: The GLIF model is capable of reproducing the diverse firing patterns of biological cortical neurons, as categorized by Izhikevich (2003) [32].

3.4 • NEURAL CODING

In classical digital computing, information is represented by combining bits into richer structures, such as floating-point or integer numbers. For instance, the luminance of a pixel is typically stored as a discrete 8-bit or 32-bit integer. Conversely, analog electronics represent values as continuous currents or voltages, offering infinite resolution within the dynamic range of the hardware.

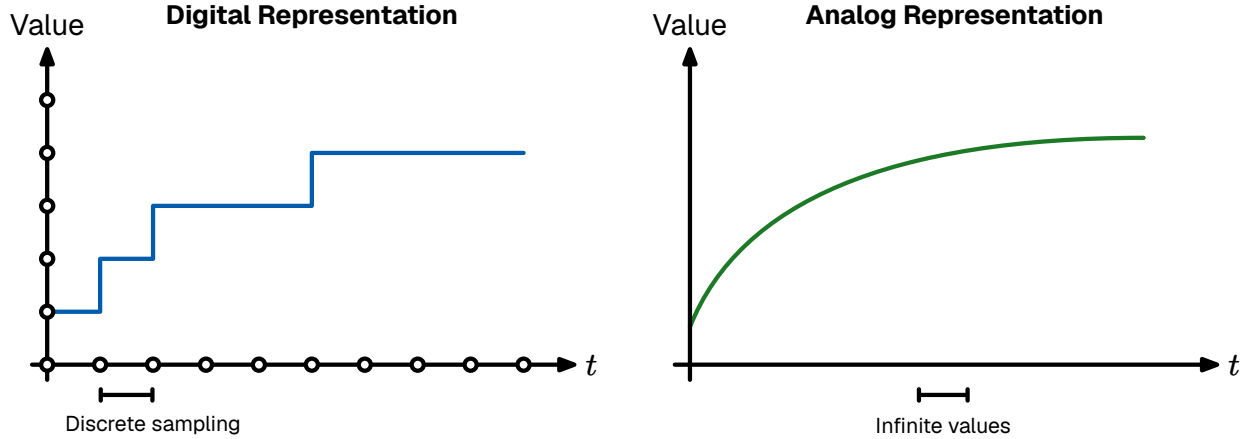

Figure 10: Comparison of digital and analog information representation. Digital representation trades resolution for compatibility on digital hardware. Analog representation offers infinite precision but is incompatible with digital hardware

The biological brain occupies a unique middle ground. While neurons operate using analog membrane potentials, their communication output—the action potential—is discrete and binary. As established in Section 3.2, the waveform of a spike is stereotypical; its amplitude functions much like a digital bit. However, unlike a digital

computer which is synchronized to a rigid clock, these spikes occur in continuous time. Therefore, information in the nervous system is encoded in a hybrid format: a discrete-amplitude, continuous-time signal, often referred to as a quantized continuous signal.

Deciphering the neural code—the set of rules by which sensory stimuli are translated into these spike sequences—remains one of the central challenges in neuroscience. Currently, several coding schemes are hypothesized to coexist, each offering different trade-offs between latency, information density, and robustness.

3.4.1 • RATE CODING

The most traditional interpretation of neural activity is rate coding. In this paradigm, information is conveyed by the mean firing frequency of a neuron over a specific temporal window. A strong stimulus (e.g., high pressure on skin) elicits a high firing rate, while a weak stimulus results in sparse activity.

This model effectively treats the neuron as an analog-to-digital converter where the precise timing of individual spikes is treated as noise; only the average count carries the signal. While rate coding is robust and easily observed in motor neurons [34], it suffers from a fundamental latency barrier. To estimate a rate with reasonable precision, the post-synaptic neuron must integrate spikes over a significant duration (tens or hundreds of milliseconds). This contradicts the rapid reaction times (often < 100 ms) observed in biological agents, suggesting that rate coding alone cannot account for time-critical processing [35].

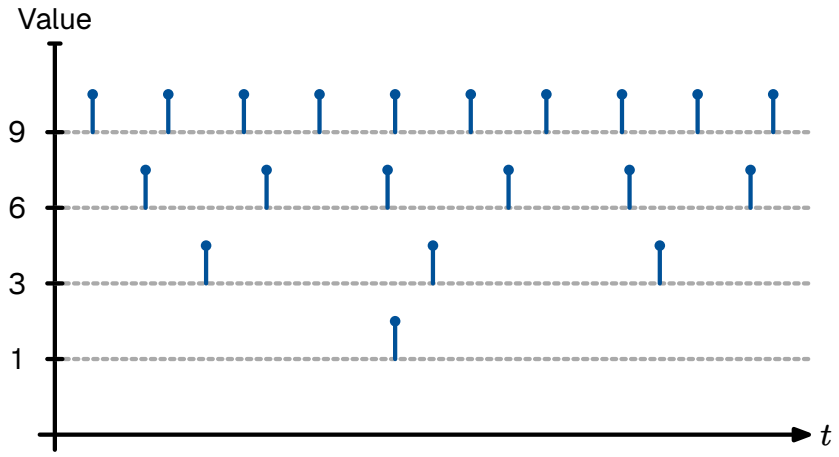

Figure 11: Rate coding: The stimulus intensity is encoded in the frequency of the spike train. Stronger stimuli elicit more spikes per second.

3.4.2 • TEMPORAL CODING

To explain the speed of biological processing, neuromorphic engineering emphasizes temporal coding. In this regime, the precise timing of a spike carries significant information. A primary example is Time-To-First-Spike (TTFS) coding.

In a TTFS scheme, the intensity of a stimulus is inversely mapped to the latency of the response relative to a stimulus onset [36]. A stronger input causes the neuron to integrate and cross the threshold faster, firing earlier than neurons receiving weak inputs. This shifts the computational model from counting spikes to a race between spikes.

In a network utilizing lateral inhibition or Winner-Take-All (WTA), the first neuron to fire inhibits its neighbors, allowing a decision to be made as soon as the first meaningful bit of data arrives. This eliminates the need to wait for a time window to close, drastically reducing latency. Furthermore, since TTFS coding prioritizes the strongest signals, it acts as a natural filter: the most prominent features arrive first, allowing the system to process signal over noise. This TTFS scheme forms the primary encoding modality for our MNIST experiments, as detailed in Section 6.3.1.

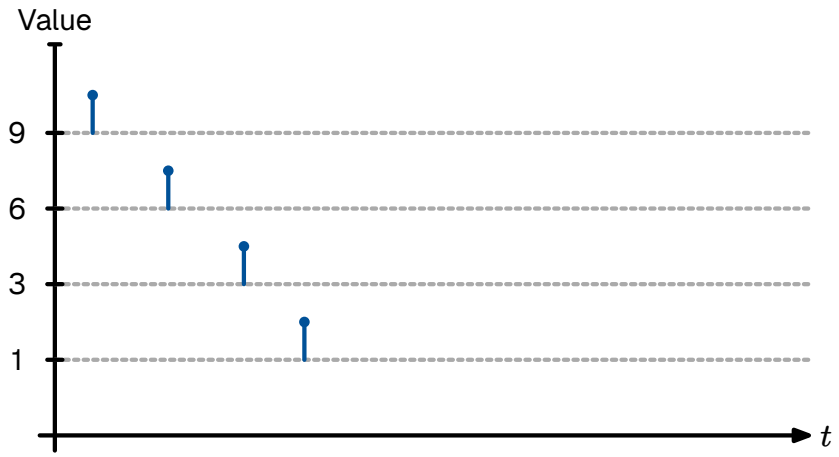

Figure 12: Temporal Coding (TTFS): Stimulus intensity is encoded in the latency of the response. Stronger inputs trigger an earlier spike compared to weaker inputs.

3.4.3 • THE PHASE AMBIGUITY PROBLEM

A critical challenge in temporal coding is the need for a temporal reference frame. In rate coding, the phase (absolute timing) is irrelevant. However, in temporal coding, a spike at time t only has meaning relative to a reference t_0. If the receiver does not know when the stimulus started, it cannot decode the latency.

In engineering, this is solved by a global clock or a frame start signal. In the brain, evidence suggests that background oscillatory rhythms (brain waves, such as theta or gamma cycles) may serve as this global reference, allowing populations of neurons to synchronize their clocks and decode relative timings accurately [37].

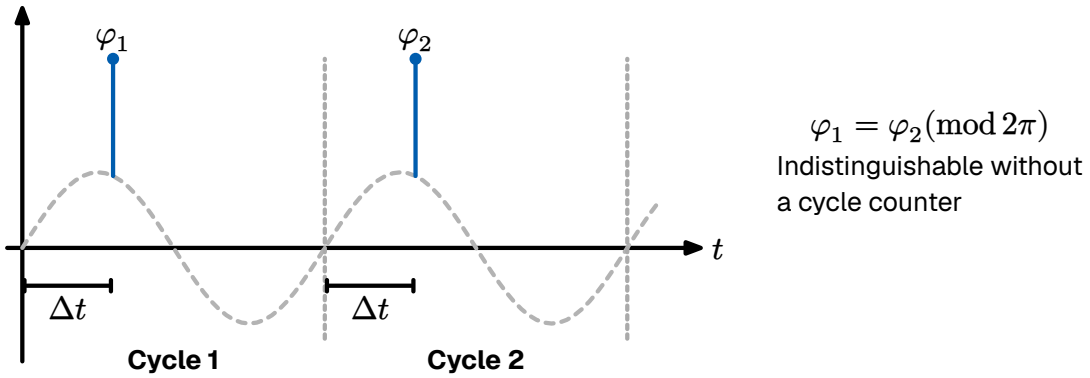

Figure 13: The phase ambiguity problem in temporal encoding. Spikes occurring at the same relative phase (φ_1 and φ_2) across different oscillation cycles are mathematically indistinguishable ($\varphi_1 = \varphi_2 (\mathrm{mod}\, 2\pi)$). Without a mechanism to track the global cycle count, downstream neurons cannot determine whether a spike represents a delayed response to a previous stimulus or an early response to a new one.

3.4.4 • POPULATION & SPARSE CODING

While single-neuron codes provide the basic signaling mechanism, the brain employs ensemble strategies to ensure robustness and precision. In population coding, variables are represented by the joint activity of a large group of neurons, each with broad, overlapping tuning curves. A classic example is found in the primary visual cortex (V1), where orientation-selective neurons each respond maximally to a preferred angle but also fire weakly for nearby orientations [38], [39]. By reading the weighted population vector across the group, the network reconstructs the stimulus with far greater precision than any individual cell could provide alone. The brain further optimizes for metabolic efficiency through sparse coding [40], [41], where only a small fraction of neurons are active at any moment. This strikes a mathematical balance between representational capacity and energy cost, and is naturally enforced by lateral inhibition circuits that suppress weaker, competing responses.

3.4.5 • COEXISTENCE OF CODES

These schemes are not mutually exclusive but complementary. A circuit may use TTFS for a rapid initial response—alerting the system to a salient change—before transitioning to rate-based activity for sustained processing. Neuromorphic systems often adopt this hybrid approach, using temporal codes for energy-efficient sparse event transmission and rate-based readouts for interfacing with downstream control systems. This thesis follows the same principle, using TTFS encoding for the transmission of visual features combined with a population-level representation at the hidden layer.

3.5 • NEURAL NETWORKS

Having established the mathematical description of the individual neuron, we now turn to the collective behavior of these units. A single neuron, regardless of its dynamical complexity, is of limited computational utility in isolation. Functional intelligence emerges only when these units are organized into specific structural topologies. We implement these topologies in our experimental framework, as detailed in Section 6.2.

The brain is not a random mesh of connections; it is constructed from recurring architectural motifs that appear across various cortical areas. Understanding these motifs is essential for designing neuromorphic systems that transcend simple feed-forward processing.

3.5.1 • SYNAPTIC EFFICACY & WEIGHTS

Before analyzing the structural topology of networks, we must define the fundamental unit of connectivity: the synapse. In the biological brain, neurons do not touch; they are separated by a microscopic gap known as the synaptic cleft. Communication across this gap is chemical, mediated by the release of neurotransmitters.

The efficiency of this transmission—determined by factors such as the amount of neurotransmitter released and the number of post-synaptic receptors—is abstracted in mathematical models as the synaptic weight (w).

In the SNN formalism, the weight represents a scaling factor for the incoming spike. When a pre-synaptic neuron j fires a spike at time t_j, it induces a Post Synaptic Current (PSC) in neuron i scaled by the weight w_j^i. Mathematically, the total synaptic input $I(t)$ is the weighted sum of all incoming spike trains:

$$I_i(t) = \sum_j w_j^i \cdot S_j(t)$$

Equation 8: The synaptic input current as a weighted sum of incoming impulses.

Synaptic weights determine not just the magnitude but also if the synapse is excitatory or inhibitory.

Excitatory Synapses ($w > 0$): These depolarize the target neuron, pushing its membrane potential closer to the firing threshold (e.g., Glutamate synapses).

Inhibitory Synapses ($w < 0$): These hyperpolarize the target neuron, pushing the potential away from the threshold (e.g., GABA synapses).

A fundamental constraint in biological networks, known as Dale's principle, states that a neuron performs the same chemical action at all of its synaptic outputs [42]. This means a neuron is strictly excitatory or strictly inhibitory; it cannot send positive signals to one neighbor and negative signals to another. While standard ANNs

often violate this rule for mathematical convenience (allowing weights to flip signs during training), bio-plausible neuromorphic architectures often enforce this constraint to mimic the distinct populations of Pyramidal (excitatory) and Interneuron (inhibitory) cells found in the cortex.

The network must maintain a precise excitation-inhibition balance. The brain operates at a critical point of instability:

Excess Excitation leads to runaway feedback loops (analogous to epileptic seizures).

Excess Inhibition leads to signal extinction (quiescence).

3.5.2 • DIRECTIONALITY

Structurally, neural topologies can be categorized by the flow of information.

In sensory peripheries (such as the retina) and early processing stages, information flows unidirectionally from input to output. This topology supports rapid, reflex-like feature extraction. This configuration is known as a feed-forward network, which is mathematically equivalent to a directed acyclic graph and serves as the standard architecture for most deep learning.

In higher cognitive areas, the dominant topology is recurrence. Neurons form feedback loops, connecting back to themselves or to distinct layers. This recurrence introduces a time component to the computation, transforming the network into a dynamical system where the current output depends not only on the input but on the network's previous state (history).

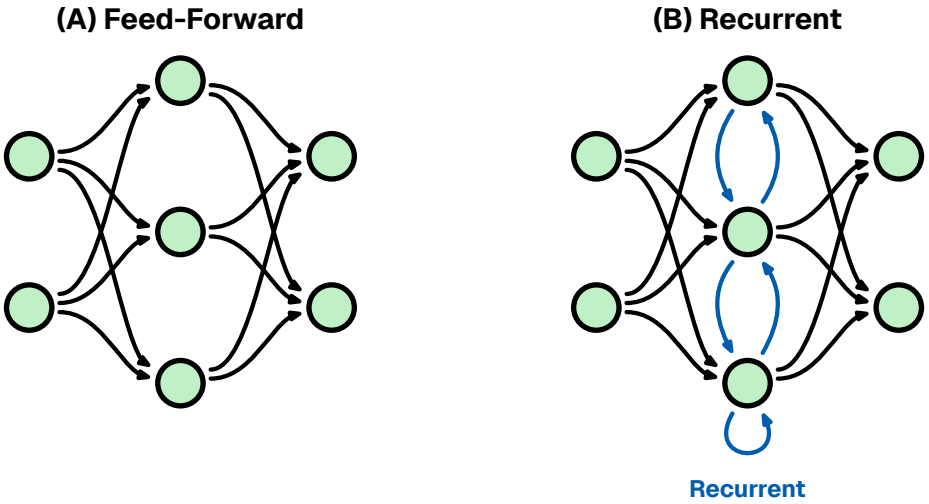

Figure 14: Network topologies. (A) Feed-Forward. (B) Recurrent.

3.5.3 • SYNAPTIC HYPOTHESIS: STRUCTURE AS FUNCTION

A foundational premise in neuromorphic engineering, derived from biological observation, is that the neuron operates largely as a generic processing unit. The functional identity of a neural circuit—whether it processes visual stimuli or governs

motor control—is determined principally by the topology and efficacy of its synaptic interconnections.

This paradigm, known as the synaptic hypothesis [43], posits that the physical configuration of synaptic weights constitutes the substrate for all computation and memory. Unlike traditional von Neumann architectures, where data is retrieved from a distinct memory module and processed in a central CPU, biological systems eliminate the distinction between data and program. Memory is not a static artifact, but a latent computational potential distributed across the network's structural graph. Consequently, learning in a neuromorphic system is realized through the physical alteration of these synaptic weights, ensuring robust, decentralized processing that is inherently resistant to localized hardware failure (graceful degradation).

3.5.4 • INHIBITION PATTERNS

A ubiquitous micro-circuit motif in the cortex is lateral inhibition. In this configuration, an active excitatory neuron stimulates distinct inhibitory interneurons, which in turn suppress the activity of neighboring excitatory neurons. This competition engenders a WTA dynamic: as one neuron—representing a specific feature or decision—becomes active, it effectively silences its competitors.

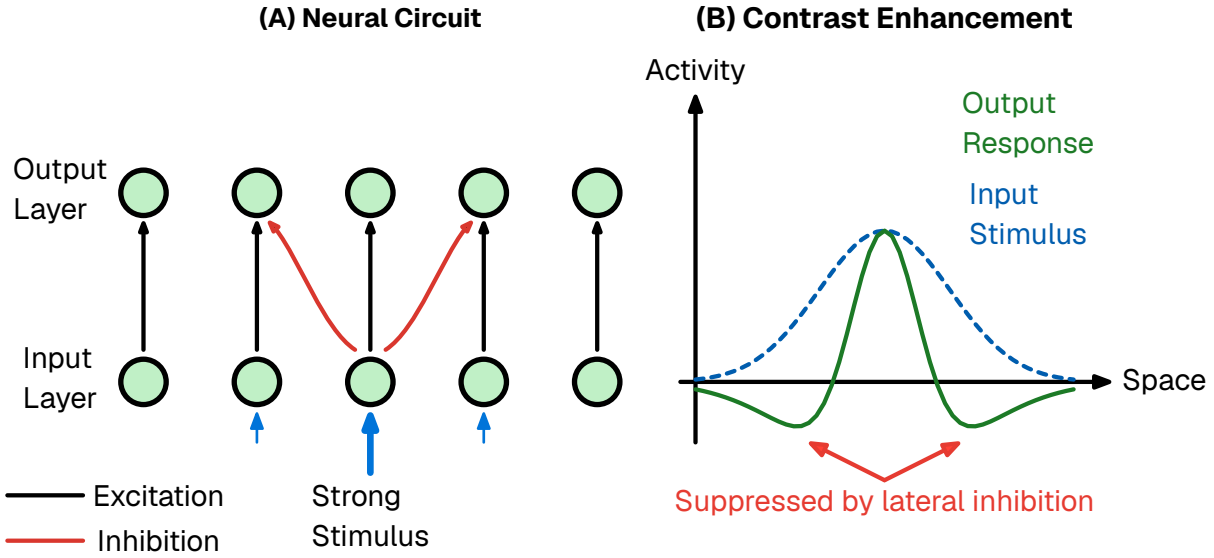

Figure 15: The mechanism of lateral inhibition. A highly stimulated neuron in the input layer strongly excites its corresponding output neuron while simultaneously sending lateral inhibitory signals to its immediate neighbors. This architectural motif acts as a spatial filter, producing a contrast enhancement effect. A broad input stimulus (dashed blue line) is transformed into a sharper output response (solid green line) characterized by an amplified center and suppressed surroundings (a mexican hat profile), thereby sharpening signal boundaries.

In the context of neuromorphic engineering, WTA circuits are indispensable; they provide a physical mechanism for both noise reduction, by actively suppressing weak, sub-threshold signals, and categorical decision making, enabling the circuit to autonomously select the most salient option without the need for a central processor

to sort or compare values. We utilize this WTA motif in our output layer to enforce categorical decision-making.

While lateral inhibition processes information in the spatial domain, Feed-Forward Inhibition (FFI) operates in the temporal domain. Structurally, this motif bifurcates an input signal into two parallel pathways: a direct excitatory route to the target neuron, and a disynaptic inhibitory route that reaches the same target with a slight synaptic delay. This architecture creates a narrow temporal window of opportunity. Because the excitation triggers the neuron immediately before the delayed inhibition abruptly truncates the response, the neuron is prevented from integrating noise over extended durations. Consequently, FFI forces the neuron to function as a precise Coincidence Detector rather than a sluggish integrator, a dynamic that is fundamental to sound localization in the auditory cortex and fine-grain timing in the somatosensory system.

Finally, while lateral and feed-forward motifs dictate spatial filtering and temporal precision, Feedback Inhibition serves as the primary mechanism for network stability and rate control. In this recursive topology, an active excitatory neuron stimulates an inhibitory interneuron, which in turn projects back to suppress the original excitatory cell or its immediate cohort. This creates a physiological negative feedback loop that dynamically dampens activity relative to the network's own output, preventing runaway excitation and maintaining the system within a sensitive dynamic range. Although continuous synaptic feedback presents significant computational overhead in discrete-time simulations, we abstract its functional goal—homeostatic stability and the prevention of dominant neurons—via the discrete threshold adaptation mechanisms detailed in our phase III learning rules.

3.6 • BIOLOGICAL LEARNING

As previously established, the functional identity of a neural circuit is not defined by a transient software state, but by its physical hardware configuration. Consequently, learning in a biological substrate cannot be viewed as a simple parameter optimization; it is a fundamental morphological process. If structure dictates function, then the acquisition of new skills or memories necessitates the physical restructuring of the connectome itself.

Because the brain lacks a central supervisor or global communication bus, this restructuring must be driven by Locality. A synapse can only change based on information physically available at the cleft: the activity of the pre-synaptic axon, the voltage of the post-synaptic dendrite, and the immediate neurochemical environment. Despite this constraint, the brain successfully credits specific synaptic events with outcomes that occur seconds or minutes later.

This adaptation occurs across multiple timescales and spatial resolutions via two distinct mechanisms: structural plasticity (the rewiring of the network topology) and synaptic plasticity (the modulation of connection strength).

3.6.1 • STRUCTURAL PLASTICITY

While synaptic weight adjustment accounts for rapid learning and pattern recognition, the long-term storage of information and the optimization of energy efficiency are governed by structural plasticity. This mechanism involves the physical genesis (synaptogenesis) and removal (pruning) of synapses and even entire axonal branches.

Synaptogenesis: When neurons are repeatedly co-active but lack a direct connection, the brain can physically grow new dendritic spines and axonal boutons to bridge the gap. This effectively alters the network's topology, creating new pathways for information flow where none existed before.

Pruning: Equally critical is the removal of redundant or noisy connections. During sleep and developmental critical periods, the brain aggressively prunes weak synapses. This sparsification reduces metabolic cost and increases the signal-to-noise ratio of the circuit by removing irrelevant pathways.

In the context of the Synaptic Hypothesis, structural plasticity represents the compiling of temporary associations into permanent hardware architecture.

3.6.2 • SYNAPTIC PLASTICITY

Once a structural connection exists, its efficacy—the magnitude of the post-synaptic response to a pre-synaptic spike—must be tuned. In biological terms, this weight corresponds to the amount of neurotransmitter released and the density of receptors on the receiving side. This fine-grained adjustment is governed by local learning rules.

3.6.3 • HEBBIAN LEARNING: RATE-BASED CORRELATION

The foundational axiom of biological learning was postulated by Donald Hebb in 1949. As mentioned in the introduction, Hebb proposed that synaptic efficiency is a function of the correlated activity between two neurons. Colloquially summarized as "Neurons that fire together, wire together," this rule implies that the brain learns by detecting statistical regularities in sensory input.

Mathematically, if neuron A consistently takes part in firing neuron B, the connection from A to B is strengthened. This mechanism allows the brain to perform unsupervised clustering, physically encoding associations between features that occur simultaneously in the environment (e.g., the smell of smoke and the sight of fire).

3.6.4 • SPIKE-TIMING-DEPENDENT PLASTICITY (STDP)

Modern neuroscience has refined Hebb's macroscopic theory into a precise, millisecond-scale mechanism known as STDP. Unlike rate-based models, STDP operates on the precise timing of individual action potentials, introducing the critical element of causality.

The STDP rule adjusts the synaptic weight based on the relative timing difference (Δt) between the pre-synaptic input and the post-synaptic output:

Long-Term Potentiation (LTP): If the input spike arrives **before** the output spike ($\Delta t > 0$), it implies the input contributed to the firing. The synapse is strengthened to reinforce this causal link.

Long-Term Depression (LTD): If the input spike arrives **after** the output spike ($\Delta t < 0$), the input was irrelevant to the decision. The synapse is weakened.

This asymmetry allows the network to self-organize, naturally filtering out random noise while reinforcing specific spatiotemporal patterns. We adapt this causal rule to a discrete temporal window for our unsupervised learning experiments in Section 6.4.2.

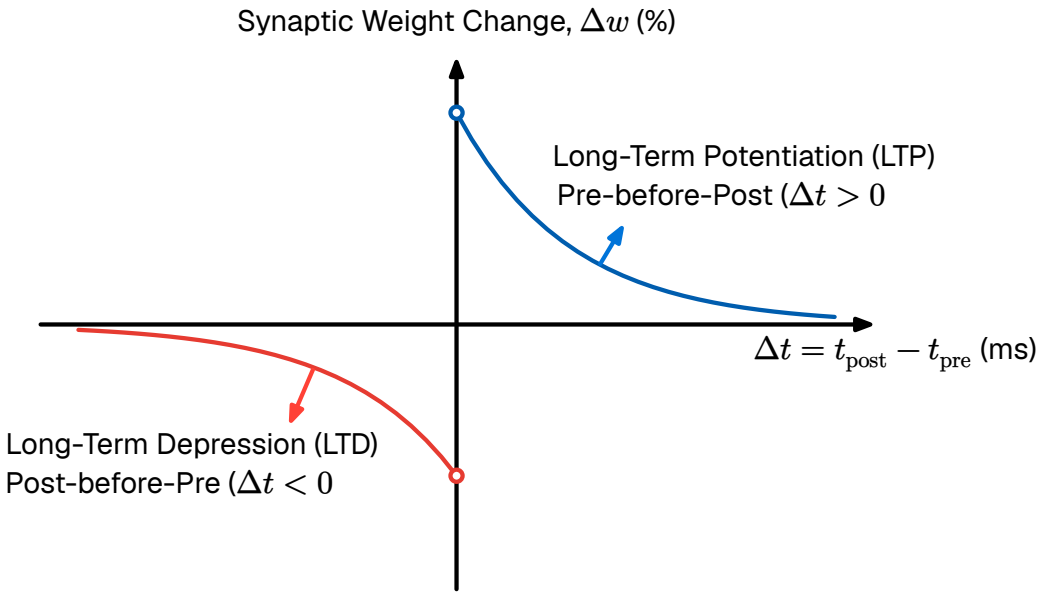

Figure 16: The STDP Learning Curve. Synaptic weight change is plotted against spike timing difference. Pre-before-post timing triggers strengthening LTP, while post-before-pre triggers weakening LTD.

3.6.5 • HOMEOSTATIC PLASTICITY

If Hebbian mechanisms (LTP) were the sole drivers of plasticity, neural networks would be inherently unstable. A positive feedback loop would emerge where strengthened synapses drive higher firing rates, which in turn induce further

strengthening, leading to runaway excitation (seizures). Conversely, unchecked LTD could silence a network entirely.

To maintain stability, the brain employs homeostatic plasticity (or Synaptic Scaling). This is a global regulatory mechanism that operates on a slower timescale (minutes to hours). It functions as a negative feedback loop: if a neuron's average firing rate exceeds a target set-point, the cell chemically downscales the strength of all its incoming synapses. This ensures that neurons remain within a sensitive dynamic range, preventing saturation regardless of how strong the inputs become.

4 • TECHNICAL DETAILS OF MACHINE LEARNING

Having covered the principles of the biological nervous system, we now turn our attention to modern machine learning to identify where its techniques and abstractions diverge from biological reality, and what computational costs those divergences impose. This chapter delineates the technical foundations of modern artificial intelligence, contrasting the established paradigms of deep learning with the emerging principles of neuromorphic engineering. We begin by analyzing the algorithmic architecture of standard deep learning, identifying the computational bottlenecks inherent in its reliance on dense matrix multiplication and backpropagation.

A critical distinction must be drawn between biological plausibility and bio-inspired engineering. From an engineering perspective, the primary objective is functional utility. An engineer may treat the brain merely as a source of heuristic inspiration rather than a blueprint to be copied dogmatically. However, the pursuit of biologically plausible systems remains vital; it offers potential advantages in robustness and energy efficiency while serving as a verification tool for neuroscience.

4.1 • OPTIMIZATION

Optimization is the selection of the best candidate from a set of available alternatives, given defined criteria. Biological learning can be viewed through this lens, where the optimal candidate is a configuration of synaptic weights that performs well on a specific task. Therefore, it is useful to establish a mathematical framework for this process.

Fundamentally, a deep learning model operates as a function approximator. We assume the existence of an unknown underlying function $f : X \to Y$ that perfectly maps inputs to their target outputs. Since this true function is unknown, we construct a family of hypothesis functions $f_{\boldsymbol{\theta}}(\boldsymbol{x})$ to approximate it. Here, $\boldsymbol{\theta} \in \mathbb{R}^d$ represents the state of the system—a vector containing all tunable parameters, such as synaptic weights or biases. The dimensionality d corresponds to the degrees of freedom of the model.

The key problems in optimization are defining the objective goal (the loss function) and finding the parameter configuration $\boldsymbol{\theta}$ that achieves that goal.

4.1.1 • SUPERVISED LEARNING

To guide the search for optimal parameters $\hat{\boldsymbol{\theta}}$, we must quantify the divergence between the model's predictions and the ground truth. We define a scalar loss function $\mathcal{L}(\hat{\boldsymbol{y}}, \boldsymbol{y})$, where $\hat{\boldsymbol{y}} = f_{\boldsymbol{\theta}}(\boldsymbol{x})$, that evaluates the error on a single data point. To ensure generalization, we seek to minimize the empirical cost function $J(\boldsymbol{\theta})$, defined as the average loss over a dataset of size N:

$$J(\boldsymbol{\theta}) = \frac{1}{N} \sum_{i=1}^{N} \mathcal{L}(f_{\boldsymbol{\theta}}(\boldsymbol{x}_i), \boldsymbol{y}_i)$$

Equation 9: The empirical cost function, defined as the average loss over a dataset of N samples.

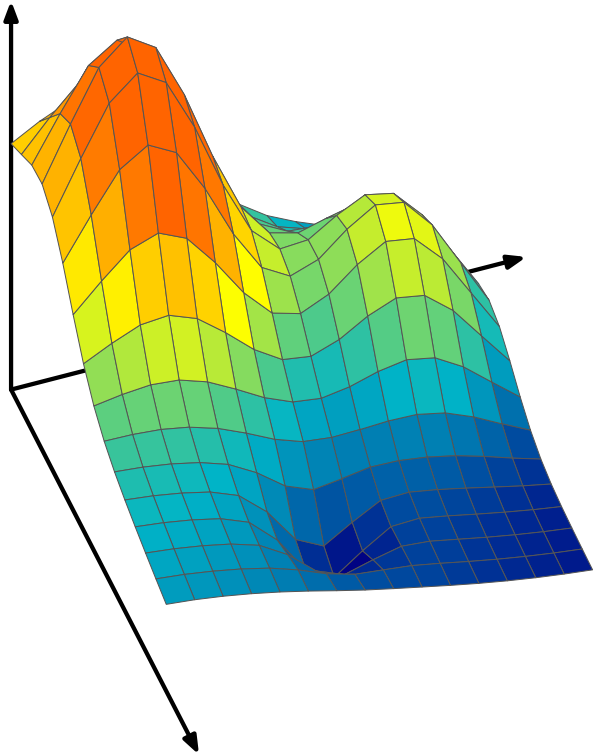

Figure 17: The optimization landscape. The system seeks to traverse the high-dimensional surface defined by $J(\boldsymbol{\theta})$ to find the global minimum $\boldsymbol{\theta}^*$, using the gradient ∇J as a navigational compass.

Geometrically, the cost function $J(\boldsymbol{\theta})$ induces an optimization landscape. Finding a low-energy state in this non-convex topology is the central challenge of AI training. We rely on iterative optimization algorithms, principally gradient descent. This method updates the system state in the direction opposite to the gradient vector $\nabla_{\boldsymbol{\theta}} J(\boldsymbol{\theta})$ (the direction of steepest descent). The update rule for iteration t is:

$$\boldsymbol{\theta}_{t+1} \leftarrow \boldsymbol{\theta}_t - \eta \nabla_{\boldsymbol{\theta}} J(\boldsymbol{\theta}_t)$$

Equation 10: The gradient descent parameter update rule, where η is the learning rate.

Because computing the gradient over the entire dataset N is computationally prohibitive, modern AI employs stochastic gradient descent (SGD), approximating

the gradient using small random subsets (mini-batches). This introduces beneficial noise, preventing the system from getting trapped in shallow local minima.

Crucially, gradient descent requires the loss function to be differentiable. As will be discussed later, this presents a significant challenge for optimizing neuromorphic systems utilizing discrete, non-differentiable spike trains.

Strictly minimizing the empirical cost carries the risk of overfitting—where the model memorizes training data, including noise, rather than learning the underlying function. In biological systems, this is naturally regulated by metabolic constraints; the brain prunes weak connections to maintain a sparse topology, effectively trading model capacity for generalization. In artificial systems, this is managed via explicit regularization penalties added to the cost function.

4.1.2 • UNSUPERVISED LEARNING

While supervised learning relies on labeled targets, biological systems predominantly learn via unsupervised learning. In this regime, the dataset consists only of input vectors $X = \{\boldsymbol{x}_1, ..., \boldsymbol{x}_N\}$. The optimization objective shifts from minimizing prediction error to minimizing representation error.

Mathematically, the goal is often to discover a lower-dimensional manifold that efficiently captures the structure of the data. A common formulation is the minimization of reconstruction loss, where the system attempts to compress the input into a latent code and reconstruct it:

$$J(\boldsymbol{\theta}) = \frac{1}{N} \sum_{i=1}^{N} \| \boldsymbol{x}_i - f_{\text{decode}\left(f_{\text{encode}(\boldsymbol{x}_i, \boldsymbol{\theta})}\right)} \|^2$$

Equation 11: The unsupervised reconstruction loss, where the system minimizes the error between the input and its decoded reconstruction.

Alternatively, the system may optimize for clustering density or distances between feature centroids. The distinction between supervised and unsupervised learning is critical for neuromorphic engineering, as biological plasticity rules (like STDP) are unsupervised, functioning by detecting statistical correlations in the input stream to build internal representations without external labels.

4.2 • DEEP LEARNING FRAMEWORK

Modern deep learning aggregates simple units into high-dimensional layers. A deep network with L layers is expressed as a composite function mapping input $\boldsymbol{x}$ to output $\boldsymbol{y}$ through nested operations:

$$\boldsymbol{y} = f_L(\cdots f_2(f_1(\boldsymbol{x})))$$

Equation 12: A deep network expressed as a composition of L layer functions.

During the forward pass, each layer performs an affine transformation (a linear rotation and scaling of data via weight matrix $\boldsymbol{W}$ and bias $\boldsymbol{b}$) followed by a non-linear activation (σ):

$$\boldsymbol{z}^l = \boldsymbol{W}^l \boldsymbol{a}^{l-1} + \boldsymbol{b}^l$$

$$\boldsymbol{a}^l = \sigma(\boldsymbol{z}^l)$$

Equation 13: The affine transformation and non-linear activation for layer l.

The non-linearity prevents the deep stack from collapsing into a single linear equation. Modern networks rely on the Rectified Linear Unit (ReLU), $f(x) = \max(0, x)$. Its derivative (0 or 1) preserves the magnitude of the gradient, allowing error signals to travel through deep structures without vanishing.[1]

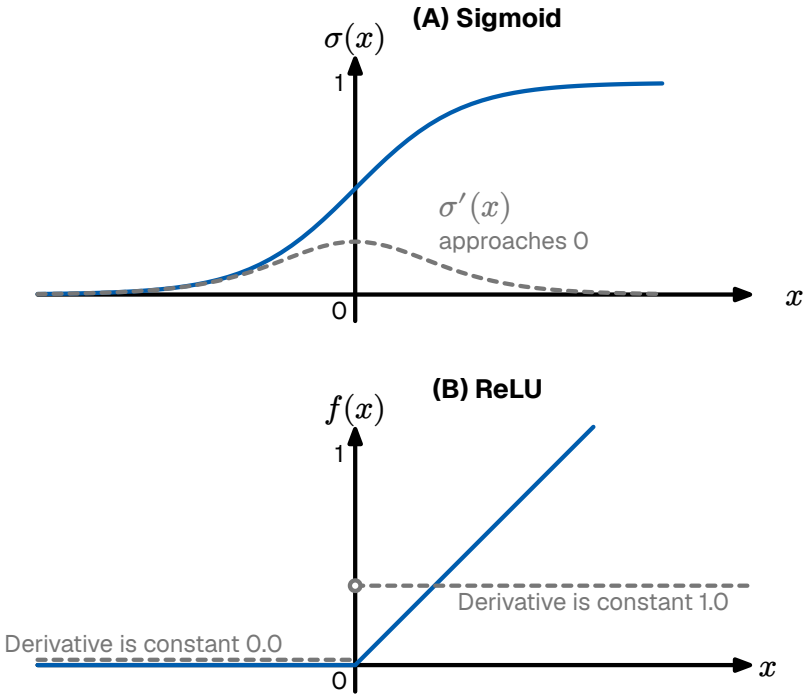

Figure 18: Activation functions. The Sigmoid (top) saturates gradients. The ReLU (bottom) preserves gradient magnitude for positive inputs.

During the backward pass, backpropagation recursively applies the chain rule via automatic differentiation to attribute the total error $J(\boldsymbol{\theta})$ to specific weights.

To achieve high throughput, these operations are vectorized. The affine transformation for an entire layer is executed as a dense matrix multiplication. This is the defining characteristic of modern AI hardware. A deep network is effectively a sequence of massive matrix multiplications, which is highly parallelizable on GPUs

[1]Strictly speaking, the ReLU function is non-differentiable at $x = 0$. While this could theoretically cause standard backpropagation to fail, it is rarely an issue in practice. Pre-activation values almost never evaluate to exactly zero, and modern deep learning frameworks handle this edge case by assigning a subgradient of either 0 or 1 (typically 0) at this point.

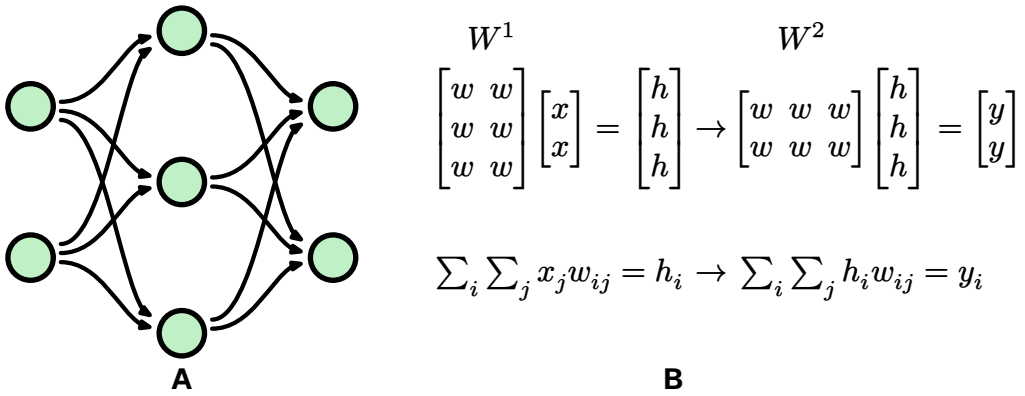

Figure 19: Deep learning as matrix multiplication. The feed-forward pass of the network topology (A) is formulated as a sequence of matrix multiplications (B), where each layer calculates the weighted sum of its respective inputs.

4.2.1 • CONVOLUTIONAL NEURAL NETWORKS (CNNS)

For visual tasks, standard fully connected networks scale poorly. Connecting every pixel to every neuron in the first layer ignores the spatial structure of the data and creates an intractable number of parameters. To solve this, deep learning utilizes CNNs.

CNNs apply small, learnable weight matrices known as kernels (or filters) that slide (convolve) across the input image. This architecture introduces two critical inductive biases:

Local Connectivity: Neurons only process a small, localized receptive field, analogous to the structure of the biological visual cortex.

Weight Sharing: The exact same kernel is applied across the entire image, drastically reducing the number of tunable parameters and establishing translation invariance (a feature learned in one location can be recognized anywhere else).

While CNNs are the standard baseline for spatial processing, they remain fundamentally synchronous and frame-based. They evaluate the entire image structure in dense mathematical passes, regardless of whether there is local activity or useful information present.

4.3 • WHY IS DEEP LEARNING INEFFICIENT?

While the matrix-centric formulation of deep learning enables high-throughput parallelization on GPUs, it fundamentally conflicts with the physical constraints of modern computing hardware. As models scale to billions of parameters, the primary bottleneck shifts from algorithmic capability to physical realizability. This inefficiency manifests in four distinct engineering dimensions:

4.3.1 • THE VON NEUMANN BOTTLENECK AND DATA MOVEMENT

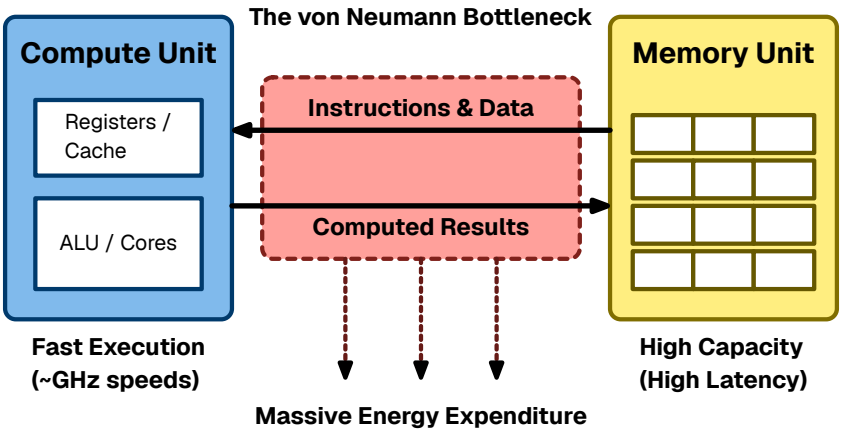

Figure 20: The von Neumann bottleneck. The physical separation of memory and compute forces massive energy expenditure on data transport.

The most significant physical limitation is the von Neumann architecture (illustrated in Figure 20), which physically separates the processing unit from the memory unit. To perform a single inference step, the processor must fetch the entire weight matrix from off-chip DRAM to on-chip registers, perform the calculation, and write the results back.

According to Horowitz [23], fetching a 32-bit value from off-chip DRAM consumes approximately 640 pJ, whereas performing a floating-point addition on that value consumes only 0.1 pJ. Thus, the system expends 99.9% of its energy transporting data, and only a fraction of a percent actually computing it.

4.3.2 • DENSE PROCESSING OF SPARSE DATA

Standard deep learning implementations rely on dense matrix multiplication (GEMM, effectively conceptualized in Figure 19). This approach is algorithmically rigid: it executes the exact same number of operations regardless of the underlying data content.

Real-world sensory data is often highly sparse, and the ReLU activation function (visualized in Figure 18) naturally produces activation maps where the majority of values are zero. However, a standard GPU is blind to this sparsity. It will dutifully fetch a zero from memory and multiply it by a weight ($0 \times w = 0$), consuming energy and clock cycles to produce a null result. Deep learning's inability to exploit this silence represents a massive structural inefficiency.

4.3.3 • THE HIGH COST OF SYNCHRONY

Deep learning hardware is fundamentally synchronous, operating in lockstep with a global clock. Driving a high-frequency clock signal across an entire silicon die forces billions of transistors to charge and discharge continuously, regardless of whether the chip is doing useful work. In high-performance processors, this clock

distribution network alone can consume 30% to 40% of the total power budget [43], [44]. Furthermore, global synchronization enforces a worst-case latency: faster computations must sit idle and wait for the slowest operations to finish before the next clock cycle begins.

4.3.4 • BACKPROPAGATION AND GLOBAL DEPENDENCIES

Finally, backpropagation imposes severe constraints on memory and latency because it is non-local in both time and space. To update a specific weight, the system must wait for the forward pass to finish, calculate the global error, and wait for the backward pass to propagate the gradient.

This creates a locking problem. The activations of every intermediate layer must be stored in high-speed memory (VRAM) for the duration of the entire pass, preventing that memory from being reused. Additionally, a local synapse cannot adapt to local changes instantly; it is shackled to the global error loop.

4.4 • PRINCIPLES OF NEUROMORPHIC ENGINEERING

As established in Section 2, Neuromorphic Engineering is the translation of biological dynamics into silicon hardware. It replaces the rigid, clock-driven logic of standard computing with the adaptive, event-driven dynamics of neural tissue. This approach rests on three architectural pillars that directly address the bottlenecks of deep learning:

Co-location of Memory and Compute (The Synaptic Principle): Neuromorphic architectures eliminate the von Neumann bottleneck by distributing memory across the silicon die. Each artificial neuron stores its own state and synaptic weights locally, processing data in situ to eliminate the energy cost of shuttling data.

Event-Driven Asynchrony (The Action Potential Principle): Neuromorphic systems abandon the global clock. They operate asynchronously, driven strictly by the arrival of data. If a part of the network is not processing information, it consumes negligible power, ensuring energy scales linearly with task complexity rather than network size.

Sparse Communication (The Spike Principle): Neuromorphic systems utilize binary Spikes for communication. Information is encoded in the precise timing of events rather than complex magnitudes, drastically reducing the bandwidth required to transmit information between neurons.

4.5 • TRAINING SPIKING NETWORKS

While the physical architecture of neuromorphic systems is highly efficient, training these networks presents a fundamental mathematical challenge. Standard deep learning relies on gradient descent, but backpropagation cannot be directly applied to native SNNs.

In a spiking network, the neuron's activation function is a discontinuous step function (the Dirac delta event threshold). The derivative of this function is zero everywhere except at the exact moment of the spike, where it is undefined. Consequently, gradients calculated using the chain rule immediately drop to zero—known as the dead neuron problem—preventing error signals from flowing backward through the network to update the weights.

To circumvent this non-differentiability and optimize network parameters, the field of neuromorphic engineering generally employs two distinct paradigms:

4.5.1 • DIRECT WEIGHT TRANSFER

A pragmatic engineering approach to bypass the dead neuron problem is offline training. In this paradigm, a standard, continuous ANN (such as a network utilizing ReLU activations) is trained conventionally using backpropagation. Once convergence is achieved, the learned weights are directly mapped onto a structurally identical Spiking Neural Network.

The underlying premise is that the continuous activation values of the ANN can be approximated by the discrete firing rates of the SNN over a set time window. While this method allows the spiking system to inherit the high accuracy of gradient descent, direct weight transfer requires careful scaling and normalization. If the weights are copied without adjustment, the resulting SNN may suffer from catastrophic saturation (firing constantly) or severe signal degradation (failing to reach the spiking threshold). This conversion process and its associated quantization constraints are evaluated in the implementation in Section 6.4.1.

4.5.2 • NATIVE LOCAL LEARNING (STDP)

To fully exploit the energy efficiency and event-driven dynamics of neuromorphic hardware, training must ideally occur natively on the spiking substrate. This requires abandoning global backpropagation in favor of biologically plausible, mathematically local learning rules.

As established in Section 3.6, STDP adjusts synaptic weights based strictly on the temporal correlation of local pre- and post-synaptic spikes. Because STDP relies exclusively on local physical events rather than global error gradients, it does not require a differentiable loss function. This allows the network to completely bypass

the dead neuron problem, enabling unsupervised feature extraction and real-time adaptation directly on the spiking architecture.

4.5.3 • SURROGATE GRADIENT DESCENT

For completeness, it must be noted that the current dominant paradigm in SNN research utilizes surrogate gradients. In this approach, the network operates using the discontinuous spike step-function during the forward pass, but temporarily replaces the undefined derivative with a smooth, continuous approximation (a surrogate) during the backward pass. While this thesis focuses on evaluating direct weight transfer and native unsupervised STDP, surrogate methods represent a highly effective hybrid approach, allowing backpropagation-like algorithms to estimate gradients across discrete spiking layers.

4.6 • NEUROMORPHIC HARDWARE TECHNIQUES

The biological brain possesses a remarkably high connectivity density. This massive degree of fan-out is physically unrealizable in standard Complementary Metal-Oxide-Semiconductor (CMOS) processes due to the planar (2D) nature of silicon manufacturing. To overcome this wiring bottleneck, neuromorphic hardware utilizes Adress Event Representation (AER), a communication protocol that mirrors the sparse, event-driven nature of biological spikes [45]. Instead of continuous data streaming, the system only transmits the address of a firing neuron across a shared, high-speed digital bus. This time-multiplexing allows a single physical wire to represent thousands of virtual axonal projections.

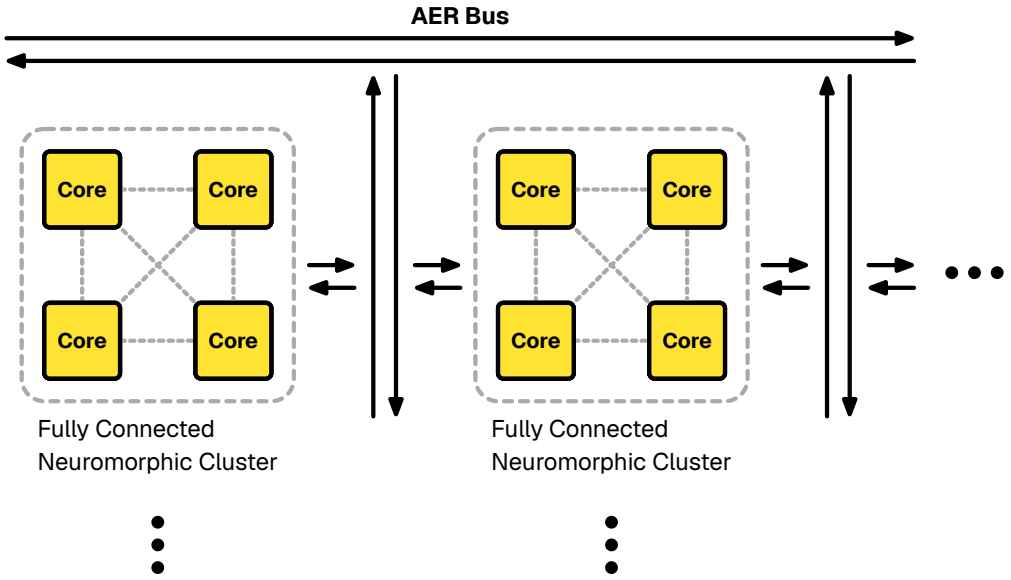

Figure 21: Hierarchical routing in neuromorphic hardware. AER buses handle sparse, long-distance communication between neuron clusters, while local, intra-cluster communication relies on dense physical wiring.

Figure 21 illustrates an example of this hierarchical implementation. While clusters of neurons communicate over long distances via AER, local communication between neurons within a single cluster can be implemented using physical peer-to-peer connection topologies, such as a fully connected crossbar array.

The computation performed within an individual neuron can be implemented in a variety of ways, ranging from general-purpose digital cores to highly specialized analog circuits. Figure 22 depicts how analog computation can be performed natively in a crossbar configuration. This array provides a direct structural surrogate for the biological neuropil. Because the architecture handles multiplication and summation natively through physical laws, it is uniquely suited to implement biological macromotifs. By routing bitline currents through local feedback loops, the hardware can instantiate complex dynamics, such as lateral inhibition and winner-take-all circuits, without the overhead of high-level software instructions. While analog computation is exceptionally energy-efficient, it introduces engineering challenges related to noise, device mismatch, and control [46], [47].

This synergy between physical topology and functional motifs allows the hardware to inherit the computational efficiency of the neocortex, effectively making the architecture itself the algorithm.

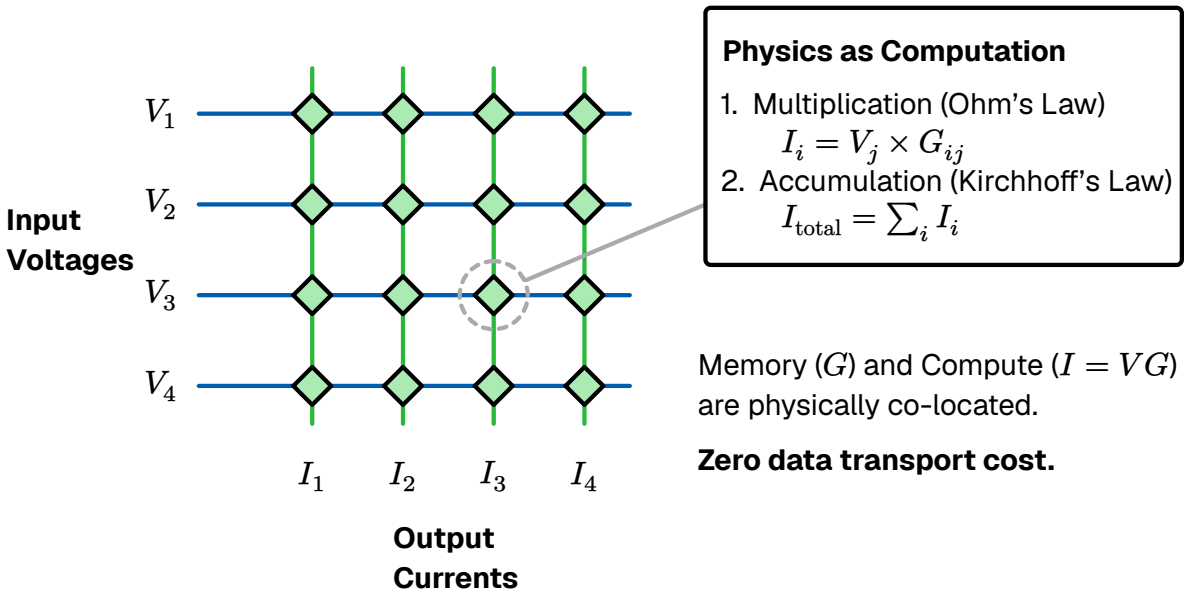

Figure 22: In-Memory Computing via a Crossbar Array. Unlike von Neumann architectures, memory and computation are physically co-located. Input voltages (V) are applied to the wordlines. Memory elements at the junctions hold programmable conductances (G). Multiplication is natively performed at each junction by Ohm's Law ($I = V \times G$), and resulting currents are summed along the bitlines via Kirchhoff's Current Law. This allows dense matrix-vector multiplications to occur in a single analog time step with zero data transport cost.

5 • RELATED WORKS

The shift towards neuromorphic computing is largely driven by the urgent need for energy-efficient machine intelligence. Recent literature highlights a rapidly expanding landscape of opportunities for neuromorphic algorithms, moving beyond isolated benchmarks and toward real-world applications. The methodologies implemented in this thesis—specifically temporal coding, sparsity, and local learning —intersect directly with several active areas of state-of-the-art research.

5.0.1 • TIME-BASED CODING AND BIOLOGICAL PLAUSIBILITY

The foundational premise of our TTFS architecture is that precise spike timing carries significant information. This is powerfully supported by recent biological evidence; in vivo recordings from the human cortex confirm that neuronal sequences during population bursts explicitly encode information through their temporal order [48].

Translating these biological temporal sequences into functional artificial networks has been a major focus of recent engineering efforts. Han and Roy have demonstrated that deep spiking neural networks can achieve extreme energy efficiency specifically by leveraging time-based coding paradigms [49]. Concurrently, theorists are continually refining how biological constraints—such as excitation-inhibition balance and adaptive homeostatic currents—can be integrated into leaky integrate-and-fire networks to achieve optimal efficient coding.

However, a persistent open question in the field concerns the compatibility of standard neuron models with the strict priority ordering required by TTFS decoding. As demonstrated in phase I of this thesis, classical leaky integrate-and-fire dynamics are fundamentally misaligned with TTFS principles: the exponential leak mechanism actively penalizes early, high-salience spikes, causing the neuron to implicitly favor discordant sequences. This motivates the development of the momentum-based and state-discounting neuron models evaluated herein, which are designed to mathematically align integration dynamics with the strongest-first priority of temporal coding.

5.0.2 • HARDWARE ACCELERATION AND SPARSITY

A core theoretical advantage of the TTFS encoding utilized in our experiments is spatial and temporal sparsity. However, realizing these efficiency gains requires bridging the gap between software simulation and physical deployment. Recent state-of-the-art implementations have focused heavily on exploiting this sparsity directly on customized hardware. For example, Sommer et al. [50] proposed novel memory interlacing and hardware acceleration architectures for sparsely active

convolutional SNNs on Field Programmable Gate Arrays (FPGAs), achieving significant speedups by ensuring processing time scales directly with the number of spikes.

Furthermore, event-driven algorithms are increasingly being paired with native event-based sensors. Applications such as spiking optical flow estimation from dynamic vision sensors have been successfully deployed on specialized neuromorphic substrates like IBM's TrueNorth [51], proving the viability of low-power, asynchronous processing in dynamic environments.

A critical finding of this thesis is that these efficiency gains cannot be realized by simply simulating sparsity on conventional GPU hardware. While the TTFS SNN achieves an 85.2% reduction in theoretical synaptic operations compared to the ANN baseline, GPU architectures optimized for dense matrix multiplication continue to execute the underlying floating-point operations for quiescent neurons masked to zero. This sparsity paradox reinforces the broader finding of the hardware acceleration literature: energy efficiency in neuromorphic algorithms is contingent on deployment on native event-driven substrates such as Intel Loihi or IBM TrueNorth, not simulation on von Neumann processors.

5.0.3 • ANN-TO-SNN CONVERSION

A significant portion of contemporary neuromorphic engineering focuses on bridging the gap between classical deep learning and spiking hardware via offline training. Foundational work by Rueckauer et al. demonstrated that continuous-valued networks could be losslessly converted into SNNs by carefully scaling weights and normalizing spiking thresholds [52]. However, the vast majority of these conversion methodologies rely on rate coding, requiring hundreds of simulation ticks to approximate continuous activation values via temporal averaging.

Applying zero-shot weight transfer directly to a TTFS encoding, as explored in phase II of this thesis, represents a distinct and more sensitive challenge: quantization errors impact temporal priority rather than just average frequency, and the absence of a rate-averaging window means the network must reach a correct decision from the very first spike wavefront. Despite this constraint, the momentum-based model C architecture achieves 94.50% top-1 accuracy on MNIST with a mean decision latency of 8.4 ticks, demonstrating that the spatial hierarchies learned by gradient descent are recoverable from a purely temporal spike ordering without any intermediate retraining or fine-tuning.

5.0.4 • UNSUPERVISED STDP AND COMPETITIVE LEARNING

The application of unsupervised STDP to benchmark datasets like MNIST serves as the standard baseline for evaluating biologically plausible learning. The foundational architecture for this approach was established by Diehl and Cook [53], who demonstrated that a spiking network utilizing STDP, lateral inhibition, and adaptive homeostatic thresholds could achieve 95% accuracy on MNIST without any labels.

However, it is critical to note that their result relied on a massive spatial population of 6,400 hidden neurons and extended rate-based integration windows to form robust, overlapping receptive fields.

Phase III of this thesis evaluates these same principles under substantially more severe constraints: a 128-neuron architecture operating under strict TTFS single-spike encoding, where each neuron fires at most once per saccade and spatial redundancy is eliminated. Under these conditions, the network achieves 44.3% top-1 accuracy —a result that, while far below the rate-coded ceiling, nonetheless demonstrates successful unsupervised geometric clustering without any global error signal. The performance gap relative to Diehl and Cook is therefore not a failure of the STDP rule itself, but a direct and quantifiable consequence of removing the spatial redundancy on which rate-coded representations depend. This provides a controlled data point for understanding the interaction between network scale, temporal encoding precision, and the representational capacity of local learning rules.

6 • METHOD

This chapter details the specific implementations of the neuromorphic architectures proposed to address the limitations of standard deep learning. Aligning with the biological constraints of sparsity, asynchrony, and locality established in previous chapters, we outline the construction and evaluation of a SNN.

To empirically validate the theoretical advantages of neuromorphic algorithms, we evaluate the system on a benchmark image classification task. The experiment is split into three distinct phases:

Phase I—Neuron Model Evaluation: Evaluating the decoding efficiency and accuracy of different simulated spiking models.

Phase II—Inference Via Weight Transfer: Evaluating the zero-shot performance of these SNNs initialized with weights directly mapped from a classically trained Artificial Neural Network (ANN).

Phase III—Native Unsupervised Learning: Training the SNN from scratch utilizing local STDP.

6.1 • DATASET & PRE-PROCESSING

To benchmark these algorithms, we require a dataset that necessitates the extraction of complex spatial features but remains computationally tractable for rapid experimental iteration. We utilize the MNIST database of handwritten digits [54].

The dataset consists of a training set of 60,000 examples and a test set of 10,000 examples of digits (0-9). Each instance is a 28×28 pixel grayscale image. While standard deep learning models routinely score over 90% accuracy on this task, making it largely a solved problem in classical AI, its well-understood feature space makes it an ideal, isolated baseline. Because the spatial hierarchy of MNIST is relatively shallow, it allows us to evaluate the efficacy of neuromorphic learning rules without the confounding variables introduced by massive, multi-layered convolutional architectures.

Crucially, the MNIST images are pre-processed by the dataset creators to be size-normalized and centered within the pixel grid using the center of mass of the pixels. This spatial alignment is a vital prerequisite for our chosen network topology. Unlike CNNs, which slide localized filters across an image, the FCFN utilized in this thesis lacks translation invariance. If a digit were shifted several pixels off-center, the FCFN would perceive it as an entirely novel pattern. The pre-centered nature of MNIST mitigates this limitation, ensuring that the network can reliably map specific geometric strokes to specific input neurons.

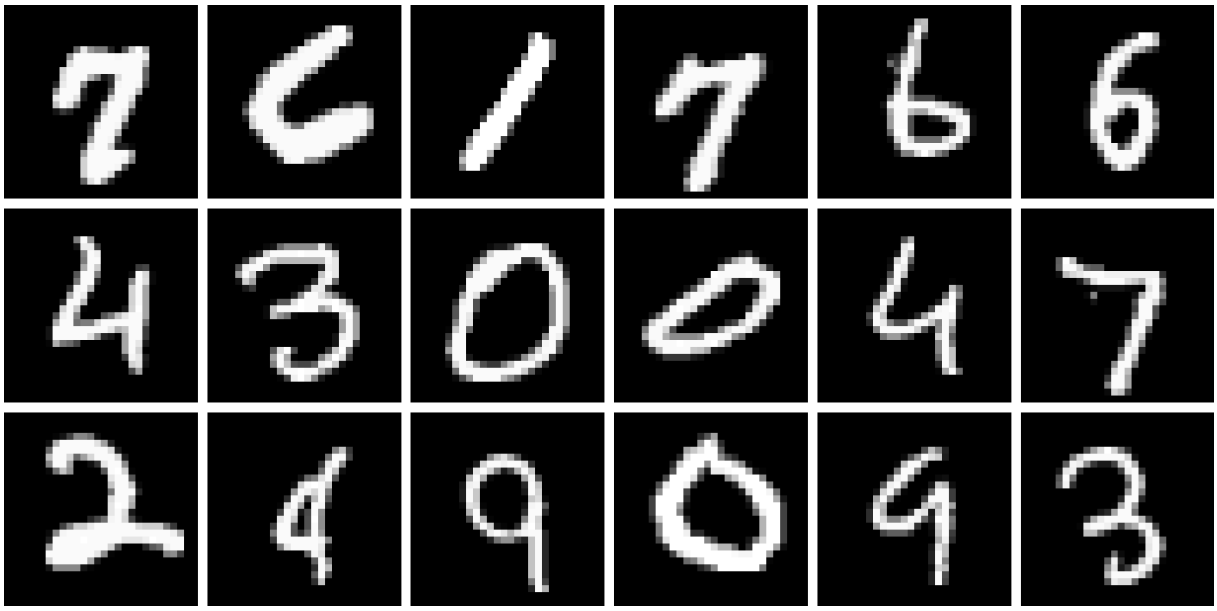

Figure 23: Sample of the MNIST dataset. The 28x28 images are normalized and flattened into 1D vectors before being translated into temporal spike events.

Furthermore, the dataset exhibits a high degree of spatial sparsity. If we look at Figure 24 we see that in a typical MNIST image, the vast majority of pixels represent the empty background. From a neuromorphic engineering perspective, this sparsity is highly advantageous. As established in the theoretical framework, event-driven systems expend energy strictly when events occur. A sparse input array ensures that the majority of input neurons remain quiescent, minimizing bus congestion and validating the energy-efficiency claims of the proposed SNN.

Before the raw images can be converted into temporal spike trains, they must undergo standard spatial pre-processing to ensure compatibility with the network's mathematical boundaries. This consists of two primary transformations:

Normalization: Raw pixel intensities in the MNIST dataset range from 0 (pure black) to 255 (pure white). To stabilize the learning algorithms and ensure consistent weight scaling, these values are strictly normalized to a continuous float range of $p_i \in [0.0, 1.0]$.

Flattening: Because this thesis utilizes a FCFN to facilitate direct weight transfer, the 2D spatial structure of the images must be unrolled. Each 28×28 matrix is flattened into a 1-dimensional vector of 784 elements.

Consequently, every individual image is presented to the system as a discrete array of 784 normalized intensities. In the classical ANN, these continuous values are fed directly into the input neurons. However, because SNNs operate exclusively on discrete events, these normalized values must be passed through a temporal encoding algorithm before inference or learning can begin.

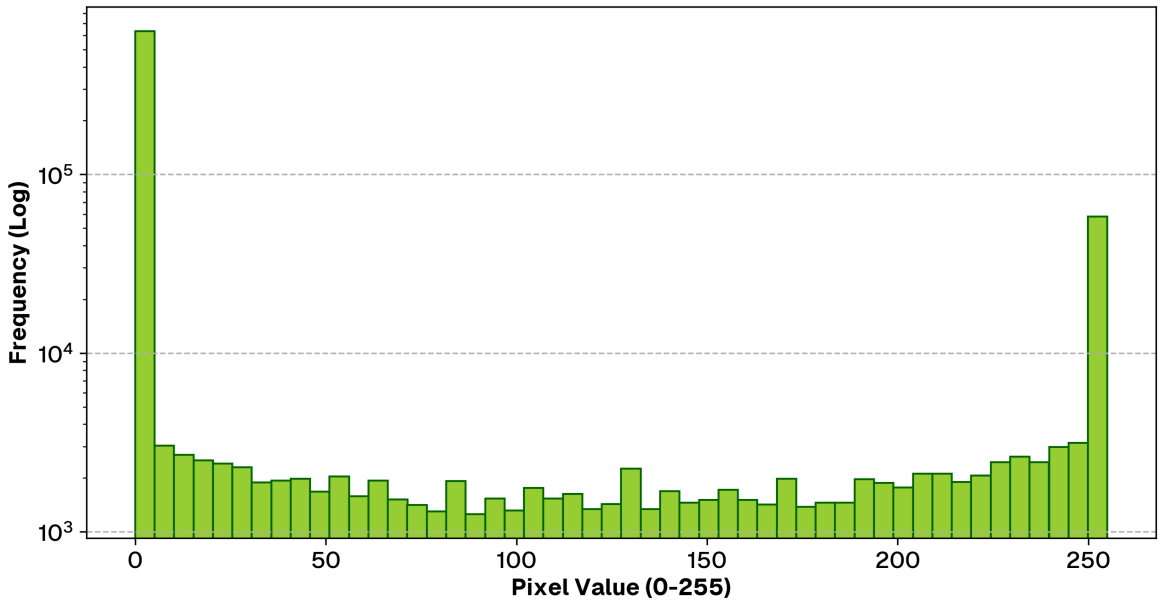

Figure 24: Histogram of pixel intensities across the MNIST dataset. The distribution illustrates a high degree of spatial sparsity, with the vast majority of pixels representing the black background.

6.2 • NETWORK ARCHITECTURE

To facilitate a direct, one-to-one mapping of synaptic weights from the ANN to the SNN, both models must share an identical macroscopic topology. That is the motivation for why this implementation utilizes a FCFN rather than a CNN.

While CNNs are the standard baseline for vision tasks due to their spatial inductive biases (the assumption that neighboring pixels share correlated features), transferring convolutional kernels into a spiking substrate introduces additional structural mapping overhead—specifically the need to physically unroll and duplicate shared weights across the spiking array. An FCFN provides a straightforward, mathematically transparent architecture for cleanly evaluating direct weight transfer and STDP without confounding architectural variables.

The network is structured as a shallow hierarchy to capture the primitive geometric features of the dataset. Let N_l denote the number of neurons in layer l. The formal architecture is defined as follows:

Input Layer (L_0): The 28×28 pixel grayscale images are flattened into a 1D vector, requiring $N_0 = 784$ input neurons.

Hidden Layer (L_1): A fully connected intermediate layer consisting of $N_1 = 128$ neurons. The synaptic connections are defined by the weight matrix $W^1 \in \mathbb{R}^{N_1 \times N_0}$.

Output Layer (L_2): $N_2 = 10$ neurons, corresponding directly to the categorical digit classes (0 through 9). The connections from the hidden layer are defined by the weight matrix $W^2 \in \mathbb{R}^{N_2 \times N_1}$.

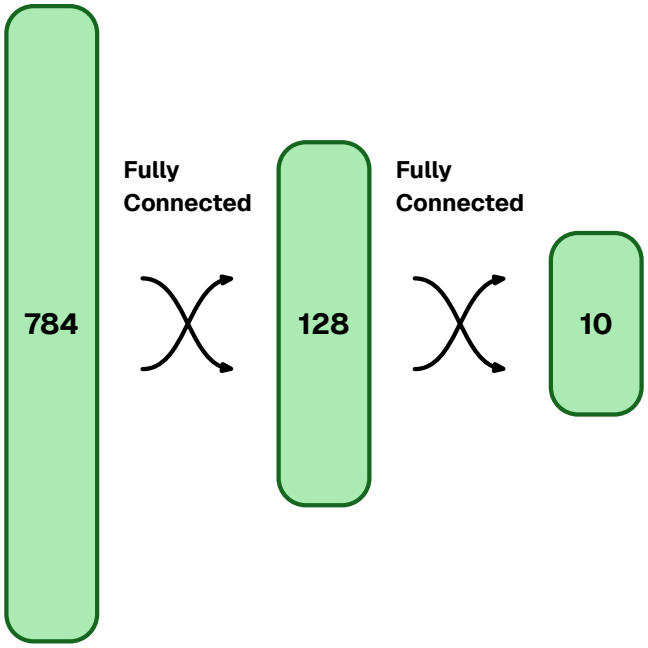

Figure 25: Network architecture. The 28x28 images are flattened and passed through a FCFN. Both the ANN and SNN share this identical macroscopic topology.

6.2.1 • WEIGHT INITIALIZATION AND BIASES

For the baseline ANN, the weight matrices are initialized using standard PyTorch defaults to ensure stable variance during the initial forward passes.

In a standard ANN layer, the pre-activation is computed as $z^l = W^l a^{\{l-1\}} + b^l$ seen in Equation 13. However, a critical architectural consideration in ANN-to-SNN conversion is the handling of these bias vectors (b^l). While standard ANNs utilize biases to shift activation functions, representing a static continuous bias in a spiking network requires either injecting a constant background current into the neuron or actively modifying the neuron's firing threshold. To maintain pure, event-driven sparsity and isolate the performance of the synaptic weights, explicit bias terms were omitted entirely from both architectures.

By omitting explicit bias terms, the activation function operates with a fixed zero-crossing threshold. To compensate for the lack of learned biases, the network relies entirely on the interplay of positive (excitatory) and negative (inhibitory) synaptic weights to naturally suppress or elevate activation.

6.2.2 • ANN-SNN COMPATIBILITY

Despite the macroscopic symmetry required for weight sharing, the microscopic dynamics of the two networks differ fundamentally. The offline ANN utilizes standard

continuous activation functions (specifically ReLU) to compute smooth gradients during backpropagation.

In contrast, the SNN replaces these continuous functions with Integrate and Fire (IF) neurons governed by a strict voltage threshold. This simulates the biological all-or-nothing action potential, acting as a hard step function. Furthermore, the spiking architecture utilizes WTA at the output layer, and during training WTA is used at the hidden layer as well. As the network integrates evidence over time, the first output neuron to reach its threshold heavily suppresses its competitors, forcing a definitive categorical decision and actively filtering sub-threshold noise.

WTA VS LATERAL INHIBITION

In biological neural networks, competitive learning is typically enforced via dense recurrent collateral connections that provide continuous lateral inhibition. When a neuron fires, it physically suppresses its neighbors' membrane voltages, forcing the network to specialize and preventing multiple neurons from learning the identical feature.

However, replicating continuous lateral voltage suppression in a discrete-time software simulation introduces significant computational overhead and chaotic voltage fluctuations during the forward integration pass. To maintain a clean, deterministic forward pass while still enforcing strict competitive specialization, this implementation completely bypasses continuous lateral inhibition in favor of an algorithmic WTA mechanism.

We utilize the concept of a saccade—a temporal window representing a single computational inference phase, analogous to the duration of a biological eye movement. Instead of exchanging inhibitory synaptic blasts during the temporal saccade, the hidden neurons integrate their membrane potentials entirely independently. The competition is resolved strictly post-hoc or at the exact moment of the first spike: the first neuron to cross its threshold triggers a strict WTA condition, claiming the feature and instantly suppressing all competitors from learning or firing further. This algorithmic abstraction perfectly replicates the functional goal of lateral inhibition —ensuring feature decorrelation—while vastly simplifying the network topology and guaranteeing simulation stability.

6.3 • SNN INFORMATION REPRESENTATION

The choice of neural code lays the foundation for information flow and dictates the efficiency of the entire system. While rate coding (encoding pixel intensity as spike frequency) is straightforward and simple to implement with standard Integrate-and-Fire neurons, it is inefficient compared to TTFS. Rate codes require integration over extended time windows to calculate an average, introducing latency and saturating

the network bus with redundant spikes. Furthermore, on digital hardware rate coding imposes additional stress on the system due to rapid switching which is very bad for transistor power draw and bus congestion.

To maximize energy efficiency and processing speed, this implementation utilizes a TTFS temporal encoding [55]. In this regime, a single spike carries the information payload. A high-intensity (bright) pixel triggers an early spike, while a low-intensity (dark) pixel triggers a late spike. This compresses the spatial information into a highly sparse, priority-driven queue; downstream neurons begin processing as soon as the most salient features arrive, without waiting for an entire frame to integrate.

As noted in Section 3.4, temporal codes suffer from phase ambiguity—downstream neurons need a reference clock to decode latency. To resolve this without relying on a rigid, global system clock, we simulate the biological concept of a saccade (the rapid movement of the eye to fixate on a target). The initial presentation of the image acts as a synchronized global event (t_0). All subsequent input spikes are evaluated relative to this saccade onset, providing a natural, biologically plausible temporal reference frame.

6.3.1 • ENCODING

Following the TTFS principles described in Section 3.4.2, we convert the continuous pixel intensities of the MNIST dataset into discrete spike trains. For a given input image, we extract the luminance of each pixel and normalize it to a bounded range, where $p_i \in [0, 1]$. 1 representing maximum intensity and 0 representing the background).

We implement a single, highly sparse linear conversion mapping to evaluate latency dynamics. The pixel intensity $p_i \in [0.0, 1.0]$ is inverted such that brighter pixels correspond to shorter delays. In our implementation, the full saccade window being 64 ticks. Furthermore, to aggressively enforce input sparsity, any pixel with an intensity below 0.1 is treated as background noise and discarded (assigned a delay of infinity, meaning it never fires). If we look at Figure 24 we see a vast majority of dark pixels that will be omitted from the simulation entirely.

$$t_i = \begin{cases} \text{round}((1.0 - p_i) \cdot 64) & \text{if } \; p_i \geq 0.1 \\ \infty & \text{if } \; p_i < 0.1 \end{cases}$$

Equation 14: Intensity-to-delay encoding implementation

Under this mapping, the brightest pixels fire immediately near $t = 0$, transmitting the most critical structural features of the digit first, while background pixels are entirely suppressed. While a biophysically exact model of photocurrent integration would yield latencies proportional to $\frac{1}{p_i}$, this linear inverse mapping provides a computationally efficient approximation that successfully preserves rank-order priority for digital TTFS encoding.

6.3.2 • DECODING WITH NEURON MODELS

Decoding the temporal information generated by the TTFS encoding requires a neuron model capable of accumulating discrete events. However, a fundamental engineering tension exists between biological fidelity, computational efficiency, and how a model dynamically reacts to temporal density.

To systematically evaluate these trade-offs, this thesis implements and benchmarks four distinct neuron models. These models range from simple arithmetic accumulators requiring global synchronization to complex, fully asynchronous dynamic systems. By adjusting the threshold bounds during phase I experiments, we evaluate how each distinct mathematical approach processes incoming spikes and updates its membrane potential $u(t)$ when an event arrives at time t, carrying a synaptic weight w_i.

MODEL A: THE SIMPLE WINDOW INTEGRATOR (STANDARD IF)

The most computationally lightweight approach is the standard IF model without any leak or decay mechanisms. In this paradigm, the neuron acts as a pure arithmetic accumulator during the simulation window, as detailed mathematically in Equation 15 and algorithmically in Algorithm 1.

$$u(t_k^+) = u(t_{k-1}^+) + w_j$$

Equation 15: Discrete recurrence relation for model A, Let k index the chronologically ordered sequence of incoming spike events. The membrane potential updates immediately after the arrival of a spike at time t_k from pre-synaptic neuron j. The superscript $+$ denotes the state immediately following the impulse.

Computational Complexity: Minimal. This model is extremely cheap to execute on digital hardware, requiring only a single addition operation $O(1)$ per incoming spike.

Temporal Dynamics: Static. While highly efficient, this model's integration is purely additive. Its temporal dynamics are rigidly tied to threshold calibration; if the threshold is low, it fires prematurely based solely on early magnitude accumulation, remaining blind to the wider temporal distribution of the spike train. The straightforward evolution of the membrane potential is illustrated in Figure 26.

Synchronization: Because the potential never naturally decays, this model relies entirely on a rigid, globally synchronized saccade (a hard reset of u to 0 at the end of the simulation window) to prevent the network from firing continuously due to lingering historical noise.

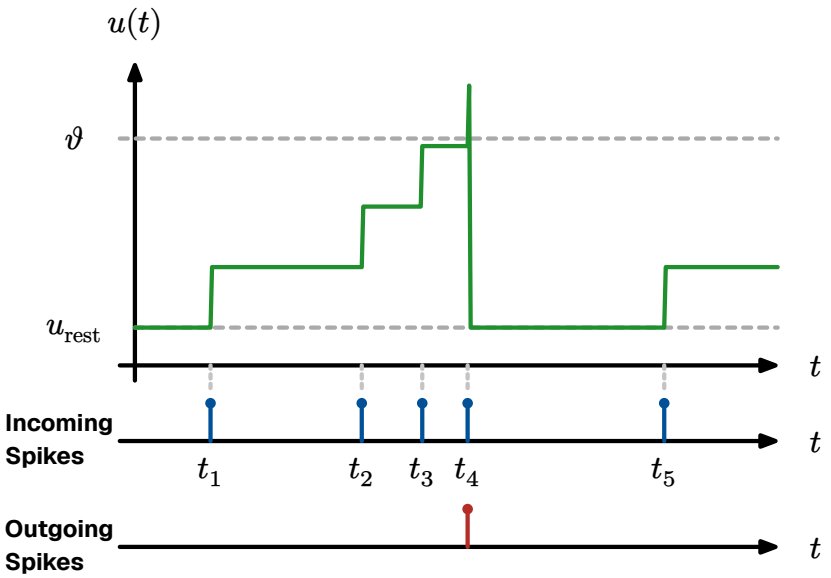

Figure 26: Evolution of state variables in model A. The membrane potential $u(t)$ undergoes incremental integration upon the arrival of each afferent spike until it reaches the firing threshold ϑ

```
simpleIntegratorNeuron(S, W, ϑ) → t_fire:
    u ← 0.0
    for each (t_i, w_i) ∈ S:
        u ← u + w_i
        if u ≥ ϑ:
            return t_i
    return ∞
```

Algorithm 1: Model A: simple window integrator algorithm

MODEL B: THE STANDARD LEAKY INTEGRATE-AND-FIRE (LIF)

Model B—the LIF model is covered in great detail in Section 3.3.1. Model B fires only if the incoming spike train has a sufficient density of spikes. A sparse spike train cannot overcome the exponentially decaying membrane potential, governed by the recurrence relation in Equation 16 and implemented via Algorithm 2.

$$u(t_k^+) = \max\left(0, u(t_{k-1}^+) \cdot \exp\left(-\frac{t_k - t_{k-1}}{\tau_m}\right) + w_i\right)$$

Equation 16: Discrete recurrence relation for model B. The $\max(0, ...)$ bounding is strictly necessary to prevent the membrane potential from dropping below the resting state of 0.0, as the network utilizes inhibitory (negative) synaptic weights

Computational Complexity: High. This model is significantly more intensive, requiring the calculation of exponential functions for every discrete event, which consumes substantial clock cycles on standard arithmetic logic units.

Temporal Dynamics: Decay-driven. The exponential leak provides a basic temporal filter that naturally favors spikes arriving in rapid succession. However, its effectiveness is highly dependent on both threshold constraints and the time constant τ_m. If the threshold is set too high relative to the input density, the signal degrades before threshold crossing, preventing firing entirely.

Synchronization: Similar to model A, while the leak reduces residual noise, it generally still requires a global saccade reset between distinct inference phases to guarantee a clean slate for the next image.

```
leakyIntegratorNeuron(S, W, ϑ, τ_m) → t_fire:
    u ← 0.0
    t_prev ← 0.0
    for each (t_i, w_i) ∈ S:
        Δt ← t_i − t_prev
        u ← max(0.0, u · exp(−Δ t/τ_m) + w_i)
        if u ≥ ϑ:
            return t_i
        t_prev ← t_i
    return ∞
```

Algorithm 2: Model B: standard leaky integrate-and-fire algorithm

MODEL C: THE CURRENT-ACCUMULATING LINEAR RAMP

To explore dynamic accumulation without the computational overhead of continuous exponential kernels, model C utilizes a linear time-dependent accumulator combined with a strict hard reset timer ($T_{\text{window}} = 10$ ticks), formalized in Equation 17 and Algorithm 3. The strict 10-tick coincidence window (T_{window}) was introduced to prevent the unbounded integration of dispersed background noise over the long 64-tick saccade. It forces the neuron to act as a highly localized coincidence detector, rapidly zeroing its state if an initial spike is not quickly supported by subsequent evidence. The arrival of a spike increments both the instantaneous potential $u(t)$ and the integration gradient $I(t)$. However, if the neuron does not reach the threshold within the coincidence window of the initial spike, the timer expires and both state variables are aggressively wiped to 0.0. This is achieved by modeling the membrane potential $u(t)$ as a system of coupled differential equations driven by a sequence of Dirac delta impulses:

$$\dot{I}(t) = \sum_i w_i \delta(t - t_i)$$

$$\dot{u}(t) = I(t) + \sum_i w_i \delta(t - t_i)$$

This formulation ensures that an afferent spike at time t_i with weight w_i exerts a dual influence: it induces an immediate translocation of the potential state while simultaneously incrementing the rate of change for all subsequent integration intervals. Because early spikes establish the baseline slope $I(t)$ for the remainder of the simulation window, they exert a disproportionate momentum on the time-to-threshold.

$$I(t_i^+) = I(t_{i-1}^+) + w_i$$

$$u(t_i^+) = u(t_{i-1}^+) + I(t_{i-1}^+) \cdot (t_i - t_{i-1}) + w_i$$

Equation 17: Discrete recurrence relations for model C

Computational Complexity: Moderate. While the model successfully replaces power-intensive transcendental operations with simple floating-point additions and a single multiplication per spike event ($I \cdot \Delta t$), it introduces state-management overhead. The system must actively track and decrement the coincidence timers for all currently stimulated neurons, requiring additional conditional logic and memory access to aggressively zero the state variables upon expiration.

Temporal Dynamics: Highly reactive. The quadratic-like growth of the potential relative to spike arrival times means earlier spikes violently accelerate the trajectory toward the threshold. By shifting the threshold bounds, we can observe how this momentum-driven integration drastically alters firing latency compared to standard static accumulators. This trajectory is visualized in Figure 27.

Synchronization: To prevent interference between successive inputs, a global reset (synchronized with saccadic eye movements) is required. In the absence of such a reset, the integration of state variables $I(t)$ and $u(t)$ may overlap across saccades. This creates a temporal ambiguity where the model is unable to differentiate an early spike in the current window from a delayed spike belonging to the prior temporal frame.

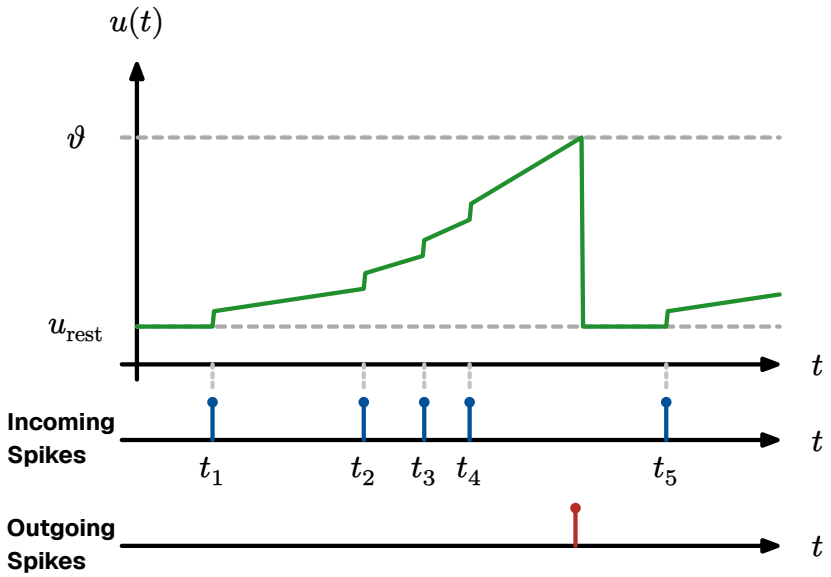

Figure 27: Evolution of state variables in model C. This demonstrates the resulting piecewise linear membrane potential $u(t)$. Note how earlier spikes increase the integration gradient, accelerating the trajectory toward the firing threshold ϑ.

```
linearRampNeuron(S, W, ϑ, T_window) → t_fire:
    u ← 0.0
    I ← 0.0
    t_prev ← 0.0
    t_start ← ∞
    for each (t_i, w_i) ∈ S:
        if (t_i − t_start) ≥ T_window:
            u ← 0.0
            I ← 0.0
            Δt ← 0
        else:
            Δt ← t_i − t_prev
        if u = 0.0:
            t_start ← t_i
        u ← u + (I · Δt) + w_i
        I ← I + w_i
        if u ≥ ϑ:
            return t_i
        t_prev ← t_i
    return ∞
```

Algorithm 3: Model C: discrete state update algorithm with coincidence window

MODEL D: THRESHOLD-SENSITIVE INTEGRATION (STATE-DEPENDENT DISCOUNTING)

Drawing inspiration from the adaptation mechanisms of the GLIF model (Section 3.3.2), model D introduces a state-dependent penalty to incoming spikes, implemented via Equation 18 and Algorithm 4. Rather than penalizing spikes based on the passage of time, this model penalizes spikes based on the current internal state of the neuron.

In this paradigm, the increase in membrane potential is inversely proportional to the current potential itself. When a spike arrives at a neuron in its resting state ($u = 0$), there is no discount, and the full synaptic weight is added. However, if a spike arrives when the neuron is already close to the firing threshold ϑ, its impact is exponentially discounted.

$$u(t_i^+) = u(t_{i-1}^+) + w_i \cdot \exp\left(-\gamma \frac{u(t_{i-1}^+)}{\vartheta}\right)$$

Equation 18: Discrete recurrence relation for model D weight discounting

Computational Complexity: Moderate. While it requires an exponential calculation (or a linear approximation), it does not need to continuously track the elapsed time

(Δt) between individual spikes, saving memory overhead compared to continuous-time dynamic models.

Temporal Dynamics: State-dependent. This mechanism creates a unique temporal compression (Figure 28). The earliest spikes contribute their absolute maximum weight, while late arrivals encounter a partially filled potential and are severely dampened. Testing this model across varying thresholds reveals an asymptotic behavior where the neuron may never fire if the initial temporal momentum is insufficient to overcome the compounding discount.

Synchronization: Similar to models A and C, this model requires a rigorous global synchronization protocol (e.g., a saccade-driven clock). Because this model drops the continuous temporal leak in favor of event-driven weight scaling, the potential does not naturally decay to zero between images and relies on a periodic reset at the conclusion of the simulation window.

```
thresholdSensitiveNeuron(S, W, ϑ, γ) → t_fire:
    u ← 0.0
    for each (t_i, w_i) ∈ S:
        η ← exp(−γ · u/ϑ)
        u ← u + (w_i · η)
        if u ≥ ϑ:
            return t_i
    return ∞
```

Algorithm 4: Model D: threshold-sensitive integration algorithm

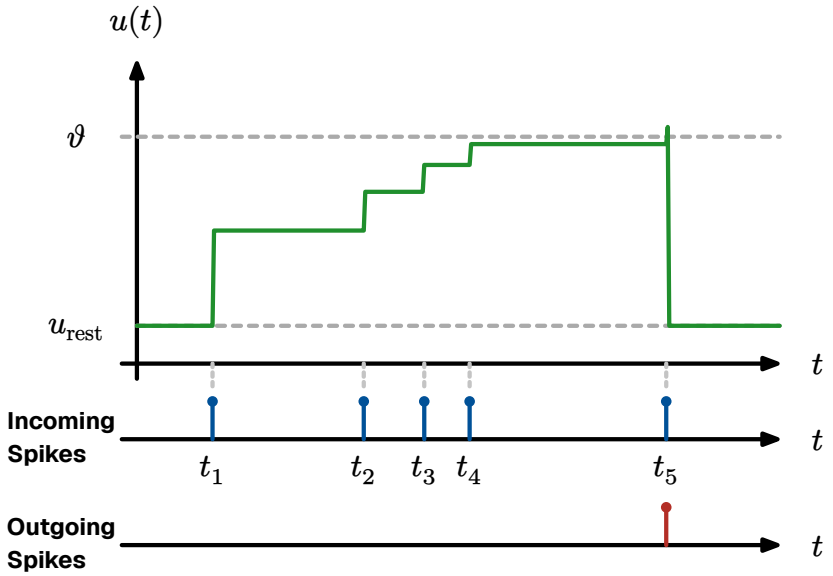

Figure 28: Evolution of state variables in model D. This demonstrates a neuron model where new incoming spikes have an exponentially decaying influence based on the current potential $u(t)$.

THRESHOLDING

To rigorously evaluate these distinct temporal dynamics, phase I of our methodology introduces a variable thresholding benchmark. Rather than relying on a single fixed threshold to measure performance, we systematically adjust the membrane threshold (ϑ) relative to the total sum of the incoming synaptic weights ($\sum w_i$).

If ϑ is configured significantly lower than $\sum w_i$, models naturally reach early saturation, firing prematurely based strictly on the magnitude of early-arriving spikes. By sweeping the threshold from low to high (approaching the total weight sum), we can empirically demonstrate how the different internal mechanics—perfect integration, leaky decay, momentum accumulation, and state-discounting—respond to the exact same temporal patterns under varying saturation constraints.

6.4 • TRAINING METHODOLOGIES

Following the optimization principles established in Section 4.1, we evaluate two distinct training methodologies for the spiking network: direct weight transfer from a pre-trained continuous model, and unsupervised STDP-based learning. In a neuromorphic context, these methods represent two different paths to network organization—one relying on mapped global gradients and the other on local, physical self-organization driven by spike timing.

The core objective of the STDP-based approach is the autonomous detection of spatiotemporal sequences. As evidenced by cortical recordings suggesting that population bursting relies heavily on the temporal order of spikes to encode information [48], our TTFS paradigm treats the relative arrival order of spikes as the defining characteristic of a pattern. A neuron learns a specific sequence (e.g., $A \to B \to C$) by strengthening the synapses of the earliest-arriving spikes. This allows the neuron to reach its firing threshold as soon as sufficient prefix evidence ($A \to B$) is integrated.

However, relying on early arrivals requires a mechanism to ensure selectivity and filter noise. Because the neuron fires before the entire stimulus has necessarily been processed, we implement a causal penalty mechanism. If a neuron fires based on an initial spike cluster but the expected terminal components of the pattern (the suffix) fail to arrive within the saccadic window, the associated synaptic efficacy is decayed. This prevents the network from becoming overly sensitive to incomplete or noisy patterns, ensuring that synaptic weights only stabilize for sequences that are consistently completed.

6.4.1 • WEIGHT TRANSFER

Following the theory of offline training and mapping discussed in Section 4.5.1, the baseline methodology for translating a conventionally trained ANN into a spiking architecture relies on direct parameter mapping. To maintain strict structural sym-

metry and perfectly isolate the effects of the temporal encoding, the floating-point weight matrices (W^1 and W^2) are transferred one-to-one from the ANN to the SNN:

$$W_{\text{SNN}}^{l} = W_{\text{ANN}}^{l}$$

Equation 19: Direct zero-shot weight transfer

Crucially, this transfer is performed zero-shot—meaning there is no intermediate retraining, no fine-tuning in the spiking domain, and no discrete quantization of the parameters. The SNN loads the exact FP32 continuous weights optimized by the ANN's gradient descent.

Because the SNN replaces continuous dot-product activations with discrete, time-dependent spike accumulation, the continuous activation scales do not naturally align with integer-based temporal thresholds. To bridge this gap, the continuous FP32 weights are multiplied by a constant scaling factor ($\times$ 64) prior to transfer. This artificially expands the dynamic range of the synaptic weights, allowing them to effectively drive the momentum parameters and membrane potentials of the SNN without degrading the spatial hierarchies learned by the ANN.

By porting these weights directly to the GPU without quantization, the simulator establishes a pure baseline to measure the temporal penalty—the isolated loss of classification accuracy incurred strictly by transitioning from static continuous activations to TTFS momentum integration.

6.4.2 • TTFS STDP INSPIRED LEARNING RULE

Our learning rule adapts the STDP mechanism from Section 3.6.4 to the TTFS domain. In standard continuous networks, synaptic weights are updated via global error gradients. In the STDP paradigm, a synapse w_{ij} connecting a pre-synaptic neuron i to a post-synaptic neuron j is updated based strictly on the temporal difference between their respective firing times.

Let t_i denote the spike time of the input neuron, and t_j denote the spike time of the output neuron. The relative arrival time is defined as $\Delta t = t_j - t_i$. Because this architecture utilizes a strict TTFS encoding (formalized in Equation 14) where neurons fire at most once per saccade, the classical continuous STDP curve is adapted into a discrete, deterministic update rule:

LTP: If $t_i < t_j$, the pre-synaptic spike arrived before (or exactly at) the moment the post-synaptic neuron fired. This indicates causality. The synapse is strengthened, with the magnitude of the update decaying exponentially the further apart the spikes occurred.

LTD: If $t_i > t_j$, the pre-synaptic spike arrived after the post-synaptic neuron had already fired. The input was irrelevant to the decision, and the synapse is subsequently weakened.

Unused Synapses (Penalty): If a pre-synaptic neuron fires but the post-synaptic neuron never fires during the saccade, a slight negative decay is applied to encourage forgetting of dead connections.

$$\Delta w_{ij} = \begin{cases} A_+ \cdot \exp\left(-\frac{t_j - t_i}{\tau_+}\right) & \text{if } t_i < t_j \quad \text{(LTP)} \\ -A_- \cdot \exp\left(-\frac{t_i - t_j}{\tau_-}\right) & \text{if } t_i > t_j \quad \text{(LTD)} \end{cases}$$

Equation 20: Additive STDP weight update. Here τ_+ and τ_- are the potentiation and depression time constants respectively, controlling the temporal width of the learning windows.

Biologically, τ_+ and τ_- typically fall in the range of 10–20 ms, reflecting the duration over which causal spike relationships are reinforced or suppressed [28]. In the discrete TTFS domain, however, where each neuron fires at most once per saccade, the exponential decay terms reduce to a single binary decision: a synapse either arrived before or after the post-synaptic spike. Consequently, the continuous time constants τ_+ and τ_- are subsumed by the flat magnitude constants A_+ and $-A_-$ in the vectorized coincidence detector implementation described below.

To prevent runaway synaptic growth, the updated weights are strictly clamped to a physical hardware range $w_{ij} \in [W_{\min}, W_{\max}]$, allowing the network to utilize both excitatory and inhibitory learned synapses.

6.4.3 • VECTORIZED COINCIDENCE DETECTION

Traditional biological STDP relies on continuous exponential decay kernels to modulate synaptic plasticity. However, deploying continuous exponentials in discrete-time software simulations introduces severe computational bottlenecks and memory overhead. To resolve this, phase III implements a pragmatic, hardware-optimized coincidence detector variant of STDP.

Rather than calculating exact exponential time-deltas, the update rule acts as a vectorized boolean filter based strictly on the firing time of the winning neuron (t_{win}):

Long-Term Potentiation (LTP): Any afferent synapse that delivered a spike before or exactly at the decision time ($t_i < t_{\text{win}}$) is deemed causal to the victory. Its weight is incremented by a fixed magnitude ($+A_+$).

Long-Term Depression (LTD): Any afferent synapse that spiked late ($t_i > t_{\text{win}}$) or failed to spike at all is classified as irrelevant or background noise. Its weight is decremented by a fixed magnitude ($-A_-$).

This approach mirrors the approximations used in digital neuromorphic ASICs, replacing transcendental operations with simple additive logical masks. All weights are subsequently clamped to a physical hardware range ($W_{\min}, W_{\max}$) to prevent sign-flipping or unbounded integer overflow.

```
applyVectorizedSTDP(T_pre, t_win, W, A_+, A_−, W_max) → W_new:
    mask_ltp ← (T_pre ≤ t_win) ∩ (T_pre ≠ ∞)
    mask_ltd ← (T_pre > t_win) ∪ (T_pre = ∞)
    W[mask_ltp] ← W[mask_ltp] + A_+
    W[mask_ltd] ← W[mask_ltd] − A_−
    W ← max(W_min, min(W, W_max))
    return W
```

Algorithm 5: The vectorized STDP weight update function

6.4.4 • COMPETITIVE SPECIALIZATION (POST-HOC HARD-WTA)

If coincidence detection is applied to all neurons simultaneously, the network suffers from catastrophic averaging—all neurons attempt to learn the most dominant spatial feature, resulting in identical, redundant weights rather than distinct feature clustering.

To force specialization without disrupting the continuous integration dynamics of the forward pass, the network enforces strict Hard WTA dynamics retroactively as a learning mask.

Upon the conclusion of the saccade, the STDP algorithm evaluates the precise timestamp of every spike. Only the single mathematically absolute first neuron to fire ($t_{\min}$) is granted permission to undergo synaptic plasticity. All competing neurons are retroactively vetoed, and their weights remain frozen for that iteration. By applying WTA post-hoc, the network isolates the learning updates without introducing chaotic voltage fluctuations during the forward inference pass. If one neuron claims a specific structural pattern (e.g., a diagonal stroke), its peers are forced to adapt to secondary patterns to win future competitions.

```
applyPostHocWTA(T_post) → (n_winner, t_win):
    t_win ← min(T_post)
    if t_win = ∞:
        return (null, ∞)
    N_tied ← findIndices(T_post = t_win)
    n_winner ← randomChoice(N_tied)
    return (n_winner, t_win)
```

Algorithm 6: Post-hoc winner-takes-all selection

6.4.5 • HOMEOSTATIC THRESHOLD ADAPTATION

Relying strictly on WTA competition introduces a secondary risk: dead neurons. A small subset of hyper-responsive neurons might win every competition, preventing the rest of the layer from ever learning. To counteract this and ensure a distributed

feature representation, the network employs a strict Homeostatic Threshold Adaptation loop mimicking the biological mechanisms detailed in Section 3.6.5.

Following every image presentation, the system regulates both the firing thresholds and the synaptic distributions of the hidden layer:

Global Threshold Decay: The adaptive thresholds ($\vartheta_{\text{adaptive}}$) of all hidden neurons are multiplied by a decay factor ($\vartheta_{\text{decay}} = 0.90$). This gradually makes quiescent, dead neurons more sensitive over time.

Winner Penalty: The single neuron that won the WTA competition receives a massive additive penalty to its adaptive threshold ($\vartheta_{\text{plus}} = 600.0$). This physically prevents it from dominating subsequent saccades, ensuring it only fires again when its highly specialized feature is prominently displayed.

Synaptic Normalization: The synaptic weights of the winning neuron are scaled to maintain a specific target L_1 norm (K_{target}). This ensures that the total synaptic drive of the neuron remains bounded, regardless of how many LTP updates it receives.

To execute unsupervised feature extraction on the MNIST dataset, the STDP rule is embedded within the saccade simulation loop. The network is initialized with randomized synaptic weights $W \sim \mathcal{U}(0, 1)$.

During the training phase, an image is presented, and the network executes a forward pass. To maintain network stability and prevent a single neuron from dominating the receptive field, we pair the Algorithm 5 rule with Hard WTA dynamics and homeostatic threshold adaptation. Only the single earliest firing neuron is permitted to learn. Following an Algorithm 5 update, the adaptive thresholds of all hidden neurons naturally decay ($\vartheta_{\text{decay}} = 0.90$). However, the single winning neuron receives a massive threshold penalty ($\vartheta_{\text{plus}} = 600.0$), mathematically suppressing it in future iterations and forcing competing neurons to specialize in distinct, non-overlapping geometric primitives. At the conclusion of the 64-tick saccade, the simulator halts, compares the timestamp arrays of the hidden and output layers, and computes the STDP weight updates in parallel before the next image is presented.

6.5 • EVALUATION METRICS

To rigorously validate the proposed SNN, the evaluation framework must bridge two distinct domains: classification effectiveness (decoding accuracy) and computational efficiency (hardware-agnostic resource usage). Because the networks undergo both supervised transfer (phase II) and unsupervised native learning (phase III), specialized metrics are required to capture the evolution of the internal representations.

6.5.1 • EFFECTIVENESS AND CLASSIFICATION PERFORMANCE

For phase II (zero-shot weight transfer), performance is measured against the standard 10,000-image MNIST test set. Since the labels are pre-defined by the source ANN, effectiveness is quantified using standard statistical tools:

Top-1 Accuracy: The percentage of images where the first output neuron to fire via the WTA mechanism matches the ground-truth label.

Confusion Matrices: Utilized to visualize the classification distribution and identify specific morphological overlaps (e.g., '4' vs. '9'). These matrices help determine if certain geometric features are disproportionately misclassified after the translation from static weights to discrete temporal decoding.

Evaluating phase III (Unsupervised STDP) requires a shift in methodology. Because the network learns without labels, output neurons do not inherently map to categorical digits; they map to geometric clusters. To generate a comparable accuracy metric, we employ a post-hoc labeling strategy:

Freezing: Synaptic plasticity is disabled (learning rate set to zero) after the STDP training phase to prevent further weight drift.

Assignment: A subset of the labeled training data is passed through the network. Each output neuron is permanently assigned the label of the digit class that most frequently triggered it to fire.

Testing: The standard 10,000-image test set is passed through the labeled network to calculate the final Top-1 accuracy.

6.5.2 • COMPUTATIONAL EFFICIENCY (HARDWARE PROXIES)

While the primary goal of neuromorphic engineering is immense energy reduction, evaluating true physical power draw ($\frac{J}{\text{inference}}$) is restricted by the use of PyTorch-based software simulators running on standard GPUs. Because these platforms incur heavy overhead simulating temporal event loops on von Neumann hardware, we abandon direct power measurements in favor of hardware-agnostic proxy metrics universally recognized in SNN literature:

SPARSITY AND SYNAPTIC OPERATIONS (SYOPS)

In a standard ANN, every forward pass requires a fixed number of Multiply And Accumulate (MAC) operations. In an SNN, computation is driven by discrete events. Because IF neurons do not require multiplication (a spike simply triggers the addition of its weight to the post-synaptic potential), MACs are replaced by simpler Synaptic Operations (SyOPs) [49].

The total computational cost for a single 64-tick saccade is estimated as:

$$\text{Total SyOPs} = \sum_{l=1}^{L} N_{\text{spikes}}^{l} \cdot F_{\text{out}}^{l}$$

Equation 21: SNN Operational Cost Proxy

Where N_{spikes} is the total spikes emitted in layer l and F_{out} is the fan-out of the neurons (number of neurons in the subsequent layer). By comparing the SNN SyOPs against the fixed MAC count of the baseline ANN, we derive a theoretical energy efficiency ratio.

TEMPORAL LATENCY METRICS

To evaluate the efficacy of the TTFS encoding, we measure the time-to-decision latency. This is defined as the exact simulation tick $t \in [0, 64)$ at which the WTA output layer reaches a decision. A lower average latency indicates a superior temporal decoder that successfully prioritizes salient information, allowing the system to theoretically power down early in the simulation window.

6.6 • EXPERIMENT SETUP AND EVALUATION PHASES

To systematically evaluate the proposed SNN architectures, the experimental framework is structured as a progressive pipeline, moving from isolated mathematical unit tests to full-scale visual classification. This approach isolates three critical neuromorphic variables: temporal integration dynamics, parameter robustness during quantization, and unsupervised feature emergence.

6.6.1 • SNN SIMULATION ENGINE

The primary testbed is a custom-built, discrete-time SNN simulator implemented in PyTorch. Unlike standard ANNs which process data in a single static glance, the SNN processes data through a dynamic temporal saccade lasting $T_{\text{max}} = 64$ ticks, requiring state variables to be tracked across the temporal dimension.

To ensure reproducibility across the network-scale evaluations (phases II and III), the baseline hardware-proxy parameters are standardized as follows. (Note: phase I overrides V_{th} to perform targeted threshold sweeps).

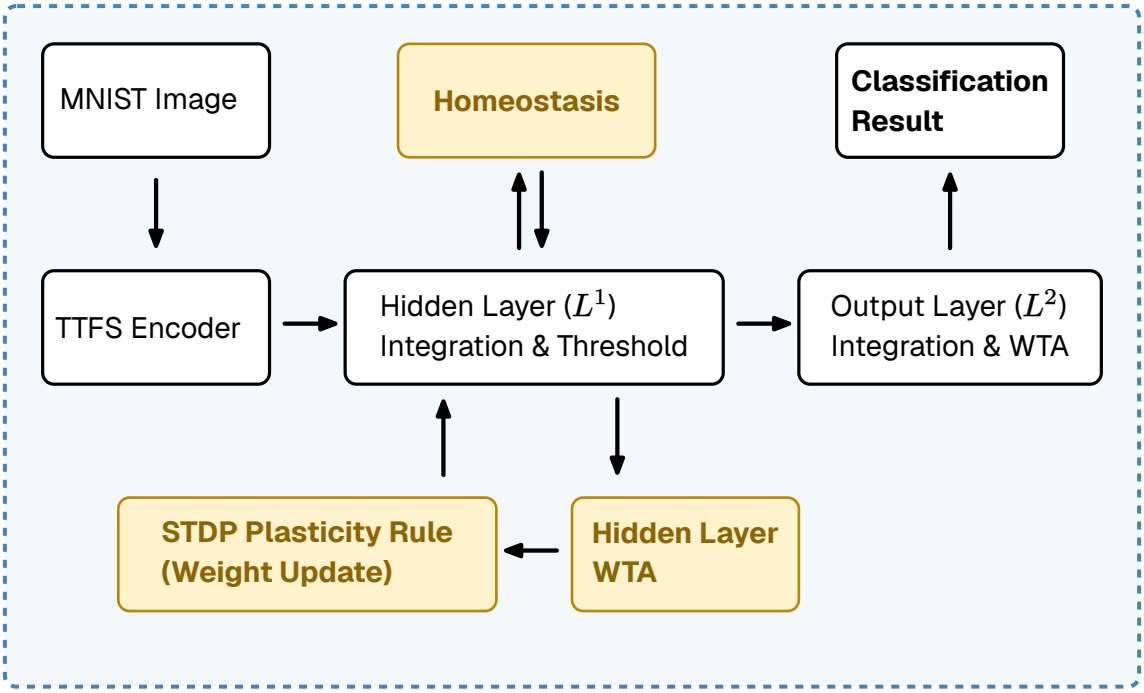

Figure 29: Software architecture and data flow of the SNN simulator. If local training is activated, the yellow modules are enabled; for pure inference, they are bypassed.

Parameter	Value	Context
$T_{\max}$	64	Total simulation ticks per image
$V_{\text{th\_C}}$	600.0 / 180.0	Baseline thresholds for model C (180.0 utilized in phase 3 inference)
Homeostasis	$\vartheta_{\text{decay}} = 0.90$ $\vartheta_{\text{plus}} = 600.0$	Adaptive threshold limits applied during STDP
Optimizer	Adam [56]	Source optimizer for phase II Baseline

6.6.2 • EVALUATION PHASES

The experiment is executed in three logical phases, building in complexity from an isolated single neuron to a fully self-organizing network.

PHASE I: TEMPORAL DYNAMICS AND THRESHOLD SWEEPS

This phase serves as a functional unit test for the isolated neuron models described in Section 6.3.2. Rather than utilizing a single static threshold, the models are subjected to synthetic spike trains across a spectrum of threshold bounds:

Saturation Regime (Low Threshold): Evaluates susceptibility to false positives when the total incoming weight vastly exceeds the threshold.

Critical Regime (Balanced Threshold): Evaluates sequence decoding when the threshold strictly requires nearly all pattern weights to coordinate.

Deficit Regime (High Threshold): Evaluates resilience when the base pattern is insufficient, testing if momentum or leak mechanics can leverage noise to force a spike.

PHASE II: THE ANN BASELINE AND ZERO-SHOT TRANSFER

Phase II benchmarks the SNN-as-Accelerator hypothesis. First, an ANN baseline is established (using a standard $784 \to 128 \to 10$ MLP without biases) to provide an ideal accuracy ceiling. Learned FP32 weights are transferred directly into the SNN. We measure the accuracy decay rate ($\Delta_{\text{Acc}} = \text{Acc}_{\text{ANN}} - \text{Acc}_{\text{SNN}}$) to quantify the information lost during this static-to-temporal translation.

PHASE III: NATIVE UNSUPERVISED LEARNING

The final phase evaluates neuromorphic self-organization. Discarding all pre-trained weights, the network is initialized randomly and exposed to the MNIST dataset strictly via unsupervised STDP.

Lateral Inhibition: A WTA mechanism is enforced; the first output neuron to fire suppresses all competitors, driving the network toward distinct feature clustering.

Homeostasis: An adaptive threshold is added to neurons that fired in the last tick to prevent single-neuron dominance and ensure a distributed feature representation across the hidden layer.

Because STDP is fundamentally unsupervised, the SNN output neurons organically form distinct feature clusters rather than mapping to pre-defined human labels (0-9). To quantify classification accuracy, a post-hoc assignment protocol is utilized. Following the unsupervised training phase, a distinct subset of the training data is passed through the frozen network. A frequency matrix M of size $N_{\text{out}} \times 10$ tracks the ground-truth label of each image that causes output neuron j to win the WTA competition. The assigned label for neuron j is then defined as $L_j = \text{argmax}\big(M_j\big)$.

7 • RESULTS

This chapter presents the empirical data gathered from the three evaluation phases. The results progress from isolated single-neuron temporal dynamics to full-network zero-shot translation, and finally to unsupervised self-organization.

7.1 • PHASE I: TEMPORAL DYNAMICS OF NEURON MODELS

Phase I evaluated the isolated response of the four neuron models to permuted temporal patterns (concordant vs. discordant) across three distinct threshold constraints: saturation (low), critical (balanced), and deficit (high). The base spatial weight sum for all trials was 300.0. The resulting output spike latencies are summarized in Table 1 and visualized in Figure 31.

Model	Regime	Threshold (ϑ)	Concordant Spike (t)	Discordant Spike (t)
Model A (IF)	Saturation	150.0	6	14
Model A (IF)	Critical	290.0	18	18
Model A (IF)	Deficit	310.0	DNF	DNF
Model B (LIF)	Saturation	140.0	6	14
Model B (LIF)	Critical	220.0	DNF	18
Model B (LIF)	Deficit	310.0	DNF	DNF
Model C (Linear Ramp)	Saturation	500.0	6	18
Model C (Linear Ramp)	Critical	1300.0	10	DNF
Model C (Linear Ramp)	Deficit	2000.0	DNF	DNF
Model D (State Discount)	Saturation	100.0	2	14
Model D (State Discount)	Critical	200.0	18	DNF
Model D (State Discount)	Deficit	310.0	DNF	DNF

Table 1: Output spike latencies across varying threshold regimes. DNF indicates the neuron Did Not Fire within the $T_{\max} = 64$ saccade window.

Under the low threshold regime, all models successfully fired early. Predictably, when the strongest inputs arrived first (concordant), all models reached the threshold significantly faster than when the strongest inputs arrived last.

However, when thresholds were critically tuned to perfectly match the total spatial weight of the spike train (balanced), model behaviors diverged significantly. Model A (IF) demonstrated complete temporal blindness, firing at $t = 18$ regardless of the input order. Model B (LIF) acted as a late-spike coincidence detector; the early spikes decayed before the threshold could be reached, meaning it only fired when the strongest input came last ($t = 18$). Conversely, both model C (linear ramp) and model D (state discount) demonstrated strong preference for early stimuli, firing faster when the strongest inputs arrived first.

Finally, under the deficit regime (where the threshold was set strictly higher than the total weight of the stimuli), all architectures failed to fire. For model C specifically, the temporal momentum was insufficient to overcome the high threshold before the 10-tick coincidence timer expired, aggressively zeroing the state variables.

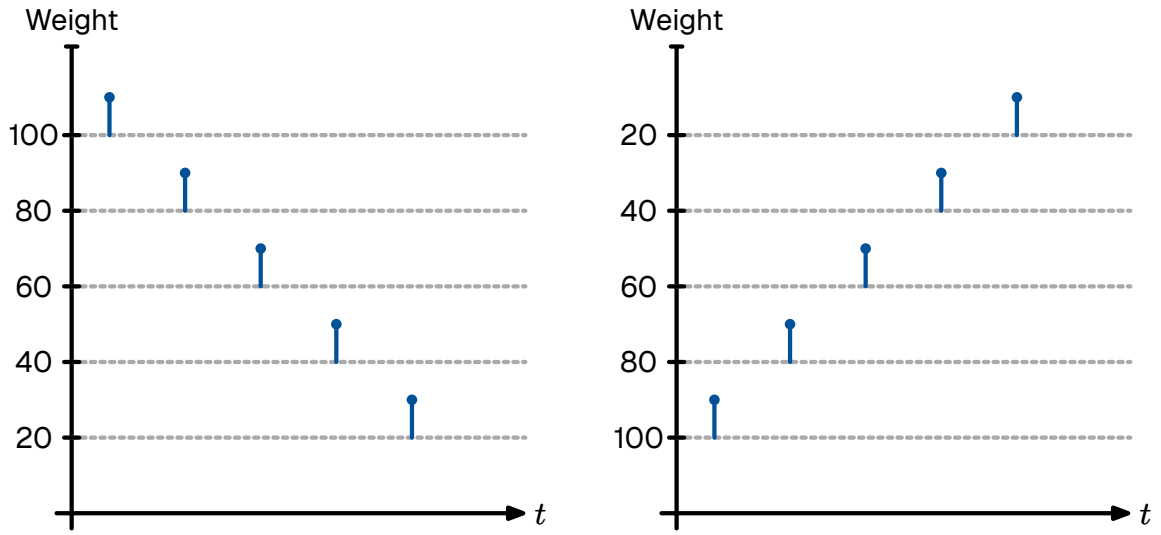

Figure 30: Phase I input spike distributions. Strongest first (concordant) on the left, weakest first (discordant) on the right.

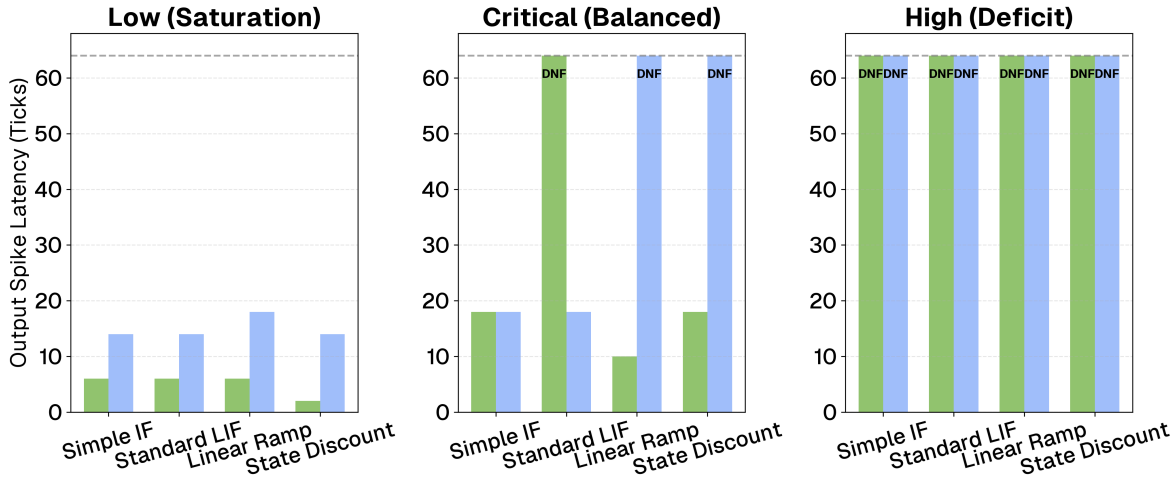

Figure 31: Input spike distributions and resulting temporal dynamics across the three threshold regimes.

Because phase II and phase III require an architecture capable of processing variable input densities while maintaining strict feature discriminability, the selected neuron model must demonstrate distinct temporal separability under critical constraints. As demonstrated in Table 1, while both model C and model D successfully suppressed the discordant pattern under the critical regime, model C (current-accumulating linear ramp) achieved a significantly faster concordant response ($t = 10$ versus $t = 18$) while remaining relatively cheap to compute. By successfully leveraging temporal momentum to decode the spike order efficiently, model C was utilized as the standardized spiking architecture for the network-scale evaluations in the subsequent phases.

7.2 • PHASE II: ZERO-SHOT WEIGHT TRANSFER

Phase II evaluated the accuracy decay incurred when translating a conventionally trained ANN to the selected SNN (model C) without any intermediate retraining.

The initial ANN baseline was trained for 1000 epochs using the Adam optimizer [56]. To satisfy the strict mapping requirements of the SNN, the network was trained with all bias terms disabled ($b = 0$). The baseline model achieved a maximum test accuracy of 98.40% on the standard MNIST test set, establishing a strong, expected baseline for a fully connected topology.

To rigorously isolate the sources of information loss during the zero-shot transfer, the SNN was loaded with the scaled FP32 weights from the ANN. This isolates the temporal penalty—the loss incurred strictly by moving from a static dot-product activation to a discrete, TTFS integration. The SNN utilized the model C (linear ramp) architecture over a $T_{\max} = 64$ saccade window. The results are summarized in Table 2.

Model Configuration	Accuracy (%)
ANN (Static FP32, Bias=False)	98.40%
SNN (Spiking Model C, FP32)	94.50%

Table 2: Performance degradation breakdown. The Temporal Penalty isolates the loss of TTFS encoding without any weight alteration besides scaling.

The zero-shot translation achieved a remarkable 94.50% accuracy, demonstrating that the momentum-based decoding of model C successfully preserves the spatial hierarchies learned by the ANN. Figure 33 visualizes the classification distribution of the SNN. The network maintained strong structural clustering, effectively utilizing the algorithmic WTA mechanism to finalize classifications.

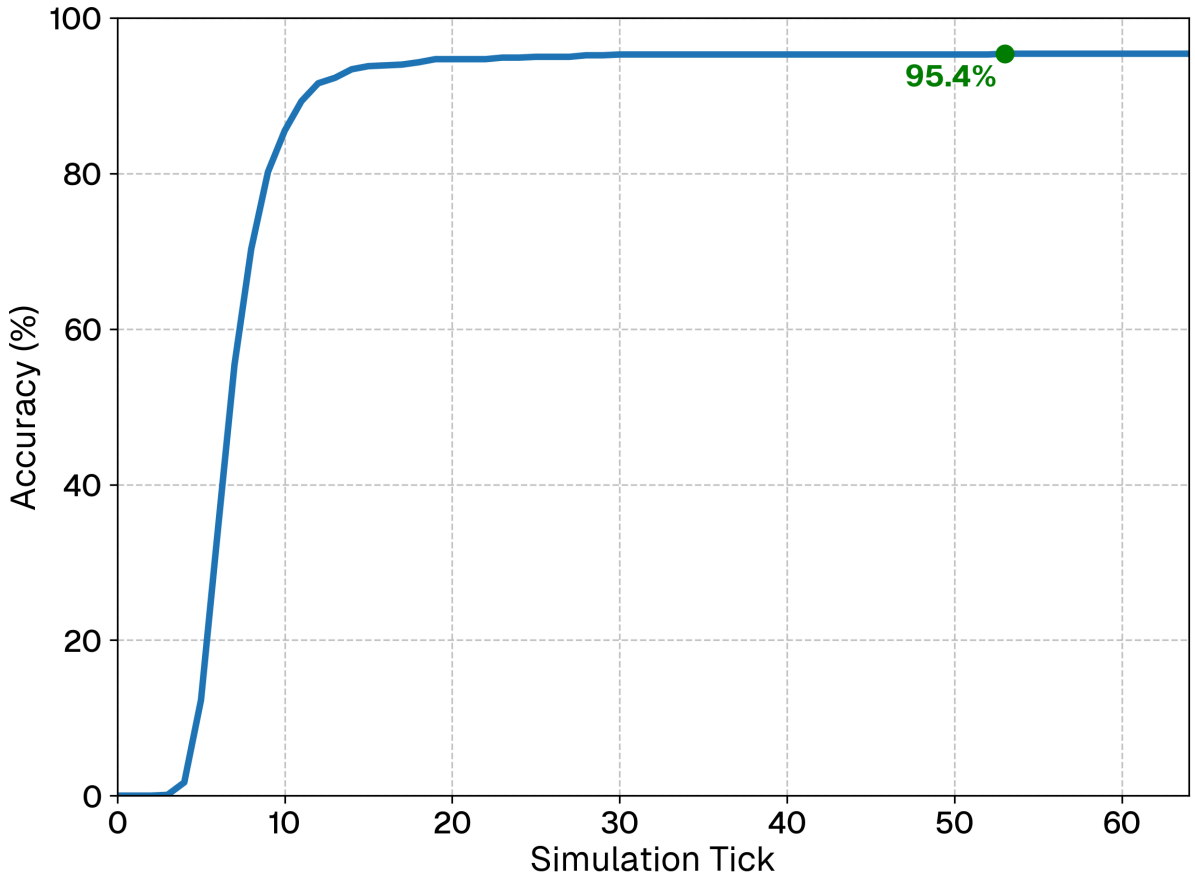

Figure 32: Cumulative classification accuracy over the 64-tick temporal window. Accuracy follows a sigmoid-like S-curve, showing that the majority of correct classifications are finalized early in the saccade.

As visualized in Figure 32, the network exhibits a rapid S-curve profile in its decision-making process. Accuracy remains at 0% for the first three ticks as the initial wavefront of spikes propagates through the hidden layer. Between ticks 5 and 15, there is a sharp surge in accuracy, capturing over 80% of the test set, followed by a distinct plateau after tick 20.

To evaluate the computational efficiency of the architecture, Table 3 compares the dense MAC operations of the baseline ANN against the sparse SyOPs of the TTFS implementation.

The zero-shot SNN achieved a mean time-to-decision of 8.4 ticks out of the maximum 64. By halting integration upon classification, the network processed an average of only 15,021 SynOps per image, yielding an 85.2% reduction in computational workload compared to the static ANN baseline.

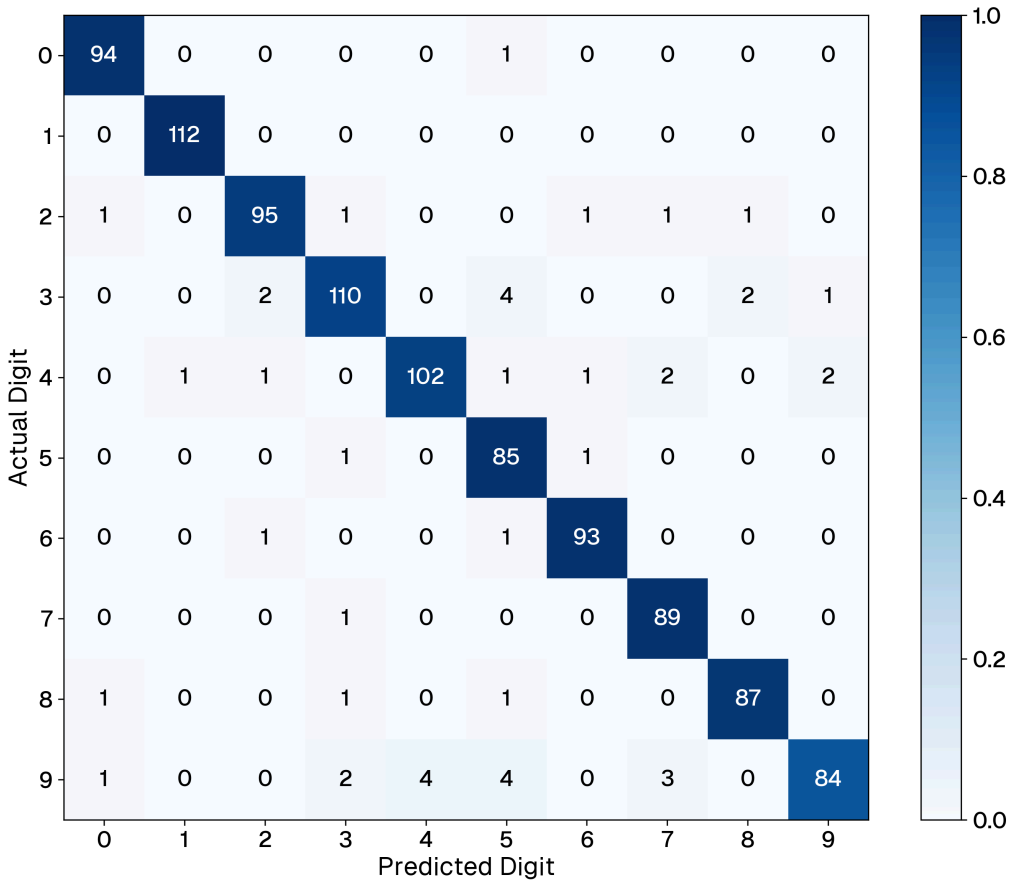

Figure 33: Confusion matrix for the zero-shot SNN on the MNIST test set. The strong diagonal indicates the temporal integration preserved the original ANN decision boundaries.

Architecture	Metric Target	Avg. Latency	Avg. Operations / Image	Compute Reduction
ANN (Static FP32)	Dense MACs	N/A (Static)	101,632 MACs	0%
SNN (Spiking FP32)	Sparse SynOps	8.4 Ticks	15,021 SynOps	85.2%

Table 3: Computational cost comparison. The SNN significantly reduces operations by leveraging temporal early-exit sparsity.

7.3 • PHASE III: UNSUPERVISED NATIVE LEARNING

Phase III evaluated the capacity of the network to self-organize without gradient descent, relying purely on local STDP based rules, WTA competition, and homeostatic threshold adaptation. After processing the unsupervised training set, the network

achieved an overall accuracy of 44.3% (Table 4). While this represents a substantial drop from the 94.50% accuracy achieved via supervised weight transfer in phase II, this result serves as a successful proof-of-concept for unsupervised geometric clustering. As will be thoroughly analyzed in Section 8, this performance gap highlights the representational boundaries of single-layer competitive topologies rather than a failure of the STDP mechanism itself.

Model Configuration	Accuracy (%)
SNN (Spiking Model C, STDP, WTA)	44.3%

Table 4: Top-1 classification accuracy after fully unsupervised STDP training.

Architecture	Metric Target	Avg. Latency	Avg. Operations / Image	Compute Reduction
SNN (Spiking STDP)	Sparse SynOps	8.3 Ticks	12,954 SynOps	87.2%

Table 5: Computational cost comparison for the unsupervised SNN. The self-organized network maintained the early-exit sparsity observed in phase II.

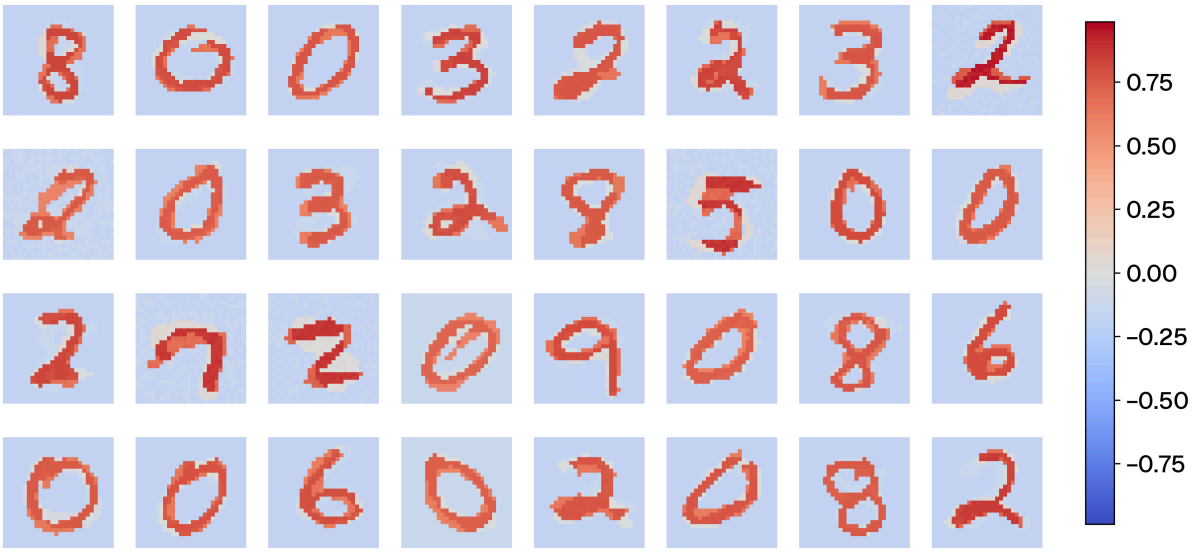

Figure 34: Hidden layer receptive fields (W^1) after unsupervised STDP training. The hard WTA competition forces vector quantization, resulting in recognizable, holistic templates of entire digits.

The self-organized network achieved a 44.3% accuracy, representing a drop from the 98.40% baseline achieved via supervised backpropagation. However, the network maintained a computational profile comparable to the weight-transferred architecture: an average decision latency of 8.3 ticks and 12,954 SyOPs per image. As detailed in Table 5, this yields an 87.2% reduction in operations relative to the ANN baseline. Furthermore, there was a marginal improvement in sparsity over phase II, increasing from 85.2% to 87.2%.

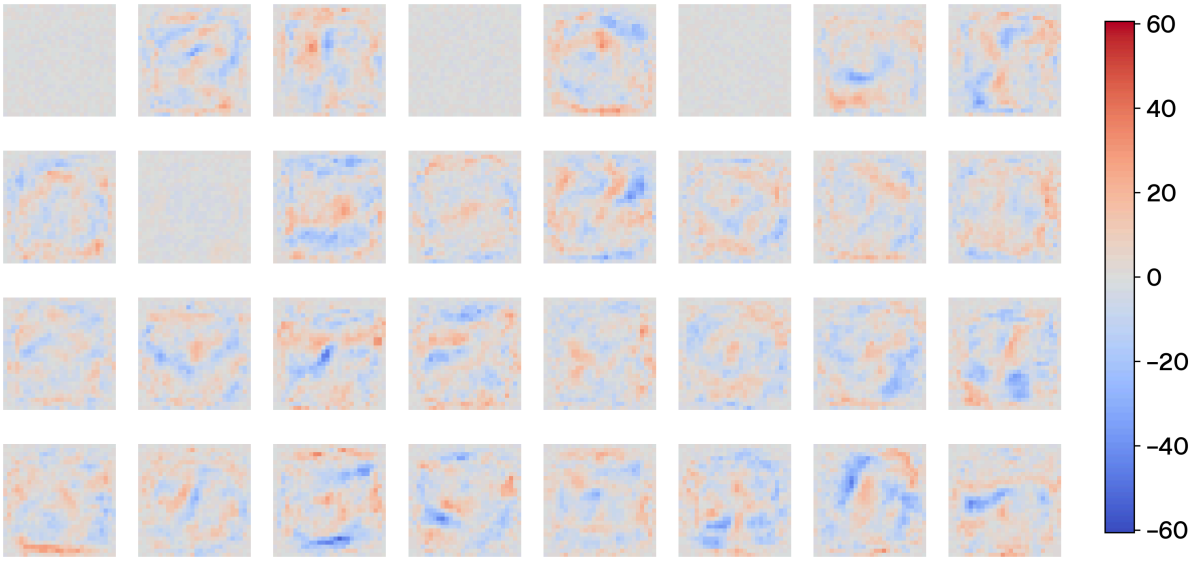

Figure 35: Hidden layer receptive fields (W^1) extracted from the baseline ANN. Gradient descent optimizes for a distributed representation, resulting in abstract, non-visual basis functions.

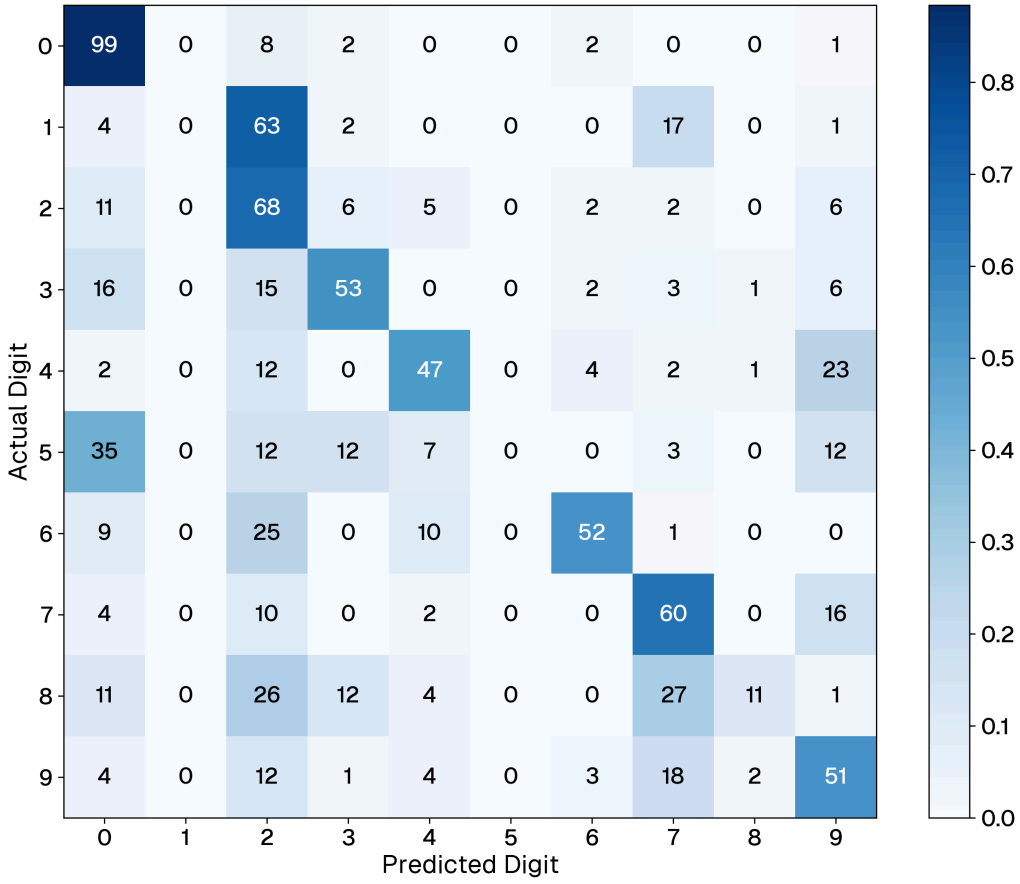

Figure 36: Confusion matrix for the unsupervised STDP network. While most classes are somewhat successfully clustered, the network struggles with certain classes (e.g., classifying '5's as '0's or '1's as '2's).

Inspection of the confusion matrix in Figure 36 reveals that performance is highly non-uniform across digit classes. Visually simple digits—particularly ‘0’—are classified with high confidence. Conversely, classification failures are concentrated among specific digit pairs and groups. The ‘5’/‘0’ confusion is the most pronounced.

8 • DISCUSSION

While the experimental results validate the core principles of TTFS encoding and spiking integration, extrapolating these methods from benchmarks to real-world deployment reveals significant engineering bottlenecks. This chapter critically analyzes the limitations observed during the implementation phase, specifically regarding data encoding, spatial representation, architectural translation, and hardware constraints.

8.1 • EVALUATION OF NEURON MODELS

The empirical observations from the phase I threshold sweeps reveal a fundamental tension between traditional biological neuron models and the specific requirements of TTFS decoding. By systematically shifting the threshold bounds, the isolated unit tests exposed the limitations of standard integration strategies under strict temporal constraints, while validating the proposed momentum-based and state-discounting architectures.

Under saturation (low threshold) conditions, all models successfully fired earlier for the concordant pattern. As hypothesized, this early firing is driven by magnitude accumulation rather than strict sequence decoding; however, this is not a failure of the models, but rather a functional property of threshold dynamics. When presented with an overwhelming density of salient early inputs, the simple IF (model A) and LIF (model B) models rapidly accumulate the heavily weighted spikes and cross the lowered threshold. In practical applications, this magnitude-driven early exit is a highly usable feature, allowing the network to make rapid decisions when sensory evidence is abundant and unambiguous.

The divergence between the models becomes glaringly apparent, however, in the critical (balanced) regime, where magnitude alone is insufficient and true temporal decoding is required. When forced to accumulate the entire sequence, model A completely failed to differentiate order, firing simultaneously for both patterns. More critically, model B demonstrated a catastrophic misalignment with TTFS principles. Due to the interaction between temporal density and the exponential leak, early weak spikes establish a baseline potential, while massive late-arriving spikes push the neuron over the threshold immediately before they decay. Consequently, the standard LIF inherently favors strongest-last sequences, actively fighting the strongest-first priority of TTFS encoding.

In contrast, the custom architectures exhibited highly desirable temporal dynamics for rank-order decoding. The state-dependent discount (model D) achieved perfect selective filtering under the critical regime, firing for the concordant pattern while completely suppressing the discordant one. Crucially, the current-accumulating

linear ramp (model C) successfully mirrored this selective behavior without the computational overhead of continuous exponential calculations. By combining integration momentum with a strict coincidence window ($T_{\text{window}} = 10$ ticks), model C effectively gated its own firing. If the temporal momentum was insufficient to reach the threshold rapidly, the timer expired and aggressively zeroed the state variables. This bounded integration window prevents the network from diverging toward hardware saturation and ensures the neuron remains highly selective to tight temporal clusters, achieving the functional benefits of model D through purely linear mechanics.

8.2 • VIABILITY OF WEIGHT TRANSFER AND EARLY EXIT

Phase II demonstrated that momentum-based TTFS integration can recover 94.50% of the ANN's classification accuracy via zero-shot weight transfer. The rapid S-curve profile and subsequent plateau observed in the cumulative accuracy (Figure 32) are a direct result of this encoding scheme. Because the most salient (high-intensity) pixels arrive first, they provide sufficient evidence for the momentum-based integration to cross the threshold for clear, high-contrast digits early in the process. The plateau after tick 20 indicates that the integration of later, lower-intensity background pixels yields diminishing returns. Ultimately, this validates the early-exit hypothesis, demonstrating that the network achieves near-peak performance while the majority of the temporal window remains unprocessed.

While the direct parameter transfer for the baseline FCFN was remarkably straightforward—functioning essentially as a literal one-to-one copy due to the network's simple topology and the deliberate omission of static biases—extrapolating this translation to state-of-the-art architectures exposes significant engineering friction. Mapping modern CNNs or recurrent topologies to a spiking substrate is decidedly not a direct process. Continuous networks inherently rely on static bias shifts, recurrent continuous states, and abstract mathematical operations like Max Pooling. Translating these mechanisms requires profound structural concessions: mathematical pooling must be replaced with dynamic lateral inhibition, shared convolutional kernels require complex physical unrolling across the neuromorphic array, and continuous bias terms necessitate the artificial injection of constant background spike trains. This dictates that while zero-shot transfer is highly viable for simple topologies, scaled SNNs cannot merely be converted ANNs; they require network topologies natively co-designed for event-driven dynamics.

Furthermore, evaluating this sparse, early-exit algorithm on conventional GPUs introduces a significant sparsity paradox. Standard deep learning frameworks and GPU architectures are heavily optimized for dense, contiguous matrix multiplications. In the SNN simulation loop, while sparsity is enforced via boolean masking, the GPU

Arithmetic Logic Units (ALUs) typically still execute the underlying floating-point multiplication cycle. When combined with the temporal loop overhead, the SNN simulator ironically consumes more absolute clock cycles on a GPU than the continuous ANN baseline. This paradox highlights that realizing the calculated SyOPs savings requires deployment on native event-driven neuromorphic ASICs or FPGAs, which utilize AER to asynchronously route data only when a spike physically occurs.

8.3 • LIMITATIONS OF NATIVE UNSUPERVISED LEARNING

Phase III demonstrated that a 128-neuron TTFS network, trained exclusively via local STDP, successfully self-organizes into a functional classifier. Despite the drop in accuracy to 44.3% compared to supervised baselines, this definitively proves that the local, event-driven rule successfully clustered geometric features without a global error signal. Furthermore, the slightly sparser weight distribution (87.2%) confirms the hypothesis that STDP naturally depresses irrelevant synapses. Because the computational profile remained comparable to phase II, it proves that early-exit temporal behavior is a structural property of TTFS, preserved independently of how the weights were learned.

However, the significant accuracy gap relative to supervised models is largely attributable to three interacting architectural constraints: plasticity dynamics, spatial representation, and competitive mechanisms.

First, a fundamental tension exists between the network's learning velocity and its stability. Unsupervised, local learning rules require aggressive hyperparameter tuning. High plasticity rates cause rapid catastrophic forgetting—overwriting previously learned geometric features with novel stimuli—while low rates fail to converge on meaningful representations. This instability directly suppresses classification accuracy, as neurons partially specialized toward one digit class are easily corrupted by subsequent exposures to morphologically similar digits.

Second, while template matching provides a robust mechanism for clustering simple, orthogonal shapes like '0' and '1', it fundamentally struggles with morphological overlaps. As seen in the confusion matrix (Figure 36), most digits share significant regions of overlapping pixel energy. Because the SNN relies on absolute temporal sequences rather than deep, spatially bounded feature extraction, it surrenders to partial template matches early in the temporal window.

This spatial vulnerability is exacerbated by the global WTA mechanism. If an input image contains multiple distinct spatial features, a global WTA allows the network to respond to only the single most salient feature, discarding complementary evidence. Biological systems address this via local lateral inhibition and deep hierarchical layers, where an active neuron suppresses only its immediate spatial neighbors.

Ultimately, this is the defining limitation of the single-layer, fully connected WTA topology: while 128 neurons can cover the primary digit archetypes, they cannot simultaneously represent the intra-class morphological variance required to resolve ambiguous cases.

8.4 • FUTURE WORK

The findings and limitations discussed in this thesis present several promising avenues for future research in neuromorphic engineering, ranging from algorithmic enhancements to physical hardware deployment.

8.4.1 • NATIVE TEMPORAL DATASETS AND THE CONTRAST BOTTLENECK

Transitioning the experimental framework from static images to native temporal datasets, such as N-MNIST or DVS Gesture datasets, is essential to fully exploit the asynchronous dynamics of the evaluated neuron models. In naturalistic environments, robust vision relies on local contrast rather than absolute luminance. However, computing local contrast mathematically requires evaluating the relative difference between the lightest and darkest regions of a receptive field. Under a pure TTFS encoding scheme, this presents a severe latency bottleneck: while the brightest pixels trigger immediate spikes, the darkest pixels are encoded as the latest possible spikes—or are masked out entirely as background noise. Forcing the network to wait for these late-arriving dark spikes to resolve a contrast ratio entirely negates the early-exit speed advantage that makes TTFS computationally appealing in the first place. Offloading this specific computation to dedicated neuromorphic sensors, such as dynamic vision sensors that natively compute and output logarithmic intensity differences as instantaneous events, bypasses this temporal delay problem and substantially improves robustness.

8.4.2 • SPATIALLY BOUNDED ARCHITECTURES AND CONVOLUTION

A major finding of phase III was that deploying unsupervised STDP within a fully connected topology inherently drives the hidden layer toward holistic template matching. Because every neuron observes the entire visual field and competes globally via a single WTA mechanism, the network learns rigid, full-digit archetypes that fundamentally struggle with translation, scaling, and morphological overlap.

Replacing this global topology with a spiking CNN inspired architecture represents a critical next step. By utilizing shared, localized receptive fields, a CNN would introduce much-needed translation invariance—allowing the network to learn discrete geometric primitives (such as individual edges or curves) rather than entire, inflexible digits. Crucially, this architectural shift would allow lateral inhibition to

be applied locally rather than globally. An active neuron would only suppress its immediate spatial neighbors within a specific feature map, enabling the network to simultaneously extract multiple distinct features from a single image without catastrophic self-suppression.

8.4.3 • STRUCTURAL PLASTICITY AND DYNAMIC TOPOLOGIES

While dynamic activation sparsity struggles to realize physical efficiency gains on traditional GPUs, structural sparsity offers a highly viable mitigation strategy. Future iterations should move beyond static, fully connected architectures and introduce aggressive structural plasticity following the unsupervised STDP learning phase. By permanently severing consistently depressed synapses (pruning) and mapping the remaining connections into block-sparse tensor formats, simulators and specialized accelerators can mathematically bypass zero-weight memory reads, directly yielding physical memory bandwidth reductions.

Furthermore, advanced learning algorithms could introduce dynamic synaptogenesis—the biologically inspired growth of new synapses—to continuously tweak the network topology over time. This dynamic rewiring would allow the network to escape the constraints of its initial random initialization. By physically growing new pathways to under-utilized neurons, the system could organically distribute the representational load, directly mitigating the dead-neuron and dominant-neuron problems observed under strict global WTA competition.

8.4.4 • HARDWARE DEPLOYMENT AND NATIVE SUBSTRATES

Ultimately, the theoretical energy efficiency of neuromorphic algorithms is strictly bounded by the physical hardware substrate. While GPU simulations are indispensable for algorithmic validation, porting the verified TTFS algorithms onto dedicated event-driven silicon, such as Intel Loihi or IBM TrueNorth, is essential to empirically measure true joule-per-inference energy consumption. These digital neuromorphic ASICs and custom FPGAs utilize asynchronous routing protocols like AER to physically realize the communication sparsity demonstrated in this thesis, ensuring that power is only drawn when a spike actively traverses the bus.

Looking further ahead, the ultimate physical realization of the neuromorphic paradigm requires transitioning away from digital SRAM entirely. Emerging non-volatile memory technologies, such as memristors and spintronics, offer the ability to implement high-density analog crossbar arrays that perfectly co-locate synaptic weights with computational logic gates. Until these analog compute-in-memory substrates overcome current manufacturing challenges regarding device mismatch and noise to become commercially viable, massively parallel digital ASICs and FPGAs remain the most tractable bridge to realizing the paradigm-shifting energy efficiency promised by neuromorphic engineering.

9 • CONCLUSION

The development of modern deep learning has achieved unprecedented performance, mastering capabilities once thought to be the exclusive domain of biological minds. Yet, as demonstrated throughout this thesis, the prevailing strategy of scaling dataset volumes and computational power is approaching strict physical and economic boundaries. The massive energy consumption and carbon footprints required to train state-of-the-art models expose a fundamental flaw: current artificial intelligence remains intrinsically bottlenecked by the synchronous, continuous-value matrix multiplications demanded by the von Neumann architecture.

The central premise of this work is that this inefficiency is not merely an engineering hurdle that can be solved by shrinking silicon transistors, but a paradigmatic issue. The biological brain demonstrates that high-level intelligence, real-time adaptation, and complex reasoning are physically possible on an energy budget of roughly 20 watts. To close the immense gap between this biological reality and the megawatt-scale GPU clusters driving modern AI, we must fundamentally reimagine how information is encoded and processed.

Taken together, the three experimental phases of this thesis trace a coherent arc from isolated neuron dynamics to self-organizing network behavior, investigating whether shifting toward asynchronous, event-driven sparse computation can measurably reduce this computational footprint.

9.1 • ADDRESSING THE OBJECTIVES

To systematically evaluate the success of the proposed neuromorphic implementations, the following subsections revisit the primary research objectives established at the outset of this thesis, assessing how effectively the evaluated architectures met their specific goals.

9.1.1 • SPARSE EFFICIENT COMPUTING

The application of TTFS encoding successfully compressed the static visual data of MNIST images into a sparse, priority-driven temporal queue. By shifting from continuous activations to discrete events, the network achieved an 85.2% reduction in theoretical synaptic operations (SyOPs) relative to the dense MAC operations of the conventional ANN baseline. Crucially, this was driven by a temporal early-exit mechanism, confirming that event-driven algorithms can drastically reduce workload by dynamically allocating compute only to regions of high information density.

9.1.2 • NEURON MODEL EVALUATION

Through systematic threshold sweeps, this research established that strict adherence to classical biological models, such as the standard LIF, can be detrimental to engineered temporal networks due to their active penalization of early, high-salience spikes. The proposed current-accumulating linear ramp (model C) emerged as a necessary engineering correction, successfully decoding the temporal hierarchy through unbounded integration momentum. This highlights that biological inspiration must be selectively adapted to serve specific computational objectives.

9.1.3 • INFERENCE VIA WEIGHT TRANSFER

To quantify the accuracy penalty of translating spatial models to temporal domains, the phase II zero-shot transfer of FP32 weights achieved a 94.50% top-1 accuracy on MNIST. By incurring a temporal penalty of only 3.9 percentage points relative to the baseline, this result successfully proves that complex decision boundaries learned by standard gradient descent can be faithfully recovered from a purely temporal spike ordering, validating hybrid deployment pipelines for edge devices.

9.1.4 • NATIVE UNSUPERVISED LEARNING

In phase III, the network was trained exclusively via local STDP and global WTA competition, achieving 44.3% top-1 accuracy without labeled data or global error signals. While the representational constraints of the single-layer topology and the lack of spatial invariance suppressed overall accuracy, the emergence of holistic digit templates confirms that local, hebbian-style plasticity rules can independently extract statistically meaningful geometric features from raw visual input.

9.2 • BROADER SIGNIFICANCE AND FINAL REMARKS

This thesis precisely characterizes both the immense potential and the current limits of the neuromorphic paradigm. The theoretical efficiency advantages of neuromorphic algorithms are real and measurable at the algorithmic level. However, they are presently obscured by the mismatch between event-driven computation and the von Neumann hardware on which they are simulated. The “Sparsity Paradox”—where GPUs expend physical energy calculating boolean-masked zeros—is not a flaw in the neuromorphic approach, but the defining engineering challenge of the field.

The historical trajectory of computing is clear. Current deep learning acts as a powerful statistical correlation engine, but it lacks the real-time, highly efficient adaptability of the biological mind. As native neuromorphic substrates mature—evolving from digital platforms like Intel’s Loihi toward hyper-dense analog crossbars—the algorithmic foundations validated in this thesis will become the blueprints for next-

generation systems. These future machines will physically co-locate memory and computation, route discrete data packets only when sensory changes occur, and learn continuously from unlabeled environmental streams. Closing the gap between a 20-watt biological brain and modern computing infrastructure is no longer a question of whether the underlying principles of sparse temporal computing are sound; it is now entirely a question of when our hardware architectures will evolve to embody them.

BIBLIOGRAPHY

[1] G. Team *et al.*, "Gemini: A Family of Highly Capable Multimodal Models." Accessed: Apr. 18, 2026. [Online]. Available: http://arxiv.org/abs/2312.11805

[2] J. Jumper *et al.*, "Highly accurate protein structure prediction with AlphaFold," *Nature*, vol. 596, no. 7873, pp. 583–589, Aug. 2021, doi: 10.1038/s41586-021-03819-2.

[3] D. Silver *et al.*, "Mastering the game of Go with deep neural networks and tree search," *Nature*, vol. 529, no. 7587, pp. 484–489, Jan. 2016, doi: 10.1038/nature16961.

[4] E. Strubell, A. Ganesh, and A. McCallum, "Energy and Policy Considerations for Deep Learning in NLP," in *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*, Florence, Italy: Association for Computational Linguistics, 2019, pp. 3645–3650. doi: 10.18653/v1/P19-1355.

[5] R. Geirhos *et al.*, "Shortcut learning in deep neural networks," *Nature Machine Intelligence*, vol. 2, no. 11, pp. 665–673, Nov. 2020, doi: 10.1038/s42256-020-00257-z.

[6] S. Laughlin, "Energy as a constraint on the coding and processing of sensory information," *Current Opinion in Neurobiology*, vol. 11, no. 4, pp. 475–480, Aug. 2001, doi: 10.1016/S0959-4388(00)00237-3.

[7] M. Davies *et al.*, "Loihi: A Neuromorphic Manycore Processor with On-Chip Learning," *IEEE Micro*, vol. 38, no. 1, pp. 82–99, Jan. 2018, doi: 10.1109/MM.2018.112130359.

[8] S. B. Furber, F. Galluppi, S. Temple, and L. A. Plana, "The SpiNNaker Project," *Proceedings of the IEEE*, vol. 102, no. 5, pp. 652–665, May 2014, doi: 10.1109/JPROC.2014.2304638.

[9] M. Glickstein, "Golgi and Cajal: The neuron doctrine and the 100th anniversary of the 1906 Nobel Prize," *Current Biology*, vol. 16, no. 5, pp. R147–R151, Mar. 2006, doi: 10.1016/j.cub.2006.02.053.

[10] W. Waldeyer, "Ueber einige neuere Forschungen im Gebiete der Anatomie des Centralnervensystems," *Deutsche medicinische Wochenschrift*, vol. 17, pp. 1352–1356, 1891.

[11] W. S. McCulloch and W. Pitts, "A logical calculus of the ideas immanent in nervous activity," *The Bulletin of Mathematical Biophysics*, vol. 5, no. 4, pp. 115–133, Dec. 1943, doi: 10.1007/BF02478259.

[12] D. O. Hebb, *The organization of behavior: a neuropsychological theory*. Mahwah, N.J: L. Erlbaum Associates, 2002.

[13] C. J. Shatz, "The Developing Brain," *Scientific American*, vol. 267, no. 3, pp. 60–67, Sept. 1992, doi: 10.1038/scientificamerican0992-60.

[14] F. Rosenblatt, "The perceptron: A probabilistic model for information storage and organization in the brain.," *Psychological Review*, vol. 65, no. 6, pp. 386–408, 1958, doi: 10.1037/h0042519.

[15] "New Navy Device Learns by Doing: Psychologist Shows Embryo of Computer Designed to Read and Grow Wiser," *The New York Times*, July 1958.

[16] M. Minsky and S. Papert, *Perceptrons: an introduction to computational geometry*, Expanded ed. Cambridge, Mass: MIT Press, 1988.

[17] M. Minsky, "Steps toward Artificial Intelligence," *Proceedings of the IRE*, vol. 49, no. 1, pp. 8–30, Jan. 1961, doi: 10.1109/JRPROC.1961.287775.

[18] D. E. Rumelhart, G. E. Hinton, and R. J. Williams, "Learning representations by back-propagating errors," *Nature*, vol. 323, no. 6088, pp. 533–536, Oct. 1986, doi: 10.1038/323533a0.

[19] A. Krizhevsky, I. Sutskever, and G. E. Hinton, "ImageNet classification with deep convolutional neural networks," *Communications of the ACM*, vol. 60, no. 6, pp. 84–90, May 2017, doi: 10.1145/3065386.

[20] K. He, X. Zhang, S. Ren, and J. Sun, "Deep Residual Learning for Image Recognition," in *2016 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, Las Vegas, NV, USA: IEEE, June 2016, pp. 770–778. doi: 10.1109/CVPR.2016.90.

[21] A. Vaswani *et al.*, "Attention Is All You Need." Accessed: Apr. 18, 2026. [Online]. Available: http://arxiv.org/abs/1706.03762

[22] V. Sze, Y.-H. Chen, T.-J. Yang, and J. Emer, "Efficient Processing of Deep Neural Networks: A Tutorial and Survey." Accessed: Apr. 18, 2026. [Online]. Available: http://arxiv.org/abs/1703.09039

[23] M. Horowitz, "1.1 Computing's energy problem (and what we can do about it)," in *2014 IEEE International Solid-State Circuits Conference Digest of Technical Papers (ISSCC)*, San Francisco, CA, USA: IEEE, Feb. 2014, pp. 10–14. doi: 10.1109/ISSCC.2014.6757323.

[24] C. Mead, "Neuromorphic electronic systems," *Proceedings of the IEEE*, vol. 78, no. 10, pp. 1629–1636, Oct. 1990, doi: 10.1109/5.58356.

[25] M. A. Mahowald and C. Mead, "The Silicon Retina," *Scientific American*, vol. 264, no. 5, pp. 76–82, May 1991, doi: 10.1038/scientificamerican0591-76.

[26] E. R. Kandel, J. Koester, S. Mack, and S. Siegelbaum, Eds., "Principles of neural science." McGraw Hill, New York, 2021.

[27] A. M. Getz *et al.*, "High-resolution imaging and manipulation of endogenous AMPA receptor surface mobility during synaptic plasticity and learning," *Science Advances*, vol. 8, no. 30, p. eabm5298, July 2022, doi: 10.1126/sciadv.abm5298.

[28] W. Gerstner, W. M. Kistler, R. Naud, and L. Paninski, *Neuronal dynamics: from single neurons to networks and models of cognition*. Cambridge, United Kingdom: Cambridge University Press, 2014.

[29] B. Metcalfe, "Action potentials recorded from the L5 dorsal rootlet of rat using a multiple electrode array." Accessed: Jan. 08, 2026. [Online]. Available: https://data.mendeley.com/datasets/ybhwtngzmm/1

[30] A. L. Hodgkin and A. F. Huxley, "A quantitative description of membrane current and its application to conduction and excitation in nerve," *The Journal of Physiology*, vol. 117, no. 4, pp. 500–544, Aug. 1952, doi: 10.1113/jphysiol.1952.sp004764.

[31] H. Markram, "The Blue Brain Project," *Nature Reviews Neuroscience*, vol. 7, no. 2, pp. 153–160, Feb. 2006, doi: 10.1038/nrn1848.

[32] E. Izhikevich, "Simple model of spiking neurons," *IEEE Transactions on Neural Networks*, vol. 14, no. 6, pp. 1569–1572, Nov. 2003, doi: 10.1109/TNN.2003.820440.

[33] W. Gerstner and R. Naud, "How Good Are Neuron Models?," *Science*, vol. 326, no. 5951, pp. 379–380, Oct. 2009, doi: 10.1126/science.1181936.

[34] E. D. Adrian and D. W. Bronk, "The discharge of impulses in motor nerve fibres: Part II. The frequency of discharge in reflex and voluntary contractions," *The Journal of Physiology*, vol. 67, no. 2, pp. 9–151, Mar. 1929, doi: 10.1113/jphysiol.1929.sp002557.

[35] S. Thorpe, D. Fize, and C. Marlot, "Speed of processing in the human visual system," *Nature*, vol. 381, no. 6582, pp. 520–522, June 1996, doi: 10.1038/381520a0.

[36] R. V. Rullen and S. J. Thorpe, “Rate Coding Versus Temporal Order Coding: What the Retinal Ganglion Cells Tell the Visual Cortex,” *Neural Computation*, vol. 13, no. 6, pp. 1255–1283, June 2001, doi: 10.1162/08997660152002852.

[37] E. Basso, M. Arai, and Y. Dabaghian, “Gamma Synchronization Influences Map Formation Time in a Topological Model of Spatial Learning,” *PLoS computational biology*, vol. 12, no. 9, p. e1005114, Sept. 2016, doi: 10.1371/journal.pcbi.1005114.

[38] D. H. Hubel and T. N. Wiesel, “Receptive fields, binocular interaction and functional architecture in the cat's visual cortex,” *The Journal of Physiology*, vol. 160, no. 1, pp. 106–154, Jan. 1962, doi: 10.1113/jphysiol.1962.sp006837.

[39] A. Pouget, P. Dayan, and R. Zemel, “Information processing with population codes,” *Nature Reviews Neuroscience*, vol. 1, no. 2, pp. 125–132, Nov. 2000, doi: 10.1038/35039062.

[40] B. A. Olshausen and D. J. Field, “Emergence of simple-cell receptive field properties by learning a sparse code for natural images,” *Nature*, vol. 381, no. 6583, pp. 607–609, June 1996, doi: 10.1038/381607a0.

[41] D. Attwell and S. B. Laughlin, “An Energy Budget for Signaling in the Grey Matter of the Brain,” *Journal of Cerebral Blood Flow & Metabolism*, vol. 21, no. 10, pp. 1133–1145, Oct. 2001, doi: 10.1097/00004647-200110000-00001.

[42] J. C. Eccles, P. Fatt, and K. Koketsu, “Cholinergic and inhibitory synapses in a pathway from motor-axon collaterals to motoneurones,” *The Journal of Physiology*, vol. 126, no. 3, pp. 524–562, Dec. 1954, doi: 10.1113/jphysiol.1954.sp005226.

[43] S. J. Martin, P. D. Grimwood, and R. G. M. Morris, “Synaptic Plasticity and Memory: An Evaluation of the Hypothesis,” *Annual Review of Neuroscience*, vol. 23, no. 1, pp. 649–711, Mar. 2000, doi: 10.1146/annurev.neuro.23.1.649.

[44] E. Friedman, “Clock distribution networks in synchronous digital integrated circuits,” *Proceedings of the IEEE*, vol. 89, no. 5, pp. 665–692, May 2001, doi: 10.1109/5.929649.

[45] J. Lazzaro and J. Wawrzynek, “Low-Power Silicon Neurons, Axons and Synapses,” *Silicon Implementation of Pulse Coded Neural Networks*. Springer US, Boston, MA, pp. 153–164, 1994. doi: 10.1007/978-1-4615-2680-3_8.

[46] Q. Xia and J. J. Yang, “Memristive crossbar arrays for brain-inspired computing,” *Nature Materials*, vol. 18, no. 4, pp. 309–323, Apr. 2019, doi: 10.1038/s41563-019-0291-x.

[47] G. Indiveri *et al.*, “Neuromorphic Silicon Neuron Circuits,” *Frontiers in Neuroscience*, vol. 5, 2011, doi: 10.3389/fnins.2011.00073.

[48] W. Xie *et al.*, “Neuronal sequences in population bursts encode information in human cortex,” *Nature*, vol. 635, no. 8040, pp. 935–942, Nov. 2024, doi: 10.1038/s41586-024-08075-8.

[49] K. Roy, A. Jaiswal, and P. Panda, “Towards spike-based machine intelligence with neuromorphic computing,” *Nature*, vol. 575, no. 7784, pp. 607–617, Nov. 2019, doi: 10.1038/s41586-019-1677-2.

[50] J. Sommer, M. A. Özkan, O. Keszocze, and J. Teich, “Efficient Hardware Acceleration of Sparsely Active Convolutional Spiking Neural Networks,” *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 41, no. 11, pp. 3767–3778, Nov. 2022, doi: 10.1109/TCAD.2022.3197512.

[51] G. Haessig, A. Cassidy, R. Alvarez, R. Benosman, and G. Orchard, “Spiking Optical Flow for Event-based Sensors Using IBM's TrueNorth Neurosynaptic System.” Accessed: Oct. 27, 2025. [Online]. Available: http://arxiv.org/abs/1710.09820

[52] B. Rueckauer, I.-A. Lungu, Y. Hu, M. Pfeiffer, and S.-C. Liu, “Conversion of Continuous-Valued Deep Networks to Efficient Event-Driven Networks for Image Classification,” *Frontiers in Neuroscience*, vol. 11, p. 682, Dec. 2017, doi: 10.3389/fnins.2017.00682.

[53] P. U. Diehl and M. Cook, “Unsupervised learning of digit recognition using spike-timing-dependent plasticity,” *Frontiers in Computational Neuroscience*, vol. 9, Aug. 2015, doi: 10.3389/fncom.2015.00099.

[54] Y. Lecun, L. Bottou, Y. Bengio, and P. Haffner, “Gradient-based learning applied to document recognition,” *Proceedings of the IEEE*, vol. 86, no. 11, pp. 2278–2324, Nov. 1998, doi: 10.1109/5.726791.

[55] A. Delorme and S. Thorpe, “Face identification using one spike per neuron: resistance to image degradations,” *Neural Networks*, vol. 14, no. 6, pp. 795–803, July 2001, doi: 10.1016/S0893-6080(01)00049-1.

[56] D. P. Kingma and J. Ba, “Adam: A Method for Stochastic Optimization.” Accessed: Apr. 19, 2026. [Online]. Available: http://arxiv.org/abs/1412.6980